The Semantics and Dynamics of Retrieval-Augmented Systems

Optimization, Corpus Evolution, and Production Serving

Volume II of the architecture study of ragkit, ragopt, rag-ttc, GEC Chat, and the TTC Garden Assistant

August 2026

Table of Contents

Abstract

A retrieval-augmented generation system is not adequately described by the static expression “retrieve, then generate.” In production it is a long-lived, stateful, concurrent service whose source world changes while queries are executing; whose indexes are derived, verified, activated, drained, compacted, and occasionally rolled back; whose retrieval behavior depends on query-time policy and remote providers as well as index bytes; whose interaction may be direct search, retrieve-then-generate answering, or an agentic sequence of tool calls; and whose customer-visible state is reconstructed from snapshots and event streams. Optimization is therefore not tuning a few ranks against a frozen benchmark. It is controlled intervention in a dynamic system with multiple semantic layers and safety constraints.

This volume studies the actual RAG functionality present in `ragkit`, `ragopt`, `rag-ttc`, the GEC administrative chat, and the TTC Garden assistant. The reviewed snapshot contains 1,003 Go files and 1,580 Go test functions across the five scopes. `ragkit` provides deterministic chunking, representations, immutable lexical/vector bundles, retrieval, fusion, reranking contracts, context construction, grounded answer validation, and within-stage execution controls. RAG-TTC adds committed-Git source snapshots, complete-corpus builds, ANN bakeoffs, connected retrieval, model-invoked search, turn-scoped evidence ledgers, and a persistent chat runtime. GEC adds access scopes, source roles, lexical synonyms, an optional cross-encoder reranker, rank sweeps, administrative tool registration, and a snapshot-plus-WebSocket frontend. Garden adds intent-routed retrieval, structured product facts, evidence-bound widgets, and multi-turn calibration. `ragopt` supplies experiment custody, exact pairing, resumability, ordered gates, and promotion reports, but deliberately has no native model of RAG dynamics.

The central correction to the earlier kernel-centered architecture is to make the *evolving RAG service* the semantic object. The study gives both denotational and operational semantics. Denotationally, a release maps a subject, conversation state, and request to a distribution over outcomes and traces. Outcomes include answers, ranked evidence, abstentions, failures, cancellation, and presentation events. Traces retain intensional facts that final answer text erases: source and release lineage, authorization decisions, remote disclosure, fallback paths, latency, cost, freshness, and tool iterations. Operationally, the system is described by coupled labelled transition systems: an index-maintenance machine, a query/interaction machine, a release-activation machine, and a frontend projection machine. The optimization controller observes and intervenes in all four without owning their domain semantics.

The proposed production architecture introduces source revisions, changes, cursors, snapshot barriers, watermarks, impact plans, incremental derivation, backend capabilities, immutable RAG releases, compare-and-swap activation, reference-counted release leases, typed query interpreters, trace schemas, and replayable frontend events. Index maintenance is treated as incremental view maintenance. A clean full rebuild is the correctness oracle for an incremental build; a base-plus-delta overlay is the recommended first production design because it preserves immutable release semantics while enabling bounded freshness. Exactly-once execution is not assumed. Correctness is instead obtained from at-least-once source delivery, semantic idempotence, content-addressed derivation, checkpointed stages, and exactly-once *activation effect* through an idempotent compare-and-swap transition.

The optimization design covers the indexing and querying phases jointly. It defines a typed dependency graph over source admission, normalization, chunking, representations, embedding, lexical/vector indexes, query rewriting, routing, candidate depth, filtering, fusion, reranking, context admission, answer and agent policy, serving policy, and presentation. Interventions are classified as semantics-preserving operational changes, approximation changes, relevance changes, knowledge changes,

policy/security changes, and user-outcome changes. Evaluation proceeds through multiple fidelities: static laws, retrieval tests, repeated answer tests, session and frontend calibration, shadow traffic, and canary release. Promotion is constraint-first and Pareto-aware across relevance, grounding, answer quality, freshness, reliability, latency, cost, capacity, security, and user outcomes; it is not a universal weighted score.

The most urgent implementation findings are concrete. GEC currently applies access filtering after retrieval and optional reranking, so unauthorized hydrated text can cross a remote reranker boundary even though it is not returned to the user. GEC opens one bundle at startup while synonyms and reranking remain outside bundle identity. GEC and Garden lack a native atomic hot-activation boundary. RAG-TTC and Garden serve agentic retrieval whose semantics cannot be reduced to one `Query -> Answer` function. The GEC frontend buffers and sorts events around hydration, but its entity reducer does not explicitly reject stale entity versions, and append patches are not duplicate-idempotent. These are not incidental implementation details; they are violations or omissions in the runtime semantics that the shared RAG packages should make explicit.

The volume concludes with Go API blueprints, state-transition rules, proof obligations, testing and model-checking targets, a package architecture, product-specific migrations, and a staged implementation plan. The intended result is not a universal chatbot framework. It is a precise operational model of RAG from source revision to frontend projection, with enough structure to optimize the whole system without losing lineage, reproducibility, or safety.

Preface and reader's guide

This is the second volume of the architecture study. The first volume concentrated on compositional kernels, typed identity, immutable artifacts, evidence custody, and optimization-loop structure. Those results remain prerequisites, but they are not the subject here. This volume treats RAG as a field of computation in its own right.

The word *RAG* is used in industry for several materially different programs. One program returns ranked evidence. A second program retrieves evidence once and generates an answer. A third exposes retrieval as a tool to an agent that may search zero, one, or many times before it produces a final projection. A fourth mixes unstructured retrieval with structured facts and typed UI widgets. All four appear in the supplied code. They share retrieval primitives, but they have different state spaces, traces, completion rules, failure modes, evaluation units, and frontend contracts. The architecture must preserve those distinctions.

Readers implementing corpus refresh should begin with Chapters 7, 12, and 16 through 20. Readers optimizing relevance should read Chapters 8 through 11 and 21 through 26. Readers responsible for query serving should focus on Chapters 13 through 15 and 27 through 30. Product owners should read Chapters 2 through 6 and 32 through 35. The appendices provide formal transition rules, API sketches, parameter catalogs, and test obligations suitable for direct conversion into tickets.

The mathematical presentation is intentionally operational. Category theory, denotational semantics, probability kernels, labelled transition systems, stream functions, and incremental view maintenance are used because each answers a concrete engineering question. They are not used to rename ordinary functions. Whenever a formal structure does not improve an API, an invariant, an optimization plan, or a test, it is omitted.

This study is based on a supplied development snapshot rather than a complete, buildable monorepo. In particular, the GEC source imports `internal/knowledgebuild`, but that directory is absent from the extracted snapshot. The design documents and call sites describe it in detail, but its implementation could not be inspected directly. The repositories require Go 1.26.x while the analysis environment provides Go 1.23.2 without network access for toolchain download. Consequently, the empirical claims in this volume are based on static source, tests, manifests, and design records; the current snapshot was not compiled or executed. These limitations are stated again in Appendix G.

Principal claims

1. The correct semantic object is a long-lived evolving RAG service, not an immutable index bundle and not an optimization run.
2. RAG behavior has at least three semantic layers: extensional outcomes, intensional traces, and operational transitions. Optimization must preserve or intentionally change each layer under explicit constraints.
3. Corpus maintenance and query serving are coupled machines. Release activation is the synchronization protocol between them.
4. Direct retrieval, retrieve-then-generate answering, and agentic retrieval are separate interpreters over a shared retrieval algebra.
5. A production RAG release must identify all behaviorally material inputs: corpus snapshot, derived indexes, retrieval policy, reranker, synonyms and rewrites, structured stores, prompts, contracts, provider identities, and presentation policy.
6. A query or turn must be pinned to one release. Evidence, context, citations, structured facts, answer validation, and frontend provenance must not mix release epochs.
7. Authorization must constrain candidate text before any remote reranker, generator, or connected-retrieval provider receives it.
8. Corpus refresh should begin with immutable base releases plus delta overlays and periodic compaction. Incremental output must be observationally equivalent to a clean full rebuild at the same source barrier, subject to declared approximate-backend tolerances.
9. Exactly-once execution is the wrong reliability target. Use at-least-once inputs, idempotent semantic stages, durable checkpoints, and compare-and-swap activation.
10. Retrieval optimization is a constrained intervention problem over a dependency graph, not a flat parameter sweep. Indexing and querying parameters interact and must be evaluated at the correct behavioral level.
11. Frontend projection is part of RAG semantics whenever evidence, citations, choices, or widgets affect the user outcome. Snapshot-plus-suffix replay requires versioning, deduplication, stale-update rejection, and explicit patch laws.
12. `ragkit` should own shared RAG-domain semantics and runtime contracts. `ragopt` should orchestrate controlled trials over those contracts without owning source, index, query, activation, or presentation meaning.

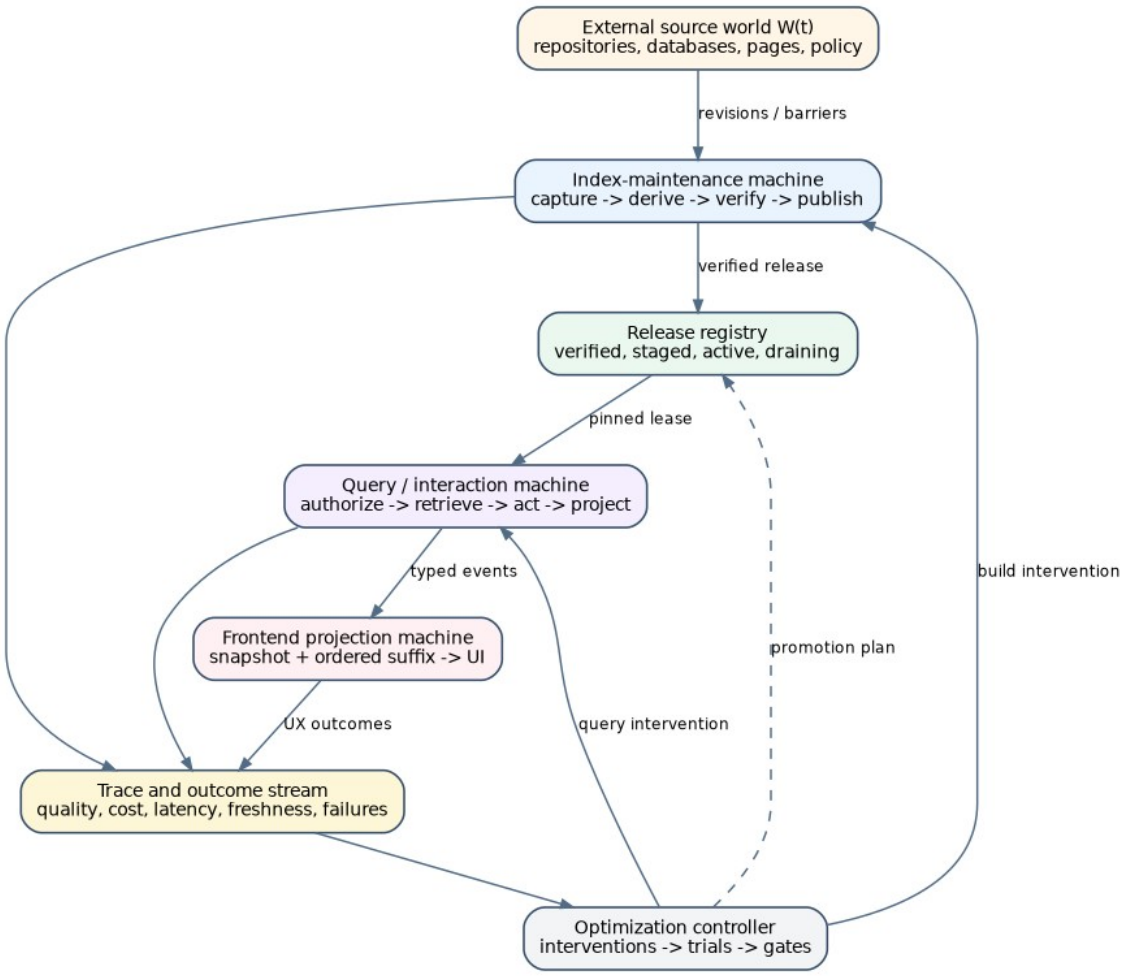
Notation and semantic conventions

Let W_t denote the external source world at time t. A source capture at barrier τ produces a finite logical snapshot S_τ. A build specification is b, a query specification is q, and a behavior-complete release is $R=(S_\tau, b, q, a)$ where a denotes auxiliary material such as prompts, rerankers, structured fact stores, contracts, and presentation policy. A subject and authorization context is u; a conversation state is c; and a request is x.

The notation $X+Y$ denotes a disjoint sum. $D(X)$ denotes a probability distribution over X; in an implementation it may be represented only by samples and retained provider transcripts. $Trace$ is a finite sequence of typed events. A deterministic relation $z \xrightarrow{\ell} z'$ is one small operational step from configuration z to z' with label ℓ. Its reflexive transitive closure is $\xrightarrow{\square}$. The symbol Δ denotes a change or differential. A signed multiset is used when insertions and deletions must compose algebraically.

A *release* is not merely an index directory. It is the immutable root of everything required to interpret a query under one declared behavior. A *deployment* is a running process or replica set that can serve one or more releases. An *activation* changes which release new leases resolve to. A *lease* pins an in-flight query, turn, or session to a release and prevents premature retirement.

A *trace-equivalence* relation may be exact or observational. Exact trace equivalence requires the same typed events and material values. Observational equivalence projects away declared operational variation, such as worker scheduling, while retaining user outcomes and protected facts such as authorization, evidence lineage, and fallback class. Every optimization candidate must state which equivalence or improvement relation it claims.



The RAG domain consists of coupled maintenance, serving, projection, and optimization machines.

Part I. The implemented RAG domain

1. Research problem, scope, and method

1.1 Why the object of study must change

The earlier architectural work asked what small common kernel could support indexing, querying, and optimization. That question was useful but incomplete. It starts from reusable mechanisms and moves outward. The present question starts from the domain itself: what is a RAG system over time, what does it mean, how does it execute, what can change, and what counts as a correct optimization?

A static pipeline description hides the decisive facts. A source revision may arrive while a query is running. An index may be partially built but not eligible for activation. A reranker may fail after lexical and vector retrieval succeeded. A generator may emit tokens before its final citation set is known. A frontend may hydrate a snapshot while newer events are already buffered. An agent may call search twice, reuse one evidence item, query a structured database, and then present a typed product comparison. A canary release may improve nDCG but disclose unauthorized text to a remote provider or violate a freshness objective. These behaviors are the field.

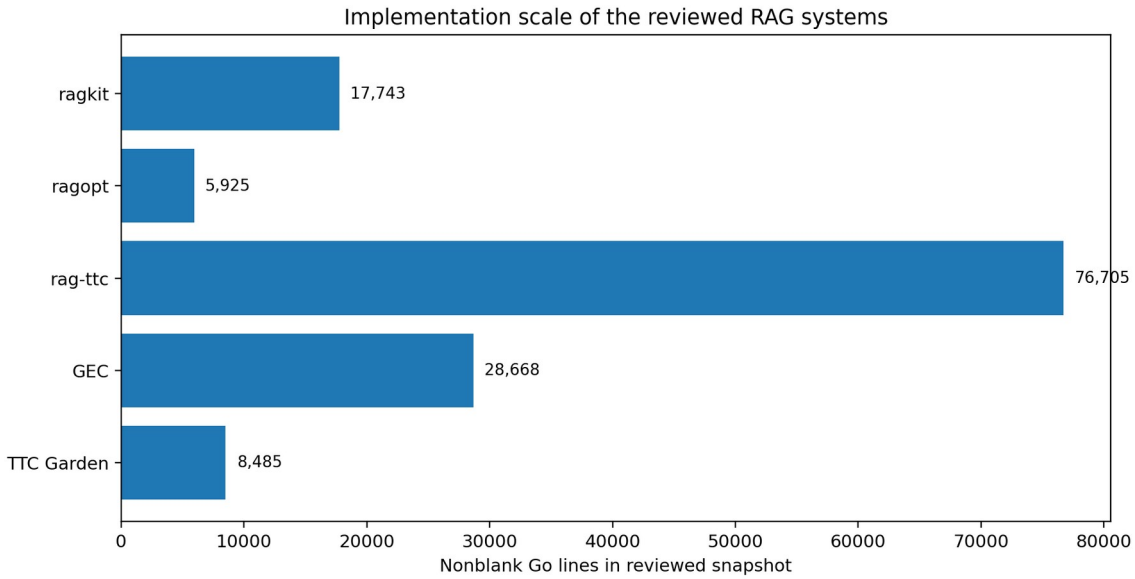
The central research questions are therefore:

1. What denotation should be assigned to an evolving RAG release and service?
2. What operational machines explain build, activation, query, agent, and frontend behavior?
3. Which state and transition boundaries should become shared APIs?
4. How should corpus changes be represented and incrementally maintained?
5. How can indexing and querying be optimized jointly without invalid comparisons?
6. Which properties can be tested, model-checked, or proved over small kernels?
7. How should the current applied systems migrate while preserving useful product semantics?

1.2 Reviewed systems

The review covers five scopes. `ragkit` is the reusable RAG package. `ragopt` is the reusable experiment and optimization-loop package. `rag-ttc` is the largest applied and experimental RAG system and contains both a copied RAG substrate and product-specific runtime functionality. GEC is an administrative chat with RAG and structured-data tools. The TTC design-system repository contains the Garden assistant and its evidence-aware product presentation.

The reviewed snapshot contains 176 files in `ragkit`, 120 in `ragopt`, 1,302 in `rag-ttc`, 1,114 in GEC, and 940 in the TTC design-system scope. The corresponding nonblank Go line counts are approximately 17,743; 5,925; 76,705; 28,668; and 8,485. These measurements are useful only as scale indicators. The architectural conclusions come from the semantics expressed in types, state transitions, tests, and call graphs, not from line counts.



Implementation scale of the reviewed systems.

1.3 Method

The analysis followed the runtime path in both directions. On the indexing side it began at source capture and corpus loading, followed normalization, chunking, representation generation, embedding, index construction, manifest verification, and bundle opening, then inspected the build and experiment commands that compose those stages. On the query side it began with frontend and chat submission, followed runtime resolution, tool registration, retrieval, ranking, evidence admission, generation, validation, event emission, and frontend reduction.

Static API analysis was supplemented by test analysis and design-record analysis. Tests reveal intended laws that comments may not state: deterministic tie-breaking, context admission boundaries, evidence-ledger capacity, session cancellation, snapshot hydration, and ANN reproducibility. Design records reveal intended operational contracts not yet present in the checked-in source: nightly refresh, resumable builds, source watermarks, and activation workflows. The volume distinguishes implemented behavior from intended design.

The optimization review traced what each harness holds fixed, what it mutates, what unit it evaluates, what failures enter the denominator, and what decision rule it applies. This is necessary because two loops can both be called “optimization” while answering different causal questions. A fusion sweep over frozen channel rankings is not an index optimization. An ANN bakeoff against an exact oracle is not an answer-quality experiment. A multi-turn calibration is not a paired retrieval benchmark. A production canary is not a substitute for any of them.

1.4 Strength of claims

Claims about source-visible ordering and identity are strong. For example, GEC’s `Search` calls `retrieve` and only then calls `filterHits`; `retrieve` may call `rerank`, which hydrates candidate chunks before invoking the reranker. The security consequence follows from the order of operations. Claims about missing behavior are also strong when no corresponding type, state, or code path exists in the supplied snapshot.

Claims about the absent GEC `internal/knowledgebuild` package are weaker. Its behavior is reconstructed from imports, command call sites, tests that reference its constants, and detailed ticket diaries. The

missing source prevents direct confirmation of implementation details. Claims about runtime performance are not made because the code could not be built in the analysis environment and because supplied benchmark artifacts are workload-specific.

2. What “RAG” means in the current code

2.1 A family of interpreters, not one pipeline

The code contains at least four operational meanings of RAG.

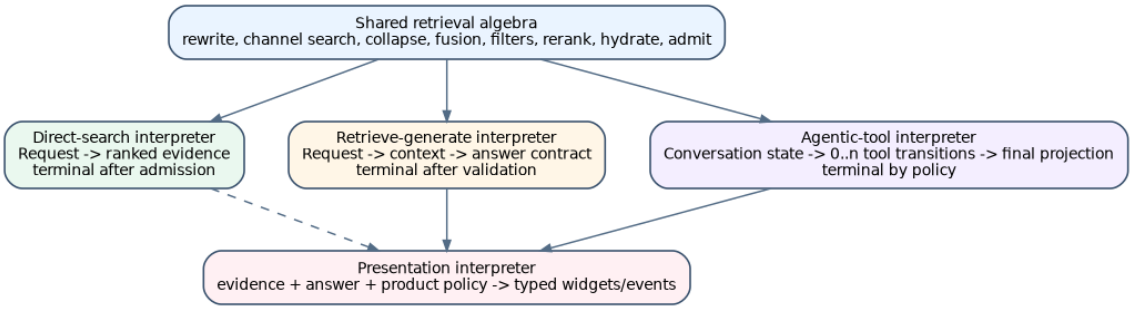
The first is **direct ranked retrieval**. A request contains a query and perhaps a route, filters, and a limit. The result is an ordered list of hydrated chunks or structured facts. GEC’s `knowledge.Service.Search`, RAG-TTC workspace search, and the TTC search tool’s internal channel execution are examples. Completion occurs when the ranked evidence list is produced.

The second is **retrieve-then-generate answering**. `ragkit/rag/answering` performs channel retrieval, fusion, optional reranking, context construction, provider generation, and grounded-contract validation. Completion occurs when a valid answer or safe abstention is produced. The retrieval trace and the generation trace are one operation, but their failures are distinct.

The third is **agentic retrieval**. RAG-TTC’s `ttcrag.SearchTool` is registered as a model-callable tool. A turn may make no search call, one call, or multiple calls under different routes. Evidence is accumulated in a turn-scoped ledger and exposed through stable labels. Completion is decided by the agent policy, not by the search function. The semantic input is conversation state; the semantic result is a trajectory and final projection.

The fourth is **retrieval plus typed product presentation**. Garden may satisfy a product-fact intent through structured data before unstructured retrieval, augment retrieved evidence with exact product facts, and expose only evidence-admitted material to grounded widget tools. The final user result is not only answer text. It includes choices, source cards, product comparisons, step widgets, and developer lineage. Presentation is an interpreter from evidence and product policy to UI events.

These modes should share data types for queries, candidates, contributions, evidence, release lineage, and traces. They should not share one overloaded service method whose optional fields attempt to encode every mode. Separate interpreters make terminal conditions and failure behavior explicit.



Three query interpreters share a retrieval algebra but have different completion semantics.

2.2 The retrieval algebra already present

Across the systems, a common algebra is visible:

$$retrieve = admit \circ hydrate \circ rerank \circ filter \circ fuse \circ collapse \circ channels \circ rewrite.$$

Not every route uses every operator. A lexical-only route omits vector search and fusion. A structured-fact route may return authoritative facts without chunk retrieval. A no-rerank route omits the cross-encoder. A representation-kind route wraps a searcher and filters searchable derivatives. A connected-

retrieval route may augment or replace local candidates. Nevertheless, the operations and their ordering are stable enough to form a shared vocabulary.

Ordering is semantically important. Filtering before remote reranking is not equivalent to filtering after it, even when the returned top-k list happens to match, because remote disclosure differs. Collapsing representations before fusion is not generally equivalent to fusing representations and then collapsing, because multiple representations from one chunk can occupy channel ranks. Applying synonyms only to lexical search, as GEC does, is not equivalent to rewriting the query globally. Hydrating before a budget check may cause unnecessary I/O or disclosure. The algebra must therefore retain stage order rather than exposing an unordered bag of plugins.

2.3 Evidence is stateful in the applied systems

In `ragkit/answering`, evidence is a bounded ordered context for one answer operation. In GEC, a per-run evidence ledger assigns labels such as `E1` and advertises its scope in tool output. In RAG-TTC, the search tool tracks already-seen chunks across multiple calls in one turn. In Garden, a per-conversation search session retains citations and structured facts so later widget tools can prove that their payload was grounded in admitted evidence.

These are not identical ledgers. Their lifetime differs: operation, run, turn, or conversation. Their element types differ: chunks, structured facts, or presentation groups. Their capacity rules differ. The shared abstraction should therefore be an explicit *evidence session* parameterized by scope and evidence kind, not a global singleton or one fixed ledger implementation.

A critical invariant is that an evidence session belongs to one release lease. Reusing a conversation-scoped ledger after activating a new release can silently mix source revisions unless the product explicitly creates a new evidence epoch. The current code does not yet make this release relationship a type-level contract because it does not have behavior-complete release leases.

2.4 Structured facts and connected retrieval

Garden and RAG-TTC show why RAG cannot be reduced to vector search over text. Some product questions are better answered from a structured fact database. Some routes invoke connected retrieval. Some final answers combine exact structured fields with explanatory source chunks. The common semantic category is not “text passage” but *typed evidence with provenance and disclosure policy*.

This does not justify collapsing SQL tools, search tools, and product databases into a universal evidence engine. Their authority and freshness differ. A structured store may be live and transactionally current while an index is a captured snapshot. A connected source may have weaker reproducibility and stronger disclosure risks. Shared types should express evidence kind, source epoch, query provenance, and policy. Product code should retain the meaning of fields and the rules for joining them.

3. Implemented indexing functionality

3.1 `ragkit`: a deterministic full-build substrate

`ragkit` defines documents, chunks, representations, embeddings, search interfaces, and immutable index bundles. A document has a stable ID, source URI, title, text, content digest, and metadata. A chunk refers to one document and an exact half-open byte range. Chunk identity includes the document, chunker, range, and text digest. A representation is searchable derived material linked to a source chunk; it is not evidence.

The chunking package supplies fixed-size, Markdown-aware, and Markdown-heading-aware policies. These transformations are deterministic and document-local. Document locality matters for future incremental maintenance: changing one document need not invalidate chunk identities for other documents. It may still invalidate many chunks inside the changed document, particularly when an insertion shifts byte ranges or heading structure.

The flow and embedding packages provide content-addressed cache keys, bounded parallel maps, retries, fail-fast or quarantine policies, rate limits, and budget admission. These mechanisms are substantial. They determine how expensive provider operations execute and resume at the *stage* level. They do not yet constitute a durable production build machine: there is no source cursor, build lease, stage checkpoint manifest, activation state, or cross-process resume protocol.

`ragkit/rag/indexbundle` builds an immutable artifact containing chunk and representation data, a Bleve lexical index, an exact SQLite vector index when configured, and a manifest. Publication uses a temporary location followed by synchronization and rename. Opening verifies schema, counts, backend manifests, corpus lineage, and query-embedding identity. Verified source documents are loaded only when the corpus path remains within the expected root and its digest matches the manifest.

This is a strong **sealed snapshot** abstraction. It answers: given a finite corpus and complete derived material, what exact searchable bundle was built, and can it be opened safely? It does not answer: what changed in the source world, which work is affected, how is a failed build resumed, which release is active, or how does an old release drain.

3.2 RAG-TTC complete-corpus builds

The RAG-TTC index build command is the most pragmatic composition of the shared substrate. It loads a full corpus JSON file, chooses a chunker, creates raw and optionally generated representations, embeds every representation through a cache, and invokes the immutable bundle builder. Generated representation kinds include summaries, contextual forms, questions, and entities. The dry-run path estimates counts and cost.

The provider caches mean that a logical full rebuild does not necessarily repeat all expensive work. Identical representation inputs can reuse generated text and embeddings. This is valuable and should be retained. However, cache reuse is not incremental *state maintenance*. The command still computes from a complete input corpus and produces a complete output bundle. It has no explicit document upsert/delete protocol, source watermark, or delta index.

RAG-TTC workspace indexing contributes a stronger source capture model. `gochunk.LoadCommitted` captures Git `HEAD`, reads tracked files from the committed tree rather than the mutable working directory, applies admission policy, and computes a deterministic snapshot digest from repository state, policy, and

document digests. The workspace command persists snapshot, admissions, diagnostics, chunk records, representations, and a build record. This is close to a proper `Snapshotter` interface and should inform `ragkit/corpus`.

The limitation is again dynamic: each workspace build is a new full committed snapshot. There is no reconciliation from the prior snapshot, no delete propagation contract, and no activated release registry. The source semantics are stronger than the maintenance semantics.

3.3 Index backend experiments

The current vector bundle uses exact SQLite search. RAG-TTC also contains an in-memory HNSW candidate and an ANN bakeoff command. The bakeoff treats exact search as an oracle, sweeps `efSearch`, measures recall and p95 search latency, and requires ranking reproducibility across a rebuild before choosing a candidate. This is a good example of an approximation-changing intervention with an explicit gate.

Its scope is deliberately narrow. It evaluates a fixed query workload and does not include update throughput, deletion behavior, memory residency, compaction, build duration, crash recovery, multi-tenant isolation, or freshness under a delta overlay. These are not defects in the experiment; they are additional dimensions required before the backend can be declared production-equivalent.

An index backend contract should therefore expose capabilities rather than only `Search`. Relevant capabilities include full build, deterministic bulk load, point upsert, point delete, snapshot reads, atomic checkpoint, compaction, exact-score availability, filter pushdown, and memory/disk reporting. Optimization can then reject candidates that cannot implement required semantics before spending quality-evaluation budget.

3.4 What is absent from the shared indexing model

The shared code has no first-class source revision. `rag.Document` describes a current logical document, not a revision with observed and effective time. There is no tombstone type, source cursor, snapshot barrier, or watermark. A corpus digest identifies one finite serialization, but it does not describe the relationship between successive corpus states.

There is also no build intent or durable build status. A filesystem bundle is either successfully returned or the call errors. Flow-level caches can preserve completed expensive items, but an operator cannot ask a shared service which source barrier is being built, which stage is blocked, which items are quarantined, whether a build can resume under the same semantic identity, or which release supersedes it.

Finally, index bytes do not identify complete query behavior. Lexical field boosts are embedded in build behavior, but reranker, synonyms, route configuration, prompts, fact databases, and presentation policies live elsewhere. The production unit must be larger than an index bundle.

4. Implemented query functionality

4.1 `ragkit/answering`

The answering service exposes strategies for BM25, vector, reciprocal-rank fusion, reranked fusion, multi-query retrieval, and HyDE-style query generation. A request carries IDs, query text, retrieval query, and retrieval configuration. A result carries per-channel hits, fallback and error observations, fused rankings, admitted evidence, timing, and a trace.

The implementation executes lexical and vector retrieval sequentially. Multi-query and HyDE generation can degrade to the original query when their provider fails, recording the failure. A generic reranker error can fail the request, whereas GEC's product-specific reranker fails open to fused order. This difference is semantically material and should be a declared `FallbackPolicy`, not an accidental divergence between applications.

Context construction admits complete chunks in ranked order under evidence-count and rune limits; it does not truncate chunks. This gives a simple prefix property: increasing the budget cannot reorder already-admitted chunks, though it can admit additional chunks. The grounded answer contract validates that cited chunk IDs came from supplied evidence and converts unsupported output into a safe abstention.

The service already emits stage observations. Those observations should be promoted from diagnostics to the intensional denotation of the request. An answer that used the original query after HyDE failed is not trace-equivalent to an answer produced by the intended route, even when the final text matches.

4.2 GEC retrieval

GEC opens one verified bundle and source corpus into an immutable `knowledge.Service` at process startup. It reconstructs a query embedder from the bundle manifest when a vector channel exists. The service is shared across sessions; run-scoped evidence lives in the tool wrapper.

The query contract adds server-owned access scopes, source roles, and route controls. Lexical query expansion uses curated synonyms; the vector and reranker queries remain raw. Hybrid retrieval fuses lexical and vector rankings with weighted RRF. An optional reranker hydrates a candidate pool, prefixes titles, invokes the cross-encoder, and blends reranker rank with fused rank. Provider failure logs a warning and returns the fused order.

The code explicitly states that reranking and synonyms are serving configuration rather than bundle identity. This is operationally convenient but semantically incomplete. Two processes can advertise the same bundle ID and return different rankings because environment-loaded synonyms or reranker configuration differ. A query trace can partially explain the difference, but release identity cannot.

The more serious issue is stage order. `Search` calls `retrieve` with an overfetch depth and applies `filterHits` afterward. `retrieve` may rerank before filtering; reranking hydrates chunk text and may call a remote provider. Thus unauthorized chunks can be sent to a remote reranker even though they are removed before the result is returned. "Never returned" is not the same security property as "never disclosed." Authorization must be applied before hydration for any remote stage, ideally through backend filter pushdown or a local authorized candidate set.

Post-ranking filtering also creates relevance starvation. The implementation overfetches by a fixed factor of eight and notes that this is adequate for the current small scope set. The general semantic contract

should be stronger: the returned list must be the top-k ranking *within the authorized subcorpus*, not the filtered prefix of an unauthorized global ranking. Backend prefiltering or per-scope indexes can implement that contract.

4.3 RAG-TTC search as an agent tool

`ttcrag.SearchTool` contains lexical and vector searchers, source and chunk catalogs, route definitions, configuration, and a turn-scoped evidence ledger. A route can alter representation-kind or source-role searchers, channel enablement, candidate depths, RRF constant, and connected augmentation. Each call returns citations, ranks, contributions, and an `AlreadySeen` marker.

This is not merely a search API with additional metadata. The ledger changes the meaning of later calls. A repeated chunk can retain its citation label while being marked already seen. Capacity limits can prevent new evidence from entering the turn. A structured route can answer an authoritative product-fact query without source chunks. The transition system must include ledger state and route observations.

The direct service path and the model tool path should use the same retrieval plan types and ranking kernels. They should not be forced into the same method. The tool interpreter needs operation names, model-visible schemas, iteration limits, tool-result serialization, and conversation-state transitions that direct search does not.

4.4 Garden's product interpreter

Garden opens a fixed RAG index bundle, product fact database, tool configuration, and embedding provider at startup. It creates fresh session search state and tool registries per conversation. Intent classification selects routes that change representation filters, source roles, channel depths, RRF, and connected augmentation. Structured product facts can satisfy some intents before local retrieval; exact facts can also augment retrieved evidence.

Grounded widget tools are particularly important. They do not accept arbitrary model-provided product facts. They project from evidence admitted by the exact session search instance, group chunks by document, align structured fields, retain field-level provenance, and suppress conflicting extracted facts. Customer and developer modes expose different projections.

This makes frontend presentation part of the semantic result. A retrieval candidate can improve answer text yet degrade the widget by failing to supply aligned product fields. Conversely, a structured fact route can produce a better customer outcome without improving text-retrieval metrics. Optimization must include the presentation interpreter when the product uses it.

5. Implemented production runtime

5.1 Startup-bound resources

GEC and Garden both resolve RAG resources at process startup. GEC opens one bundle and then applies environment-configured synonyms and reranking. Garden opens the index, fact database, tool configuration, and provider, then creates per-conversation tool state. RAG-TTC's simple serve command similarly opens a fixed bundle. This is operationally straightforward and safe for immutable objects, but activation requires a process restart and there is no shared draining or rollback protocol.

A restart is not a semantic activation mechanism. It can mix old and new replicas behind a load balancer, interrupt in-flight sessions, and leave release identity implicit in deployment configuration. A production RAG runtime needs a release manager that can acquire immutable handles, atomically change the active head, preserve old handles for in-flight work, and expose exact release IDs in traces and frontend provenance.

5.2 Persistent chat and submission state

RAG-TTC and the GEC/Garden chat stacks are more mature than a simple HTTP question-answer endpoint. They maintain conversations, turns, timelines, submissions, idempotency keys, cancellation, authorization, and persistent observations. Runtime composition occurs per conversation or request. Tool calls and results are events in a durable interaction, not transient local function calls.

This state determines RAG behavior. An agent policy sees prior messages and tool results. A turn may be cancelled after retrieval but before generation. A duplicate submission should not create duplicate turns. A runtime failure to persist observations can be fatal even when the model returned text, because the transcript is part of the operational contract. Shared RAG APIs must fit into this stateful host rather than replacing it.

5.3 Frontend hydration and live events

GEC's WebSocket manager receives a hello frame, subscribes with a prior snapshot ordinal, optionally hydrates a snapshot, buffers live UI events that arrive before hydration completes, sorts the buffer, and then appends the suffix. Events and snapshots are mapped into a common timeline entity model. The Redux reducer upserts entities, merges sparse terminal data, and supports append and replace stream patches.

This is close to a formal snapshot-plus-suffix protocol, but the reducer laws are underspecified. `updatedOrdinal` is set to the maximum of old and incoming values, yet incoming fields are merged even when the incoming entity is stale. A stale update can therefore overwrite newer content while retaining the newer ordinal. Duplicate append patches append twice unless transport or upstream storage deduplicates them. The current transport may make these cases rare, but the reducer itself is neither stale-safe nor duplicate-idempotent.

A production event envelope should contain an event ID, stream ID, entity ID, entity version, global or stream ordinal, operation, patch mode, causation ID, and release/turn lineage. The reducer should reject lower entity versions, deduplicate event IDs, and define exactly-once semantics for append patches or replace them with versioned full values. These requirements become especially important when answer streaming is exposed through reconnecting mobile or browser clients.

5.4 Runtime identity

Garden records a broader runtime identity than GEC: profile registry, prompt source, RAG index path, fact database path and digest, augmentation behavior, and tool configuration path. This is useful but remains partly path-based and not one immutable release root. GEC's runtime fingerprint comes mainly from an inference profile and omits complete RAG behavior.

A behavior-complete release must be material rather than locational. A path may be overwritten; an environment variable may change; a provider alias may resolve to a new model. The release manifest should record content digests or immutable provider version identities and should itself be content-addressed. A deployment can then state exactly which release each turn used.

6. Implemented optimization and its boundary

6.1 `ragopt` is experiment custody

`ragopt` has a clear domain-neutral role. Candidates are immutable exact-one-mutation snapshots. Evaluation schedules incumbent and challenger cells at the same case and repeat coordinates. The run store copies inputs, appends durable cell results, supports resume, and records terminal status. Comparison requires exact pairs and represents missing metrics explicitly. Gates are ordered predicates. Reports make promotion evidence reviewable and do not directly apply production changes.

This is the right substrate for reproducible intervention. It intentionally does not know what a chunk, RRF constant, release lease, source watermark, citation, or frontend widget means. The current package therefore cannot by itself answer the RAG-specific questions in this volume. It needs a domain adapter built on shared RAG semantics.

The control boundary should remain: `ragopt` may request a build, execute a query interpreter, consume native metrics, and propose activation. It should not define source connectors, incremental index semantics, authorization, evidence admission, or release activation rules. Putting production refresh entirely inside `ragopt` would make the optimizer the owner of ordinary product operation, creating an opaque semantic boundary.

6.2 Current retrieval sweeps

GEC's RRF sweep evaluates combinations of rank constant and vector weight. It obtains channel rankings once and re-fuses them in memory for each cell. This is efficient and causally clean for the parameters under study. The sweep chooses by hit rate, then MRR, then deterministic parameter tie-breakers.

The sweep does not optimize indexing. Chunking, representations, embeddings, lexical analysis, and vector backend are frozen. It does not evaluate reranking or answer behavior. It uses one fixed evaluation set and lacks uncertainty estimates. This is a useful *fusion subproblem*, not a general retrieval optimizer.

RAG-TTC's ANN bakeoff is another well-formed subproblem. It compares approximate rankings with an exact oracle and gates recall and p95 latency. It says little about end-to-end answer quality, and it deliberately excludes embedding latency from vector search timing. Again, the correct response is not to criticize it for limited scope but to place it in a larger multi-fidelity field.

6.3 Answer and session evaluation

`ragkit` provides standard retrieval metrics such as precision, recall, hit rate, MRR, and nDCG and rejects mixed target granularity. GEC adds answer-quality judging and detailed strata. Garden's calibration runner drives multi-turn cases against the real chat server, waits for a stable terminal answer, and asserts answers, choices, word count, and source kinds. RAG-TTC contains tool-loop and answer-quality experiments with frozen arm contracts.

These evaluations occur at different semantic levels. Retrieval metrics are cheap and diagnostic but cannot prove answer faithfulness. Judge-based answer metrics can detect some failures but are stochastic and can share model biases. Session calibration exercises routing, tool loops, persistence, and presentation but is expensive and often weakly paired. Production telemetry measures latency and failure under real load but usually lacks counterfactual relevance labels.

The architecture should preserve all levels and specify how evidence moves between them. A candidate that fails a static authorization law should never enter answer evaluation. A candidate that cannot improve retrieval on a paired holdout should usually not consume session-calibration budget. A candidate that passes offline tests must still pass shadow and canary operational gates.

6.4 The missing RAG optimization field

A complete field needs four things the current packages do not jointly provide:

- a typed parameter and intervention model covering both build and query behavior;
- a dependency graph that determines which artifacts and evaluations are invalidated;
- a behavior-complete release that can be evaluated and activated as one unit;
- a multi-objective, multi-fidelity decision model that includes freshness, security, reliability, capacity, and user outcome.

This volume supplies those abstractions. `ragopt` remains the campaign and custody engine. `ragkit` supplies the RAG-domain model, build/query interpreters, release semantics, and native evaluation contracts.

Part II. Denotational and operational semantics

7. The evolving RAG service as the semantic object

7.1 State across time

A production RAG service is a tuple of evolving states rather than one function:

$$\Sigma_t = (W_t, C_t, A_t, Q_t, F_t, O_t).$$

Here W_t is the external source world, C_t is captured source state and cursors, A_t is the artifact and release registry, Q_t is in-flight query and conversation state, F_t is frontend projection state, and O_t is retained observations and experiment evidence. Different components advance these coordinates on different clocks. Source capture can lag the source world. An artifact can be verified but inactive. A conversation can hold a lease on an older active epoch. A frontend can be at event ordinal n while the server has committed $n+k$.

This asynchronous state is the reason a single pure function is insufficient as the top-level model. Pure functions remain the correct model for many stages: normalization, deterministic chunking, exact fusion, context admission, contract validation, and event reduction. The service is the composition of these pure kernels with state, concurrency, partial failure, and external probability.

7.2 Four coupled machines

The architecture distinguishes four machines.

The **maintenance machine** captures source state and derives verified releases. Its unit of progress is a source barrier and build stage. Its terminal success is a published release; activation is separate.

The **query machine** acquires a release, applies authorization, runs one of the query interpreters, and emits a terminal outcome plus trace. Its unit of progress is a typed stage or agent transition.

The **activation machine** maintains a mapping from product scope to the release that new work should acquire. Its unit of progress is a compare-and-swap state transition. It also tracks draining leases.

The **projection machine** combines a snapshot and event suffix into frontend state. Its unit of progress is one versioned event.

The optimization controller is deliberately not a fifth semantic authority. It observes the four machines, constructs candidates, and requests their native operations. It may propose a new activation, but the release registry enforces activation rules.

7.3 A categorical reading

Let $\mathbf{RagDet}$ be a category whose objects are typed immutable values and whose morphisms are total deterministic stages that either return a value or an explicit typed rejection. Sequential composition is ordinary function composition. Deterministic independent channel execution forms a symmetric monoidal product when both results are retained:

$$(f \otimes g)(x, y) = (f(x), g(y)).$$

This model is useful for normalization, chunking, representation projection from retained provider outputs, exact indexing, ranking, context admission, validation, and frontend reduction. It makes associativity and identity laws directly testable.

Provider calls and timeouts are not morphisms in this deterministic category. They can be represented in the Kleisli category of a probability-and-trace effect, or more concretely as Markov kernels:

$$K : X \rightsquigarrow Y \times Trace.$$

Composition integrates over intermediate distributions while concatenating traces. Markov-category language is useful here because independent provider effects can be composed without pretending they are deterministic. In implementation, the system need not expose abstract categorical types. It needs explicit randomness, transcripts, and trace identities so that the same distinctions are preserved.

Index maintenance adds time and differences. Source states form a stream, and deterministic derivations can often be lifted to incremental transformations over changes. Signed multisets provide addition and subtraction; integration reconstructs current state from the change stream. This is the engineering connection to incremental view maintenance and DBSP: the semantics of a maintained index can be stated as the integral of derived changes, while correctness is equality with applying the original transformation to the integrated source state.

7.4 Interpreters over a free retrieval signature

A practical way to express shared query behavior is an operation signature rather than one concrete pipeline:

- `Rewrite(query, policy)`
- `Search(channel, query, filters, k)`
- `Collapse(hits, policy)`
- `Fuse(rankings, policy)`
- `Authorize(candidates, subject, policy)`
- `Rerank(query, candidates, policy)`
- `Hydrate(ids, release)`
- `Admit(evidence, budget, policy)`
- `Generate(context, contract, provider)`
- `Emit(event)`

A typed plan built from these operations is a syntax tree. A direct-search interpreter executes only the retrieval prefix. An answering interpreter extends it with context, generation, and validation. An agent interpreter exposes selected operations as model tools and repeatedly interprets tool requests. A simulator can interpret the same plan against retained results. A static analyzer can inspect it for remote disclosure, missing authorization, unbounded loops, or incompatible release use.

This is a stronger abstraction than a list of middleware. A middleware list usually cannot state which future operation consumes hydrated text or whether authorization dominates every remote disclosure path. A typed operation graph can.

7.5 Refinement rather than universal equivalence

No useful production implementation will reproduce every trace bit for bit. Parallel channel scheduling, cache hits, and network latency vary. Correctness is therefore a refinement relation parameterized by protected observations.

Let π_P project a trace to a protected property set P, such as final ranked chunk IDs, release ID, authorization decisions, remote disclosure IDs, evidence lineage, and outcome class. Implementation I refines specification S when every implementation trace has a specification trace with the same protected projection:

$$I \sqsubseteq_P S \Leftrightarrow \forall t \in Traces(I), \exists s \in Traces(S) : \pi_P(t) = \pi_P(s).$$

An optimization candidate must declare P. Increasing workers may claim to preserve ranked output, disclosure, and failure class while improving latency. Switching exact search to HNSW cannot claim ranked-output preservation; it claims bounded recall loss and improved resource use. Changing chunking is knowledge- and relevance-changing and requires broader evaluation. This declaration prevents accidental semantic drift from being labeled operational tuning.

8. Denotational semantics of corpus capture and indexing

8.1 Source revisions

A source item is identified by a stable logical key k. A revision is:

$$r = (k, v, t_o, t_e, d, m, p),$$

where v is a source revision identifier, t_o is the time observed by the connector, t_e is the source's effective time when known, d is a content digest, m is metadata, and p is the payload or immutable payload reference. The distinction between observed and effective time matters for late events and backfills.

A logical source snapshot is a finite partial map:

$$S : SourceKey \rightharpoonup Revision.$$

A change is either `Upsert(r)`, `Delete(k, tombstone)`, or `Barrier(token, watermark)`. Connectors may deliver changes at least once and out of order within declared bounds. The normalizer resolves source semantics into a deterministic current map. A source-specific ordering rule, not wall-clock arrival alone, decides which revision wins.

The snapshot barrier is essential. Without it, "build the current corpus" has no stable meaning while the source is changing. A connector that supports repeatable snapshots can return a token identifying a consistent read. A change stream can emit a barrier meaning all changes through cursor c have been delivered. The release records this token and watermark.

8.2 Normalized document state

Normalization maps source revisions to zero or more logical documents:

$$N : Revision \to FiniteMap(DocumentID, Document).$$

The result can be zero documents when an item is excluded or cannot be safely normalized. It can be multiple documents when one source item yields independently retrievable logical sections. The mapping must be deterministic under a versioned normalization specification and must retain source-revision lineage.

Current `rag.Document` can remain the retrieval-level document type, but production capture needs an envelope containing source key, source revision, observed/effective times, admission decision, normalization specification, and document digest. Metadata strings alone are too weak for update planning and temporal queries.

The corpus state at barrier τ is:

$$D_\tau = \bigcup_{r \in S_\tau} N(r),$$

with explicit conflict rejection if two revisions produce the same document ID under incompatible lineage. Deterministic canonical ordering is used only for serialization; the semantic state is a finite map.

8.3 Derivation stages

Let C_b, P_b, and E_b denote chunking, representation, and embedding transformations under build specification b:

$$\begin{aligned} Chunks_\tau &\quad C_b(D_\tau), \\ Reps_\tau &\quad P_b(Chunks_\tau, \omega_P), \\ Vectors_\tau &\quad E_b(Reps_\tau, \omega_E). \end{aligned}$$

The transcripts ω_P and ω_E retain provider outputs or their immutable content references. Once those transcripts are fixed, the derivation can be deterministic. Without them, P_b and E_b are stochastic kernels.

The index builder consumes chunks, representations, and vectors:

$$I_b : (D, C, P, E) \to IndexArtifacts + BuildFailure.$$

For an exact deterministic backend, two builds from the same canonical inputs should have the same logical manifest and rankings even if physical bytes differ because of backend serialization. If byte identity is required, the backend must declare deterministic serialization. The manifest should distinguish logical index identity from material artifact digest.

8.4 Build denotation

A complete deterministic build with retained provider transcripts is:

$$B_b(S_\tau, \omega) = Verify\Big(Index\Big(Embed\Big(Represent\Big(Chunk\big(Normalize(S_\tau)\big)\Big)\Big)\Big)\Big).$$

Its codomain is a disjoint sum of verified release material and typed failure. A warning is not silently collapsed into success; it is part of release observations and can invalidate a gate.

In the stochastic form:

$$[\![B_b]\!] : S_\tau \rightsquigarrow (ReleaseMaterial + BuildFailure) \times BuildTrace.$$

A build trace includes source cursor, stage cache hits, provider calls, resource admissions, retries, quarantined items, counts, durations, and verification results. The release material identifies the exact retained provider outputs used. Rebuilding with fresh stochastic outputs creates a different release even when the source and build specification are unchanged.

8.5 Release construction

A RAG release is a manifest over behaviorally material components:

$$R = (S_\tau, B, Q, G, X, P, V).$$

B identifies build semantics and index artifacts. Q identifies retrieval and query policy. G identifies generation and agent policy. X identifies auxiliary structured stores and connected-source configuration. P identifies presentation and projection policy. V identifies validators and contracts. Each coordinate is content-addressed or uses an immutable external version identity.

The release ID is a canonical digest of this manifest. Paths, environment variable names, and mutable model aliases are not sufficient. Operational settings that can change outcomes under deadlines, such as timeout partitioning and fallback policy, belong in Q or G. Purely scheduling settings that are observationally irrelevant under the declared equivalence can be deployment configuration rather than release material.

This broader release corrects a concrete current defect. In GEC, the bundle ID excludes synonyms and reranking. In Garden, index, fact database, tool config, prompt, and profile are resolved separately. A unified release allows one trace field to identify the complete behavior used for a turn.

8.6 Index meaning

An index artifact denotes a search relation, not merely bytes. For lexical index L and vector index V:

$$[\![L]\!](q, f, k) = \text{ordered lexical hits satisfying filter } f,$$

$$[\![V]\!](e(q), f, k) = \text{ordered vector hits satisfying filter } f.$$

Tie policy, score domain, filter semantics, and result completeness are part of the denotation. An approximate vector index denotes a distribution or set of allowed rankings parameterized by construction state and search parameters. Its contract should state recall relative to an oracle over a workload distribution, not pretend to be extensionally equal to exact search.

A backend that applies filters after top-k search has a different denotation from one that retrieves top-k within the filtered subcorpus. The interface must not call both behaviors simply `Search(filter, k)`. Capability and completeness should be explicit.

9. Denotational semantics of retrieval

9.1 Query context

A query context is more than text:

$$x=(q,u,c,\rho,d,\kappa),$$

where q is user and rewritten text, u is subject and authorization state, c is conversation state, ρ is route or intent, d is a deadline budget, and κ is request metadata such as locale, tenant, comparison ID, and idempotency key. The release lease is supplied by the runtime rather than the caller.

A direct retrieval denotation is:

$$[\![Search_R]\!] : x \rightsquigarrow (RankedEvidence + SearchFailure) \times Trace.$$

Even deterministic local retrieval becomes effectful when query embedding, connected search, or remote reranking is involved.

9.2 Channel semantics

For each channel i, let H_i be an ordered sequence of representation hits with stable IDs. A channel may fail. The route policy defines whether failure is terminal, degrades to other channels, or triggers a fallback route. This policy is part of the denotation:

$$Channels_R(x) \in D\left(\prod_i (H_i + F_i)\right).$$

Parallel and sequential execution have the same outcome only when there are no shared deadlines, cancellation races, provider quotas, or adaptive routes. Under a common deadline, scheduling changes which channels finish and therefore changes the outcome distribution. The trace must record schedule-relevant facts, and operational tuning must state whether outcome variation is allowed.

The abstract algebra can treat channels as independent, while the operational semantics supplies deadline and cancellation labels. This separation permits a deterministic reference interpreter used in tests and a concurrent production interpreter used in serving.

9.3 Collapse and representation semantics

Representations are searchable views of source chunks. A collapse function maps representation hits to chunk hits. A common policy retains the best rank or score per chunk. Let $owner(r)$ be the source chunk of representation r. Then:

$$collapse(H) = sort\left(\left\{\left(c, min\{rank_H(r) : owner(r) = c\}\right)\right\}\right).$$

Different collapse policies are not equivalent. Keeping the best representation emphasizes any matching view. Aggregating multiple representation ranks rewards chunks supported by several generated views. Capping representations per chunk changes diversity. The policy must be named in query identity and evaluated jointly with representation generation.

`ragkit` correctly treats representations as non-evidence. Hydration projects a chunk ID back to source text. A generated question or summary can cause a chunk to rank, but it should not itself be cited as

authoritative source material unless the product explicitly classifies generated evidence and validates it under a different policy.

9.4 Fusion

Weighted reciprocal-rank fusion for channel set I is:

$$score(c)=\sum_{i\in I:c\in H_i}\frac{w_i}{k_0+rank_i(c)}.$$

The result is sorted by descending finite score with a stable ID tie-break. Channel names should be ordered canonically before accumulation to avoid map-iteration nondeterminism. Non-finite weights or scores should be rejected.

GEC's sweep exploits a useful factorization: once channel rankings are frozen, many fusion parameters can be evaluated without rerunning retrieval. This optimization is correct for fusion-only candidates. It is invalid when a candidate changes query rewrite, channel k, filters, representation collapse, or index behavior, because the frozen rankings are no longer sufficient statistics.

9.5 Authorization semantics

Let $A_R(u,e)$ be the release policy deciding whether subject u may use evidence item e. The authorized candidate set is:

$$H^u=\{e\in H:A_R(u,e)=allow\}.$$

The required result is the top-k ranking over H^u, not merely the allowed elements among the first m global candidates. A correct searcher can implement this with index filter pushdown, separate partitions, or sufficient authorized overfetch with a proof of completeness. Fixed overfetch without such a bound is a heuristic.

Authorization also constrains effects. Define $disclose(e,p)$ as sending evidence text e to provider p. The non-disclosure invariant is:

$$\forall disclose(e,p)\in t, A_R(u,e)=allow\wedge P_R(u,e,p)=permit,$$

where P_R is provider-specific data policy. Filtering the returned list does not establish this invariant. The query plan should produce an authorization certificate over the exact candidate IDs before hydration for remote consumption.

9.6 Reranking and degradation

A reranker consumes a query and candidate evidence and returns an ordering or scores. The candidate pool policy, document-text composition, model identity, and blend strategy are behavior inputs. GEC blends fused and reranked ranks with a second fusion rather than replacing the fused order. This preserves some channel-agreement signal but defines a distinct ranking function.

Failure policy is a sum type:

$$RerankPolicy=FailClosed+FallbackFused+FallbackLexical+Abstain.$$

The result should carry `Degraded` when a fallback is used. Evaluation must count fallback frequency and assess fallback outcomes. A fail-open reranker can look excellent in successful offline calls while silently reverting under production provider errors.

9.7 Evidence admission

Ranking and evidence admission are separate. Admission selects an ordered subset under count, rune/token, diversity, source, and policy constraints. The simple prefix admission used in `ragkit` has desirable monotonicity under increasing budget, but more advanced policies may optimize coverage or diversity.

Let H be ranked evidence and b a budget. An admission policy returns $E \subseteq H$ and a certificate showing every item satisfies lineage and policy:

$$Admit_R(H, b, u) = (E, \gamma).$$

The certificate should identify the release, subject-policy decision, evidence kind, source revision, and exact text digest. Generation and presentation consume the certificate rather than a raw list.

10. Denotational semantics of answers, agents, and presentation

10.1 Retrieve-then-generate

The answering interpreter is:

$$Answer_R = Validate_R \circ Generate_R \circ FormatContext_R \circ Search_R.$$

Because search and generation are effectful, this equation denotes Kleisli composition rather than ordinary function composition. Each stage can add typed trace events and fail. The final outcome is:

$$AnswerOutcome = Answered + Abstained + Failed + Cancelled.$$

`Answered` contains answer text or structured answer, citation mapping, admitted evidence certificate, usage, and release ID. `Abstained` is a successful policy result, not a provider failure. `Failed` retains the stage and partial trace. `Cancelled` records the cancellation boundary and whether any events or provider disclosures already occurred.

10.2 Grounding semantics

A grounding contract relates claims and citations to admitted evidence. At minimum, every cited ID must belong to the evidence certificate. Stronger contracts can require claim-level entailment, citation coverage, or evidence-kind restrictions. Let a be a proposed answer and E admitted evidence:

$$Grounded_v(a, E) \in Valid(a') + SafeAbstention(r) + Invalid(e).$$

`ragkit` implements a structural version: parse the contracted output and ensure citation IDs were supplied. This is a small reliable kernel. Semantic entailment judges can augment it but should not replace the structural check.

Grounding is release-relative. A citation label such as `E1` is only meaningful with the evidence-session ID and release ID that assigned it. Persisted transcripts should retain the stable chunk and source revision IDs behind presentation labels.

10.3 Agentic semantics as a transition system

An agentic RAG turn cannot be represented faithfully by one retrieval composition. Let agent state be:

$$a = (c, M, L, n, d),$$

where c is conversation context, M is model-visible message/tool state, L is evidence ledger, n is the remaining iteration budget, and d is the remaining deadline and cost budget.

A policy kernel chooses a next action:

$$\Pi_R(a) \rightsquigarrow SearchCall + StructuredCall + OtherToolCall + FinalCandidate + PolicyFailure.$$

Tool calls transition the state and append observations. The turn denotation is the distribution over terminal trajectories obtained by iterating Π_R until a final, failure, cancellation, or budget terminal state. A least fixed-point presentation is possible when the policy is well-founded by an iteration budget. Operationally, a bounded labelled transition system is clearer.

The terminal condition is part of the agent policy. It can require a final answer, a valid choice set, a grounded widget, or an abstention. “The model stopped calling tools” is not sufficient unless validated.

10.4 Evidence-ledger effects

The ledger introduces history dependence. Let `add(L,e)` return an updated ledger, stable presentation label, and freshness marker. It must satisfy:

1. **Stable labeling:** adding the same evidence identity again returns the same label.
2. **Boundedness:** distinct admitted evidence and total material remain within declared limits.
3. **Release coherence:** every element belongs to the ledger’s release epoch.
4. **Order determinism:** equal event sequences produce equal ledger snapshots.
5. **No authority escalation:** generated or connected material cannot be reclassified as source evidence through insertion.

A conversation-scoped ledger may intentionally span turns. In that case the product must declare whether the release is pinned for the whole conversation or whether evidence epochs are separated and old evidence is visibly marked. Garden’s per-conversation tools make this question concrete.

10.5 Presentation denotation

Let p be product presentation policy. Presentation maps terminal answer state and evidence to a finite event trace:

$$Present_{R,p} : (Outcome, E, c) \to UITrace.$$

The projection may contain messages, sources, choices, tool disclosure, product cards, comparisons, steps, warnings, and developer provenance. Typed widgets should carry stable evidence references, not only copied text. Garden’s grounded widget mechanism is an applied instance of this principle.

Presentation changes can be user-outcome changes even when answer text is identical. Hiding a source, changing choice order, suppressing a conflicting product field, or failing to emit a completion event can alter behavior. The release manifest should identify presentation schemas and projection policy where they are server-controlled.

10.6 Streaming denotation

A streaming answer denotes a prefix-closed set or distribution of event sequences. Every nonterminal prefix is an observable state. Cancellation may occur after any prefix. The final event should commit terminal status and the exact citation/evidence set.

Let T be an event sequence. Prefix closure means:

$$T \in \mathcal{T} \Longrightarrow \forall T' \preceq T, T' \in Prefixes(T).$$

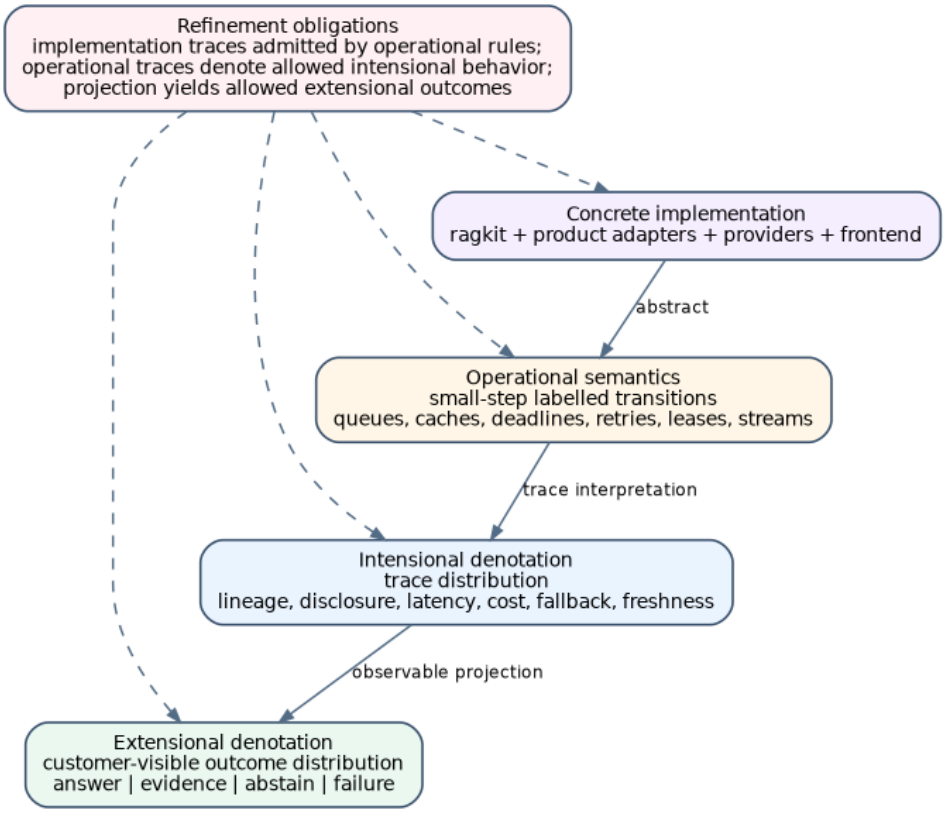
This matters for safety. A model can emit an unsupported claim before final validation. If the frontend displays raw tokens, final validation cannot retract their disclosure. Production designs should either validate buffered structured output before customer presentation or use a streaming contract that constrains provisional content and supports explicit retraction semantics.

11. Extensional, intensional, and probabilistic semantics

11.1 Why final text is insufficient

Two runs can produce the same answer text while differing in every operational property that matters: one uses fresh authorized evidence and one uses stale cached evidence; one calls a remote reranker with restricted content and one does not; one uses the intended vector route and one falls back to lexical; one costs ten times more; one takes 300 ms and one exceeds the frontend timeout; one cites the active release and one mixes old source cards with a new answer.

The **extensional denotation** projects to customer-visible terminal outcomes. The **intensional denotation** retains the trace and resource behavior. The **operational semantics** explains which transitions generate that trace.



The implementation refines operational, intensional, and extensional semantic layers.

11.2 Trace schema

A useful trace is typed, hierarchical, and causally linked. Each event should contain:

- trace, span, parent, request, turn, conversation, and comparison IDs;
- release, source barrier, build, query-plan, and evidence-session IDs;
- stage and operation kind;
- start/end or monotonic duration information;
- input and output semantic digests;
- provider and cache decision;

- authorization and disclosure summary;
- outcome class, warning, fallback, or error code;
- usage, cost estimate, queue time, and resource admission;
- causation links to frontend events and experiment cells.

Raw prompts and evidence text should not be indiscriminately logged. The trace schema should separate safe structural metadata from protected payload references. Product policy controls whether immutable encrypted payload artifacts are retained.

11.3 Probabilistic release semantics

Given subject u, conversation c, and request x, a release denotes:

$$[\![R]\!](u, c, x) \in D(Outcome \times Trace).$$

The distribution includes model stochasticity, provider failures, network timing, approximate-search variability if construction is nondeterministic, and scheduling under deadlines. A deterministic replay mode conditions on retained provider transcripts and recorded source state. It is not the same as the live distribution; it is a counterfactual interpreter useful for diagnosis.

Optimization compares releases as statistical experiments. A candidate is preferable only relative to a utility order or constraints over outcomes and traces. There is generally no total order. One release can improve answer correctness while increasing latency and another can improve freshness while increasing cost. The natural result is a Pareto set under hard safety and reliability constraints.

11.4 Behavioral equivalence classes

Several equivalence relations are useful:

Material equivalence requires identical release manifests and artifacts.

Retrieval equivalence requires the same authorized ranked evidence for a declared query set.

Answer equivalence requires the same validated answer and citations, possibly ignoring wording under a semantic judge.

Trace equivalence requires the same protected operational projection.

User-outcome equivalence requires the same final customer-visible state and allowed timing class.

Safety equivalence requires identical authorization, disclosure, and policy outcomes even if ranking changes.

Optimization candidates should declare the finest equivalence they preserve. A worker-count change may target retrieval and safety equivalence. ANN cannot. A prompt rewrite cannot claim answer equivalence. A frontend sorting change cannot claim user-outcome equivalence.

11.5 Counterfactual and paired semantics

A paired trial evaluates incumbent R_0 and candidate R_1 on the same case z and repeat seed/transcript coordinate j:

$$\delta_{z,j} = m(R_1, z, j) - m(R_0, z, j).$$

Pairing removes case difficulty from the difference and supports confidence intervals over per-case changes. For deterministic retrieval there may be one repeat. For stochastic generation, repeated samples or common random numbers can reduce variance where provider controls permit.

A replay trial can reuse the same source snapshot and captured user request but must not reuse incumbent retrieval results when the candidate changes an upstream dependency. The dependency graph determines which artifacts are valid counterfactual inputs.

12. Operational semantics of index maintenance

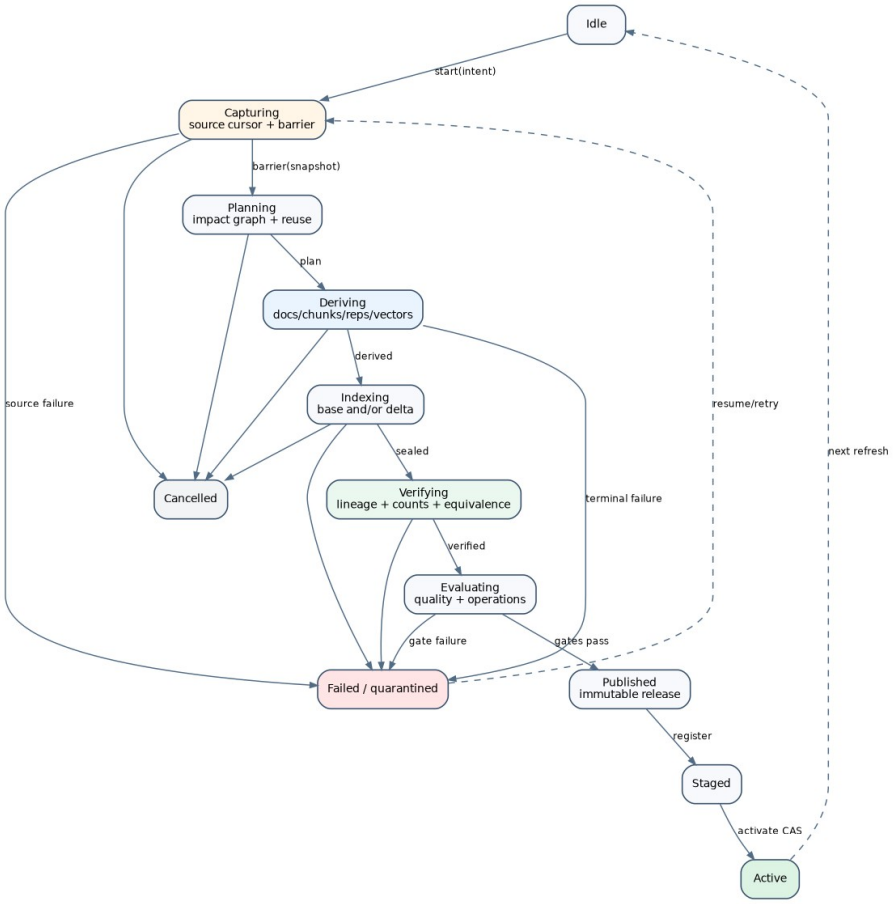
12.1 Build configuration

A maintenance configuration is:

$$\langle i, c, s, p, w, a, h, o \rangle_B$$

where i is build intent and locked release specification, c is source cursor/barrier state, s is the current stage, p is the impact plan, w is queued and in-flight work, a is partial immutable artifact state, h is checkpoint and lease state, and o is the ordered event log. The coordinator advances this configuration by labelled rules.

The stage set is not a universal list; it is a typed DAG. A typical order is capture, normalize, chunk, represent, embed, lexical index, vector index, verify, evaluate, publish. Independent branches can run concurrently after their dependencies are satisfied.



A production index-maintenance labelled transition system.

12.2 Capture rules

A start rule acquires a build lease for intent ID i. A connector snapshot or change stream advances the cursor. A barrier closes the logical input:

$$\frac{acquireBuild(i)=ok}{\langle i,\bot,idle,\ldots\rangle_B \xrightarrow{start(i)} \langle i,c_0,capturing,\ldots\rangle_B}$$

$$\frac{next(c)=Barrier(\tau,w)}{\langle i,c,capturing,\ldots\rangle_B \xrightarrow{barrier(\tau,w)} \langle i,c',planning,\ldots\rangle_B}.$$

Duplicate changes are accepted because normalization is idempotent by source revision identity. A conflicting duplicate with the same revision ID and different digest is a source-integrity failure.

12.3 Impact planning

The planner compares the captured logical document map with the base release lineage and computes affected derivations. It uses semantic fingerprints for normalization, chunking, representation, embedding, and index policy.

For each logical object x, a derivation key is:

$$K_f(x)=H(specID(f),canonicalInput(x),dependencyIDs).$$

A cached artifact can be reused only when this key matches and the artifact verifies. Worker count, retry count, and queue implementation are excluded unless they can change output. Prompt, model version, normalization code identity, and representation kind are included.

The plan is itself an immutable artifact. It lists additions, updates, tombstones, reusable derivations, required provider work, backend operations, expected counts, and estimated resource budgets. This enables dry-run review and exact resume.

12.4 Derivation and retry rules

A work item can execute more than once. The semantic effect is committed once by key:

$$\frac{K \notin A \quad execute(K)=y \quad verify(K,y)}{\langle \ldots,w,A,\ldots\rangle_B \xrightarrow{commit(K,y)} \langle \ldots,w',A[K\mapsto y],\ldots\rangle_B}.$$

If two workers produce the same key, the first verified commit wins and the second result must be byte- or logical-equivalent. A mismatch quarantines the key as nondeterministic. This is a more meaningful correctness target than exactly-once task execution.

Provider budget admission occurs before the call. Retry labels retain attempt number and error class. Permanent input errors can quarantine one item under a declared policy; provider-wide or invariant failures terminate the stage. The release records whether quarantined items were excluded and gates decide whether publication is allowed.

12.5 Checkpoints

A checkpoint is valid only at a semantically closed boundary: all committed outputs it references are immutable and verified, and the remaining plan is derivable from the checkpoint plus the original intent. Checkpointing arbitrary in-memory queues is unnecessary.

A checkpoint manifest includes build intent, source barrier, plan ID, completed stage/item keys, artifact references, warnings, budget usage, and event-log position. Resume verifies every referenced artifact

and reconstructs queues from the plan. This design is portable across local process restarts and external schedulers.

`ragkit/flow` remains the inner executor for bounded maps and cached provider work. The coordinator owns durable stage state. Conflating them would either make `flow` a workflow framework or leave production state implicit.

12.6 Verification and publication

Verification is a separate stage, not a collection of assertions scattered through build code. It checks:

- source barrier and normalized corpus identity;
- document/chunk/representation/vector lineage;
- counts and referential integrity;
- embedding model and dimension compatibility;
- index backend manifests and filter capability;
- tombstone application;
- determinism or declared approximation tolerances;
- release-manifest completeness;
- optional full-build equivalence sample or oracle;
- policy gates such as no unscoped GEC documents.

Publication writes immutable material and registers a verified release. It does not make the release active. A failed evaluation can leave a verified but ineligible release for diagnosis.

12.7 Failure, cancellation, and quarantine

Build failure is typed by stage and retryability. Cancellation is an operator decision and should not be represented as failure. Quarantine can apply to an item, stage output, or complete release. A quarantined release cannot be activated without an explicit override authority recorded in the activation event.

The event log is append-only. Current status is a pure reduction over events. This gives resume equivalence: replaying the same valid event prefix reconstructs the same coordinator state. External queues may redeliver commands; the reducer and semantic commit keys absorb duplicates.

13. Operational semantics of query and agent execution

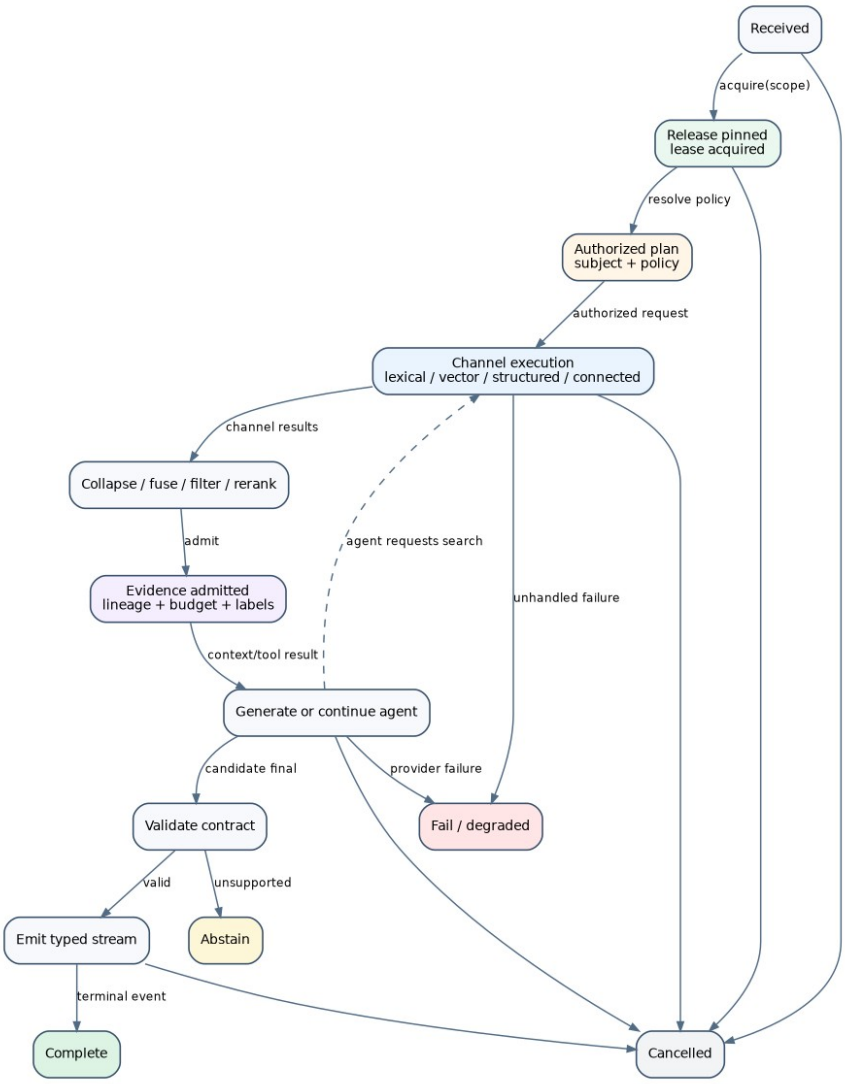
13.1 Query configuration

A query configuration is:

$$\langle x, u, \lambda, p, s, h, E, g, e, d \rangle_Q$$

where x is request and conversation input, u is subject policy context, λ is a release lease, p is a typed query plan, s is current stage or agent state, h is channel and candidate state, E is the evidence session, g is generation state, e is emitted event history, and d is remaining deadline/cost budget.

The query cannot access index or auxiliary artifacts before a lease is acquired. This makes release pinning a structural property rather than a logging convention.



A small-step query and interaction machine.

13.2 Release acquisition

The acquire rule resolves product scope, rollout cohort, and session policy to a release:

$$\frac{resolve(scope, cohort, session) = Rlease(R) = \lambda}{\langle x, u, \bot, p, received, \ldots \rangle_Q \xrightarrow{acquire(R)} \langle x, u, \lambda, p, authorizedPlan, \ldots \rangle_Q}.$$

If a conversation is pinned, the resolver returns its existing release epoch. If pinning is per turn, each new turn resolves independently. The product must choose and expose the policy.

Acquisition failure is terminal before any provider call. The lease is closed in every terminal transition, including cancellation and panic recovery.

13.3 Authorization domination

The plan analyzer verifies that every remote text-consuming operation is dominated by an authorization operation. At runtime, authorization produces a certificate over candidate IDs and provider policy. A remote rerank step requires that certificate:

$$\frac{\gamma = authorize(u, H, p)\ valid(\gamma, R, p)}{\langle \ldots, H, \ldots \rangle_Q \xrightarrow{rerankRemote(p, \gamma)} \langle \ldots, H', \ldots \rangle_Q}.$$

No rule exists for remote reranking raw unauthorized candidates. This is the operational form of the security invariant and directly identifies the required GEC change.

13.4 Channel execution and deadlines

Each channel has allocated budget d_i and fallback policy. A start event records query digest, filters, k, and release artifact. Completion appends hits and timing. Timeout yields a typed channel timeout, not an empty ranking.

A join rule decides when to proceed. A strict hybrid route waits for all required channels. A best-effort route can proceed after a deadline with completed channels. An adaptive route may start vector search only when lexical confidence is low. These are different plans and must have different IDs.

Concurrent execution is safe only for state-independent channels. Shared provider rate limits and caches are operational resources, but their scheduling should not mutate semantic hit values. If they can cause deadline-dependent omission, the trace records it and the route's denotation includes that variability.

13.5 Ranking transitions

Collapse, fusion, local policy filtering, and deterministic admission are pure transitions over immutable values. Their inputs and outputs can be content-digested. Reranking can be local or remote. Hydration resolves exact chunks from the leased release.

The ordering should be explicit in the plan. A secure default is:

1. apply index-level authorization and source filters during channel search;
2. collapse and fuse authorized IDs;
3. locally hydrate only the bounded candidate pool;
4. recheck authorization against hydrated lineage;
5. invoke permitted remote reranking;
6. admit final evidence under context and diversity policy.

For backends without filter pushdown, a local metadata prefilter can operate on chunk metadata before text hydration. If this cannot guarantee top-k completeness, the outcome should include `AuthorizedRecallBoundUnknown` rather than silently claiming complete search.

13.6 Generation and validation

Generation can stream provisional events or return a complete candidate. The contract determines when customer-visible output may be emitted. A buffered structured mode validates before presentation. A token streaming mode emits provisional content and requires a stronger provider/prompt contract.

A generation failure after evidence has been emitted can return a partial outcome containing search results and trace. `ragkit` already returns partial information in some error paths. The shared outcome type should make partial evidence explicit rather than hiding it behind a non-nil result plus error convention.

Validation transitions a candidate to `Answered`, `Abstained`, or `Invalid`. Invalid output can be retried under a bounded repair policy, converted to safe abstention, or fail. Every retry consumes budget and remains in the trace.

13.7 Agent transitions

An agent state chooses an action. A search action invokes the shared search interpreter under the same lease and evidence session. The tool result is serialized and appended to model state. A structured action invokes a product-owned tool and returns typed evidence. A final action invokes validation and presentation.

The iteration rule decrements a bounded counter. The model cannot extend it. Tool schemas and descriptions are part of release behavior because they affect policy choice. This directly explains why self-optimization of a tool description must produce a new release candidate and be evaluated at the agent/session level.

Agent retries require idempotency. A tool call has a call ID. Repeating the same call ID returns the retained result or an explicit conflict if inputs differ. This prevents duplicate connected-source effects and stabilizes replay.

13.8 Cancellation

Cancellation is observed at stage boundaries and through context propagation. Its semantics should answer:

- whether queued channel calls were prevented;
- whether remote calls had already received text;
- whether partial tokens or UI events were emitted;
- whether the evidence session remains reusable;
- whether the turn is terminal and persisted;
- when the release lease is released.

A frontend cancellation marker is not sufficient if the server continues expensive tool calls. Conversely, a server cancellation without a terminal event leaves the frontend uncertain. The query and projection machines need a causally linked terminal cancellation event.

14. Release activation, pinning, and concurrency

14.1 Release lifecycle

A release moves through:

$$Registered \to Verified \to Staged \to Active \to Draining \to Retired,$$

with side states `Quarantined` and `Rejected`. Verification concerns artifact integrity. Staging concerns operational readiness and preload. Activation changes resolver state. Draining prevents new leases while preserving old ones. Retirement permits cleanup after policy retention.

No in-place mutation is allowed. Rollback activates a prior immutable release through the same protocol. A repaired release receives a new ID.

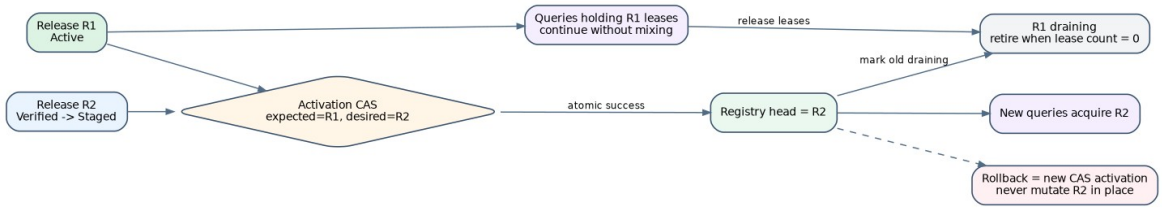
14.2 Compare-and-swap activation

Activation takes product scope, expected current release, desired release, actor, reason, and gate evidence. The registry atomically checks expected state and writes a new activation event:

$$CASActivate\left(scope, R_{old}, R_{new}\right).$$

If another actor has already changed the head, the operation fails with a conflict. This prevents stale automation from overwriting a newer human activation.

Exactly-once activation is obtained by idempotency key and compare-and-swap effect, not by exactly-once delivery of the activation command. Repeating a successful command returns the retained event.



Atomic activation changes the release resolved by new leases while old work drains.

14.3 Lease semantics

A lease is an immutable handle to opened release resources and a reference in the registry. It provides the release manifest, searchers, chunk/source stores, structured fact snapshots, prompts, validators, and policy. Its `Close` decrements the active reference count or releases an epoch guard.

Opening every artifact per query is inefficient. The release manager can pool opened handles. The important contract is that the handle is immutable and remains valid until all leases close. Preloading a staged release verifies that every artifact can open before activation.

Resource cleanup occurs only when a release is not active, has zero leases, and has passed retention or audit policy. A process crash may leak registry counts; leases therefore need epochs or TTL/heartbeat semantics at the process level. Local in-process references remain exact.

14.4 Pinning policies

Three useful policies exist.

Per request pinning gives each search or answer call the current release. It maximizes freshness but can mix releases within a multi-call agent turn unless the turn passes its lease through every call.

Per turn pinning is the default recommendation for chat. All tool calls, evidence, answer validation, and presentation for one turn use one release. The next turn can use a newer release, with prior evidence retained as historical lineage.

Per conversation pinning provides maximum consistency for long workflows but can hold stale or retired resources indefinitely. It is appropriate only when conversation semantics require a stable corpus epoch and leases have bounded lifetime.

Garden's conversation-scoped RAG session suggests either per-conversation pinning or explicit evidence epochs. GEC's run-scoped ledger fits per-turn pinning. The API should force the choice.

14.5 Canary and cohort routing

The resolver can map a rollout cohort to a candidate release. Cohort assignment must be stable for the experiment unit: user, conversation, or request. The assignment and candidate probability are recorded. Safety-critical scopes may be excluded.

Canary resolution must still return a normal release lease. Product code should not branch on "experiment mode" throughout the query path. The only difference is which immutable release the resolver selects. This keeps online evaluation close to production semantics.

14.6 Mixed-release prohibitions

The following are invalid:

- lexical search from one release and vector search from another;
- evidence text from one release with chunk metadata from another;
- answer prompt or grounding contract changed after retrieval;
- product fact database from a new release with old tool configuration when the manifest binds them;
- frontend source cards hydrated from current release rather than the turn's release;
- a conversation ledger that silently relabels old evidence under a new release.

These can be prevented by making artifact access possible only through the lease and by including release ID in evidence and event types.

15. Frontend and event-stream semantics

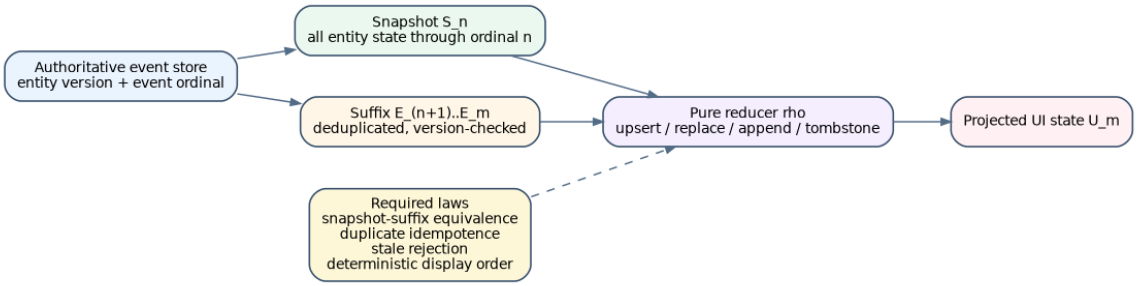
15.1 The frontend is a replica

A browser is a partial replica of authoritative conversation state. It initializes from a snapshot and applies a suffix of events. Network reconnect, duplicate delivery, out-of-order delivery, and sparse updates are normal distributed-systems conditions, not exceptional UI bugs.

Let authoritative state after event n be S_n. Let reducer ρ apply an event to state. Correct snapshot hydration requires:

$$S_m = \rho^{\square}\left(S_n, \left[e_{n+1}, \ldots, e_m\right)\right).$$

The server may send a snapshot and live events concurrently. The client buffers events until snapshot application, discards events at or before the snapshot ordinal, orders the suffix according to protocol, and reduces it.



Frontend state is a pure reduction of an authoritative snapshot and versioned event suffix.

15.2 Event envelope

A robust event envelope is:

```
Event {
    event_id
    stream_id
    stream_ordinal
    entity_id
    entity_version
    operation          // upsert, patch, tombstone
    patch_mode         // replace, append, structured merge
    payload
    release_id
    conversation_id
    turn_id
    causation_id
    occurred_at
}
```

Global ordinals are useful for snapshot suffixes; entity versions are necessary for stale-update rejection. Event IDs provide duplicate detection. Causation links a UI event to a tool, answer, or cancellation transition. Release and turn IDs preserve RAG provenance.

15.3 Reducer laws

The reducer should satisfy at least six laws.

Determinism. Equal initial state and equal event sequence yield equal state.

Snapshot-suffix equivalence. A server snapshot through ordinal n plus the suffix produces the same state as replaying all events through m.

Duplicate idempotence. Applying the same event ID twice has the same result as applying it once.

Stale rejection. An event with lower entity version cannot overwrite higher-version fields.

Tombstone monotonicity. A tombstoned entity cannot reappear without a strictly higher valid incarnation or explicit recreation rule.

Display determinism. Projection order is a pure function of stable fields and declared tie-breakers.

Append patches are not naturally idempotent. They require exactly-once event IDs and deduplication, or they should carry sequence offsets so duplicates and gaps can be detected. Replace patches are simpler and often preferable after reconnect.

15.4 Analysis of the current GEC reducer

The current WebSocket manager buffers pre-hydration entities, bounds the buffer, sorts it, applies a single snapshot, and then flushes the suffix. These are sound ingredients. The Redux reducer preserves original creation ordinal, merges sparse incoming data, and takes the maximum update ordinal.

However, it does not guard the merge with a version comparison. An incoming stale entity can overwrite fields while `updatedOrdinal` remains the maximum. The reducer also applies append patches directly to existing strings; duplicate delivery duplicates content. Correctness may currently depend on stronger server delivery assumptions than the client type expresses.

The migration should add event ID and entity version checks before changing transport shape. Tests can then generate duplicated and permuted suffixes and verify convergence under allowed reorderings. Existing snapshots remain compatible through a versioned adapter.

15.5 Streaming answer consistency

A message stream can be modeled as an entity with monotonically increasing version and content prefix. An update at version $v+1$ either replaces the full content with a longer prefix or appends bytes starting at a declared offset. Terminal status closes the stream.

The laws are:

- content never shrinks unless a typed retraction occurs;
- append offset equals current content length;
- duplicate event IDs are ignored;
- terminal content and citation set are immutable;
- cancellation is terminal and causally linked to server cancellation;
- a final validated answer can replace provisional content only under an explicit operation visible to the UI.

These semantics let frontend tests replay real transcripts and let production monitoring detect gaps rather than silently presenting corrupted streams.

Part III. Corpus evolution and index maintenance

16. Source revisions, changes, cursors, and barriers

16.1 The source contract

A shared RAG package should not assume that a corpus is one JSON file. It should accept source connectors that can provide a consistent snapshot, an ordered or partially ordered change stream, or both. The minimum interfaces are semantic rather than transport-specific:

```
type SourceKey string
type RevisionID string
type Cursor string
type SnapshotToken string

type DocumentRevision struct {
    Key         SourceKey
    Revision    RevisionID
    ObservedAt  time.Time
    EffectiveAt *time.Time
    ContentSHA  digest.Digest
    Metadata    map[string]string
    Payload     artifact.Ref
}

type Tombstone struct {
    Key        SourceKey
    Revision   RevisionID
    ObservedAt time.Time
    Reason     string
}

type Change struct {
    Cursor    Cursor
    Upsert    *DocumentRevision
    Delete    *Tombstone
    Barrier   *Barrier
}

type Barrier struct {
    Token     SnapshotToken
    Watermark time.Time
}
```

A connector can expose `Snapshot` when the source supports a repeatable read and `Changes` when it supports a durable cursor. A filesystem connector may scan and produce a synthetic barrier. A Git connector can use commit/tree identity, as RAG-TTC already does. A database connector can use transaction snapshot or changelog position. A web connector may offer only best-effort observed revisions and a capture batch token.

The common contract should not pretend all connectors have the same consistency. Each connector declares:

- whether snapshot reads are repeatable;
- whether cursors are durable and monotone;
- delivery order and duplication guarantees;
- delete visibility;
- maximum lateness or reordering, if any;
- the meaning of its watermark;
- payload immutability and retention;
- subject and data-class policy.

The release manifest records these guarantees so freshness and reproducibility claims are honest.

16.2 Stable logical identity

Source key, source revision, document ID, and chunk ID answer different questions. A source key identifies the logical external item. A revision identifies one version of that item. Normalization may create one or many document IDs. Chunk IDs identify exact derived spans under one chunker and document revision.

Using content digest alone as source identity loses delete and rename semantics. Two different source items can contain identical text and still require separate lineage and policy. Conversely, a source key can remain stable while content changes. The model therefore carries both logical and content identity.

Renames are source-specific. Git can represent a delete and add unless similarity analysis is applied; a database row key remains stable. RAG semantics usually need only stable source lineage and correct current content, not a universal rename detector. Connectors can emit an optional predecessor relation when meaningful.

16.3 Admission and policy

Admission is not a preprocessing convenience. It determines the knowledge and disclosure boundary. RAG-TTC's Git snapshot excludes vendor/generated content and applies size and test-data policy. GEC documents carry access scopes and source roles. Garden distinguishes product pages, guides, and structured product facts.

An admission decision should be retained as:

```
type Admission struct {
    Key         SourceKey
    Revision    RevisionID
    Decision    AdmissionDecision // admit, exclude, quarantine
    RuleID      string
    PolicyID    digest.Digest
    Explanation string
}
```

Excluded items do not silently disappear from operational reports. Counts and reasons support corpus coverage analysis. A policy change is a knowledge- and security-changing intervention even if source content is unchanged; it invalidates normalized corpus state and downstream artifacts for affected items.

No document lacking required access metadata should default to public. GEC's current `scopesAllow` behavior treats unscoped content as non-returnable, which is the safe default. Build verification should reject rather than merely hide such documents, because a different query path might omit the filter.

16.4 Normalization as a versioned function

Normalization converts source payload into canonical text and metadata. It handles HTML removal, whitespace, metadata names, redaction, product-field rendering, and prompt-injection hygiene. It must be versioned because a code change can alter every downstream digest even when source revisions are fixed.

A normalizer should be a pure function over an immutable payload artifact and a specification:

```
type Normalizer interface {
    Spec() NormalizeSpec
    Normalize(context.Context, DocumentRevision) ([]NormalizedDocument, error)
}
```

`NormalizeSpec` includes semantic version, configuration, parser identity, and data policy. The output includes exact source lineage, warnings, and extracted fields. Any use of current time, locale, network

fetch, or mutable database state must be explicit as an additional input; otherwise reproducibility is false.

16.5 Barriers and consistent snapshots

A barrier separates open-ended ingestion from one release intent. Suppose changes continue arriving after barrier τ. They belong to a later release unless the current build is explicitly amended and receives a new source snapshot identity. This avoids moving-target builds.

For a full snapshot connector, the barrier token may be a Git commit, database snapshot ID, or corpus manifest digest. For a change stream, a barrier means all source changes through cursor c have been materialized into the logical document map. The coordinator persists both cursor and normalized snapshot digest.

Cross-source releases require a vector of barriers, one per connector. There may be no global transaction across sources. The release records capture times and watermarks, and product policy defines acceptable skew. A knowledge index and structured fact database may intentionally have different epochs; the release makes the skew visible.

16.6 Late and corrected events

A connector can observe an older effective revision after a newer one. The source adapter must define conflict order. A common rule is highest source revision sequence, then effective time, then deterministic revision ID, never raw arrival order. Corrections can supersede prior revisions explicitly.

When a late event changes the logical state for a barrier already activated, it cannot mutate the release. It triggers a new corrective release. Audit links the correction to the affected prior release and measures stale exposure duration.

16.7 Deletion semantics

Deletion has three layers:

1. **Logical tombstone:** the source item is no longer part of current corpus state.
2. **Query invisibility:** new release views cannot return its documents, chunks, representations, or structured projections.
3. **Physical erasure:** stored artifacts are removed after retention, legal, and audit policy.

These layers must not be conflated. A delta overlay can make a document immediately invisible with a tombstone while old immutable base bytes remain until compaction. In-flight leases on an old release may still access it unless deletion policy requires emergency revocation. Security deletions may need a registry-level quarantine that prevents new and existing queries from using affected releases.

17. Incremental derivation algebra

17.1 From snapshots to differences

Let corpus state be represented as a multiset or finite map. The difference between states is a signed change ΔD such that:

$$D_{t+1} = D_t \oplus \Delta D.$$

For a deterministic transformation F, an incremental form F^{Δ} should satisfy:

$$F\left(D_t \oplus \Delta D\right) = F\left(D_t\right) \oplus F^{\Delta}\left(D_t, \Delta D\right).$$

This is the fundamental incremental correctness equation. It says maintained output after applying derived changes equals a fresh evaluation of the original transformation. The right side may reuse prior output; the left side is the oracle.

Not every stage has an efficient local differential, but every stage can be incrementally maintained by recomputing the affected partition and taking a set difference. The package should optimize only after specifying the equation.

17.2 Document-local chunking

Current chunkers operate independently per document. Let $C\left(d\right)$ be the chunk set for document d. Then corpus chunking is the disjoint union:

$$C\left(D\right) = \biguplus_{d \in D} C\left(d\right).$$

A document upsert affects only old and new chunks for that document:

$$\Delta C = -C\left(d_{old}\right) \uplus C\left(d_{new}\right).$$

Content-based chunk IDs cause unchanged chunks to survive when their byte range and text remain equal. An insertion near the beginning can shift ranges and invalidate later chunks even if text is unchanged. A future chunker can use structural anchors to increase identity stability, but this changes lineage semantics and requires evaluation.

Global chunkers, cross-document deduplication, or corpus-level clustering break document locality. Their specs must declare a larger invalidation scope. The impact planner derives affected closure from stage properties rather than assuming locality universally.

17.3 Representation differentials

Representations are derived per chunk and kind. For deterministic raw or breadcrumb representations, unchanged chunk identity and representation spec imply exact reuse. Generated summaries or questions additionally depend on prompt, provider/model version, decoding policy, and retained output.

A representation key can be:

$$K_P = H\left(chunkDigest, kind, promptID, modelID, decoderID\right).$$

When the key is unchanged, cached output is valid. When a prompt changes, every representation of that kind is affected even if chunks are unchanged. This is why the optimization dependency graph

matters: a prompt intervention invalidates representation and embedding artifacts but need not recompute normalization or chunking.

Generated output can be nondeterministic. Content-addressed cache turns one sampled output into retained material. A candidate that intentionally resamples must declare a new stochastic replicate rather than reuse the old key.

17.4 Embedding differentials

Embedding is pointwise over representation text under model and normalization identity:

$$E(P)=\{(id(p),e_m(text(p))):p\in P\}.$$

Additions require embeddings; deletions remove vector entries; unchanged representation digests reuse vectors. A model, dimension, pooling, or normalization change invalidates the whole affected representation set.

Provider aliases are unsafe cache identities. The manifest should use an immutable model version or provider-reported deployment revision. If the provider cannot supply one, the release records the alias plus observation time and retained vector digests; reproducibility is then material rather than semantic.

17.5 Lexical-index differentials

A lexical index supports upsert and delete when the backend exposes stable document IDs and snapshot or commit semantics. The logical differential is straightforward. Physical scoring statistics such as document frequency change globally, but an incremental backend updates them internally. The correctness oracle compares search results to a clean build at the same state.

Field mapping and analyzer changes require rebuilding the affected index because existing terms were produced under different semantics. Title boosts and token filters are part of index spec. GEC synonym expansion currently occurs at query time; moving synonyms into analyzer configuration would change both invalidation scope and query semantics.

17.6 Vector-index differentials

Exact vector storage can upsert and delete rows directly. ANN structures are more complicated. Some support dynamic insertion but weak deletion, background repair, or nondeterministic topology. A backend contract should distinguish:

- logical update acceptance;
- visibility epoch;
- delete/tombstone semantics;
- query snapshot isolation;
- compaction requirement;
- recall degradation under updates;
- deterministic rebuild behavior.

An ANN candidate can pass static recall and still degrade after many updates. Optimization and acceptance tests need update-sequence workloads and periodic exact-oracle checks.

17.7 Algebra of tombstones

Represent current output as a signed multiset or as `(base, additions, tombstones)`. A query view is:

$$V = (B \cup A).$$

The tombstone set must dominate both base and older additions. A newer upsert after deletion carries a higher source revision and can reintroduce the logical key with a new derived identity. Comparing only chunk IDs is insufficient; tombstones should target logical document/revision lineage so every derived representation is excluded.

17.8 Incremental equivalence tests

For generated random document states and change sequences:

1. build $F(D_0)$ cleanly;
2. apply changes incrementally to obtain M_n;
3. build $F(D_n)$ cleanly;
4. compare canonical logical outputs;
5. compare exact-backend rankings over generated queries;
6. compare approximate backend metrics within declared tolerance;
7. verify deleted lineage is absent;
8. restart from checkpoints at arbitrary points and repeat.

This property test should be part of backend certification. It is more important than unit tests for individual update methods because it checks the composed invariant.

18. Backend update models and the base-plus-delta design

18.1 Four update models

There are four common production models.

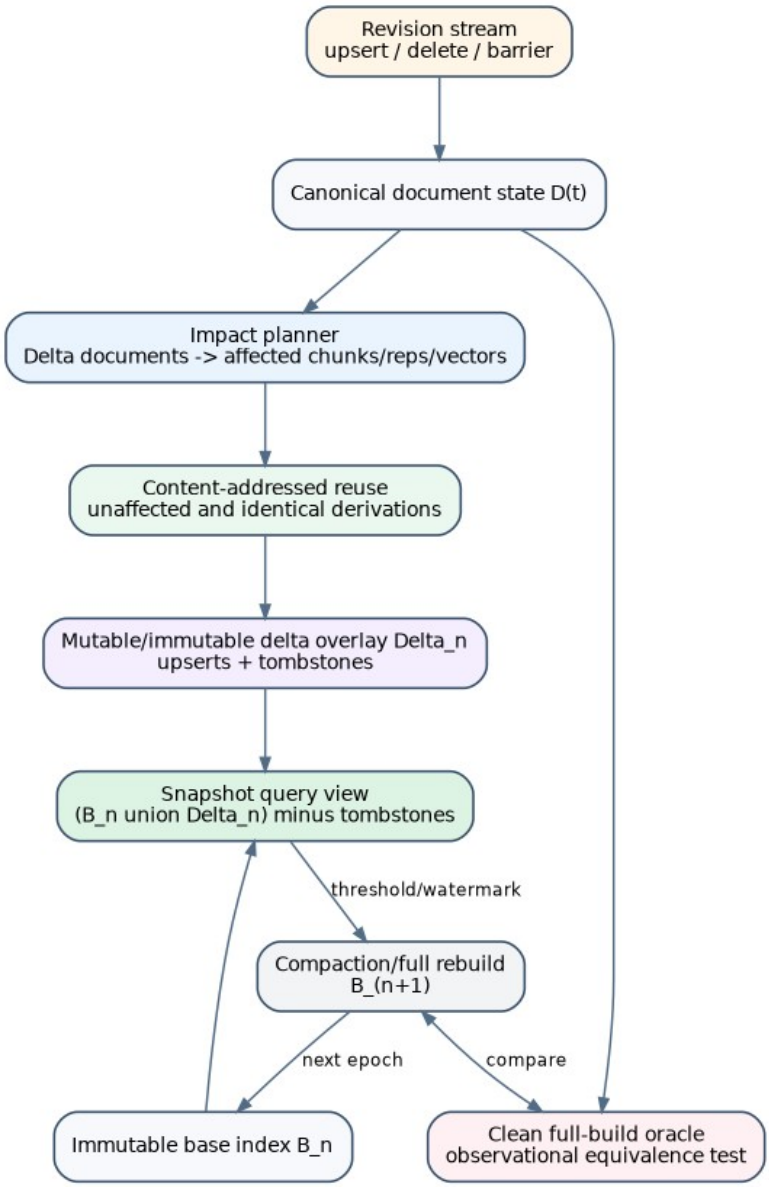
Full immutable rebuild. Every release contains a complete new index. This has the simplest correctness and rollback semantics but the worst freshness and rebuild cost.

In-place mutable index. Changes update the active index. Freshness is good, but query snapshot consistency, rollback, and audit are hard. A failed update can corrupt the active state.

Immutable base plus delta overlay. A stable base is queried together with a smaller delta containing upserts and tombstones. New releases can publish deltas quickly; compaction periodically produces a new base.

Partitioned immutable segments. Changes create immutable segments and tombstone maps; queries search many segments and merge results. This generalizes the overlay but requires segment management and score comparability.

For the current systems, base plus delta is the recommended first dynamic model. It extends existing immutable bundles without forcing a mutable active index and keeps rollback as release selection.



Incremental maintenance with immutable base, delta overlay, tombstones, and a clean-build oracle.

18.2 Release view

A release can reference one base artifact and zero or more ordered deltas:

```
type IndexView struct {
    Base       artifact.Ref
    Deltas     []artifact.Ref
    Tombstones artifact.Ref
    Watermark  corpus.Watermark
}
```

The query runtime opens the view under one lease. Lexical and vector searchers query base and deltas, apply tombstones and logical-key supersession, normalize scores if necessary, and fuse segment candidates deterministically. The view is immutable even if a delta was built recently.

A new small change produces a new delta and a new release manifest. It does not mutate the prior active release. Activation can therefore switch atomically and rollback instantly.

18.3 Score comparability

Lexical scores from separately built segments may not be directly comparable because collection statistics differ. Options include:

- search base and delta separately and fuse ranks rather than raw scores;
- use a backend that maintains global statistics;
- rescore candidate documents against a shared global corpus model;
- compact frequently enough that delta bias is bounded and evaluated.

Rank fusion is pragmatic and aligns with existing RRF infrastructure. It can, however, overweight small delta segments. The release query policy must identify segment-fusion behavior, and optimization should include freshness strata so new content is neither suppressed nor unfairly promoted.

Vector similarity is usually more comparable when all vectors use the same embedding model and normalization. Approximate backends can still have segment-specific recall. Query each segment with sufficient k, merge by exact similarity where vectors are available, and evaluate against a clean exact view.

18.4 Delta size and compaction policy

Compaction triggers can depend on:

- number or byte size of deltas;
- tombstone ratio;
- query fan-out and p95 latency;
- ANN recall degradation;
- base age and source watermark lag;
- operational maintenance window;
- cost of continuing overlay search versus rebuild.

Compaction builds a new complete base at a fixed source barrier. It is verified against the overlay view before activation. Once active and drained, old segments can be retired under retention policy.

Compaction policy is operational when it preserves view semantics. Under approximate indexes and deadlines it may change ranking and latency; then it is part of release behavior or at least protected operational configuration.

18.5 Backend capability interface

A shared capability model might be:

```
type Capabilities struct {
    FullBuild          bool
    Upsert             bool
    Delete             bool
    SnapshotRead       bool
    FilterPushdown     bool
    DeterministicBuild bool
    ExactScores        bool
    Compact            bool
}

type Builder interface {
    Spec() IndexSpec
    Build(context.Context, BuildInput, EventSink) (artifact.Ref, error)
}

type DeltaBuilder interface {
    BuildDelta(context.Context, BaseDescriptor, []IndexChange, EventSink) (artifact.Ref, error)
```

```
}

type Opener interface {
    Open(context.Context, IndexView) (SnapshotSearcher, error)
}
```

`SnapshotSearcher` guarantees immutable view behavior for its lifetime. Product query code should not call point updates on it.

18.6 Full rebuild remains first-class

Incremental maintenance does not eliminate full builds. Full builds serve as:

- a correctness oracle;
- a compaction operation;
- recovery when lineage or cache integrity is uncertain;
- migration across incompatible schemas or backends;
- periodic defense against accumulated approximation drift.

The coordinator should support both from one `BuildSpec`. An incremental plan can fall back to full build when affected closure exceeds a threshold or backend capability is insufficient.

19. Durable build coordination and resumability

19.1 Coordinator responsibilities

The production coordinator owns:

- build intent registration and idempotency;
- source capture and barrier custody;
- impact-plan construction;
- stage scheduling and resource admission;
- immutable artifact and cache references;
- append-only build events and checkpoints;
- retry, quarantine, cancellation, and operator commands;
- verification, evaluation, and release registration;
- metrics and status projection.

It does not own product source meaning, retrieval quality metrics, or activation authority. Product adapters supply connectors, normalization policy, and evaluators. `ragkit` supplies the coordinator contracts and local implementation. An external scheduler can later drive the same state machine.

19.2 Build intent identity

A build intent identifies the desired semantic result:

```
type BuildIntent struct {
    Product          string
    SourceRequest    corpus.CaptureRequest
    BaseRelease      release.ID
    BuildSpec        BuildSpec
    QuerySpec        query.Spec
    AuxiliaryAssets  []artifact.Ref
    EvaluationPolicy eval.PolicyID
}
```

The intent ID excludes worker count, retry backoff, and machine placement. It includes every input that can alter release material or eligibility. Re-registering the same intent returns the existing run or terminal result.

A refresh intent and an optimization intent can share the same type but differ in locks. A content refresh changes source barrier while freezing build/query policy. An optimization candidate freezes source barrier while changing one declared asset. The two loops must not be conflated because their causal interpretation differs.

19.3 Stage DAG and progress

The coordinator derives a DAG of stage instances from impact plan. Progress is measured by semantic work units, not only percent. For example:

```
capture: barrier acquired
normalize: 8,402 / 8,402 source revisions
chunk: 8,397 / 8,397 admitted documents
represent.summary: 12,110 / 16,884 chunks, 9,332 cache hits
embed: 55,100 / 61,004 representations, 54,911 cache hits
lexical-index: sealed
vector-delta: building
verify: pending
```

Each count has a denominator fixed by the plan. Dynamic discovery creates a new plan version or explicit subplan, not a silently changing denominator.

Operator status should show blocked resources, retry storms, cost budget, source watermark, quarantines, and the last committed checkpoint. This information exists partially in current flow observations and design records but lacks a shared durable projection.

19.4 Scheduler model

The first implementation should be a fixed coordinator, not a general workflow language. RAG builds have known semantic stages and domain-specific verification. A local process with a durable SQLite event store and blob/artifact backend is sufficient for one active build. Cloud scheduling can invoke one coordinator job rather than creating a queue message for every chunk.

Within stages, `ragkit/flow` handles bounded concurrency and provider calls. This separation gives operational flexibility: the same coordinator can run locally, in a container job, or under a workflow service while preserving events and checkpoints.

19.5 Leases and fencing

A build lease prevents two coordinators from committing the same intent concurrently. It includes a fencing token. Artifact commits and checkpoint writes include the token; stale workers cannot publish after lease loss.

Long provider calls may finish after cancellation or lease expiry. Their content-addressed outputs can be stored in a neutral cache if they verify, but they cannot advance the cancelled build without a valid fence. This salvages expensive work without violating run custody.

19.6 Resume equivalence

Let event prefix P reduce to coordinator state s. Resume reconstructs pending work from $(intent, plan, P)$ and continues with suffix U. The terminal release should be equivalent to uninterrupted execution with the same provider outputs:

$$reduce(P \cdot U) = runFromStart(intent, \omega).$$

Property tests can interrupt after every event boundary, reopen the store, and compare terminal manifests. Tests should also duplicate commands and provider completions to verify idempotence.

19.7 Publication and activation separation

The coordinator can register a release as verified and eligible. It can produce a promotion plan containing expected current release, candidate release, gate report, rollout cohort, and rollback recommendation. A distinct activation authority applies it.

This separation supports both scheduled content refresh and optimization. A routine content refresh may have an automated policy that activates after integrity, freshness, and regression gates. A relevance optimization may require human review. Both use the same activation API.

20. Freshness, time, and temporal correctness

20.1 Four clocks

RAG freshness is not one timestamp. At least four clocks exist:

1. **Source effective time:** when the information became true in the source domain.
2. **Connector observed time:** when the RAG system saw the revision.
3. **Release activation time:** when new queries began using it.
4. **Query time:** when the user asked.

A fifth clock, frontend presentation time, matters for live systems. Lag can be decomposed:

$$staleness = (t_{observed} - t_{effective}) + (t_{activated} - t_{observed}) + (t_{query} - t_{activated_{content}}).$$

The last term is zero for content captured in the active release and positive for newer unseen revisions.

20.2 Freshness objectives

A product can define:

- maximum source-to-observation lag;
- maximum observation-to-activation lag;
- maximum active-release watermark age;
- percentile objectives by source class;
- emergency correction objectives for security-sensitive content;
- maximum cross-source skew inside a release.

A static documentation corpus may tolerate daily refresh. Product prices or inventory may require structured live tools instead of index refresh. The architecture should not force rapidly changing authoritative facts into a text index when a structured source has better temporal semantics.

20.3 Watermark propagation

Every derived artifact carries the source barrier vector. A release exposes watermarks to the query trace. Answers and source cards can optionally display “knowledge through” time when users need temporal context.

A query can impose freshness requirements. If the active release watermark is older than requested policy, the runtime can abstain, invoke a live structured/connected source, or route to a fresher release. It should not silently answer from stale material while claiming current authority.

20.4 Temporal evaluation

Random train/test splits over a static evaluation set do not measure corpus evolution. Evaluation should include temporal holdouts:

- queries whose relevant content was added after the optimization training window;
- updates that modify previously correct facts;
- deletions and retractions;
- late-arriving revisions;
- source-policy changes;
- queries issued during base-plus-delta periods and after compaction.

Metrics include time-to-retrievable, time-to-answerable, stale-answer rate, deleted-content exposure, and freshness-conditioned relevance. The candidate and incumbent should be evaluated at the same source barriers for relevance comparisons; refresh performance is evaluated as a separate time process.

20.5 Session consistency under refresh

A per-turn lease ensures one turn does not mix epochs. Across turns, the conversation can move to a newer release. Prior assistant messages and evidence remain historical facts from their original releases. Follow-up query rewriting may reference those messages. The trace should preserve release lineage so diagnosis can reconstruct why a newer turn differs.

A product can choose to notify the model that the knowledge release changed between turns or simply treat prior evidence as quoted conversation context. For regulated or administrative use, explicit epoch markers are preferable.

20.6 Emergency invalidation

Some deletions cannot wait for normal draining. A registry can mark a release `Revoked` for a policy scope. New acquisitions fail or resolve to a safe prior/new release. In-flight requests check revocation before remote disclosure and final emission. This is stronger than immutable draining and should be reserved for security or legal incidents because it can break ongoing turns.

Emergency invalidation must be auditable: affected source keys, releases, actors, reason, start/end time, and queries blocked or completed. Physical artifact erasure follows separate policy.

Part IV. Retrieval optimization across indexing and querying

21. The optimization field

21.1 Optimization as controlled intervention

A RAG candidate is not an arbitrary configuration file. It is a controlled intervention on a behavior-complete release specification. Let baseline specification be θ_0 and intervention ι produce $\theta_1 = \iota(\theta_0)$. The intervention declares:

- the parameter or asset changed;
- the causal hypothesis;
- the dependency closure invalidated;
- the equivalence properties expected to hold;
- the metrics expected to improve;
- hard safety and operational constraints;
- required evaluation fidelity;
- rollback and incompatibility implications.

This declaration makes "one mutation" meaningful. Changing a chunker implementation and its size in one opaque file is not necessarily one semantic mutation. Conversely, changing one representation prompt can require rebuilding millions of downstream vectors; it remains one causal intervention.

21.2 Parameter layers

The field spans the following layers.

Source and policy. Connector selection, admission, access scopes, source roles, normalization, metadata extraction, redaction, and source weighting.

Chunking. Algorithm, boundaries, target size, overlap, minimum size, heading and AST context, hierarchical parent links, and stable-anchor strategy.

Representations. Raw text, breadcrumbs, contextual text, summaries, generated questions, entities, product fields, prompts, models, and number of generated views.

Embedding. Model, dimension, normalization, distance metric, batching, provider, and quantization.

Index. Lexical analyzers, field boosts, filters, exact or approximate vector backend, HNSW construction/search parameters, partitions, overlays, and compaction.

Query interpretation. Intent classification, routing, synonym expansion, multi-query, HyDE, metadata extraction, channel enablement, per-channel depth, and fallback.

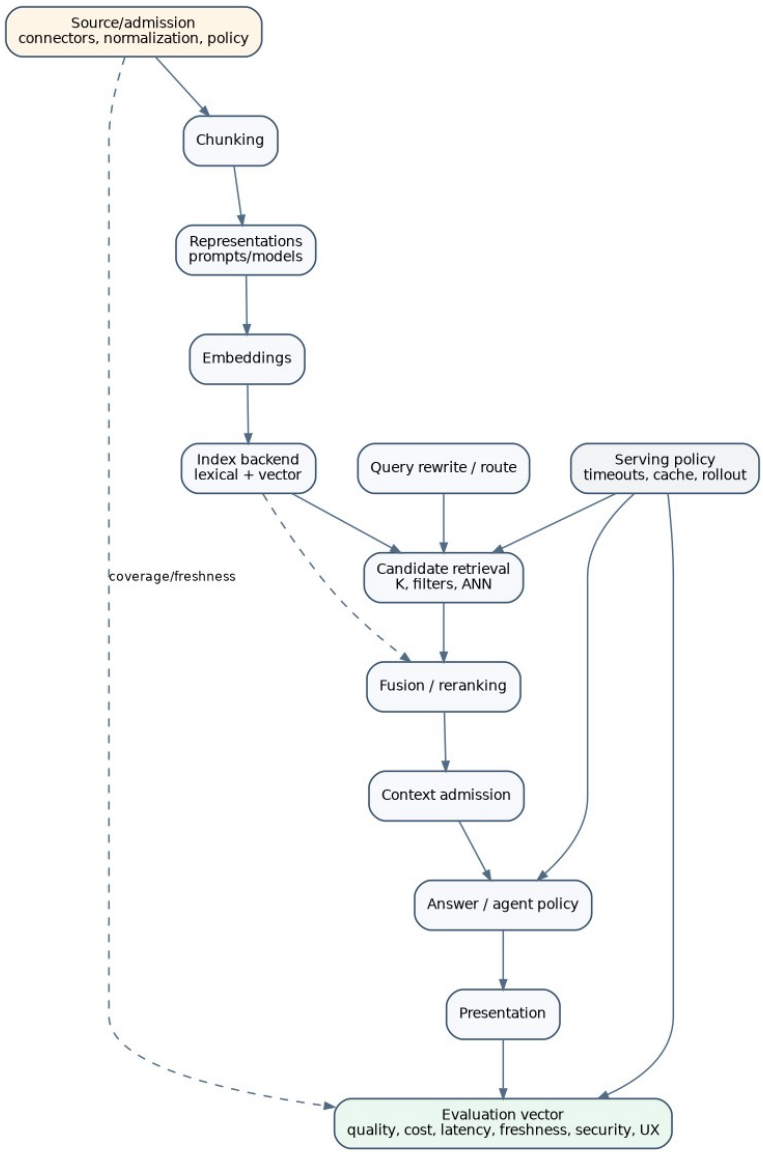
Ranking. Representation collapse, fusion constant and weights, score normalization, deduplication, diversity, reranker model, pool, blend, and failure policy.

Context. Count and token budget, source diversity, redundancy, ordering, summarization, and conversation carryover.

Answer and agent. Prompt, provider, grounding contract, tool schemas, tool descriptions, iteration budget, repair policy, and abstention.

Serving. Caches, concurrency, timeout partition, queue policy, rate limits, release pinning, and cohort routing.

Presentation. Source projection, choice generation, widget policy, customer/developer views, and conflict suppression.



The RAG optimization space is a dependency graph from source semantics to user outcome.

21.3 Intervention classes

Interventions should be classified before evaluation.

A **semantics-preserving operational intervention** claims to preserve protected outcomes and traces except resource fields. Examples may include worker count or batch size when there are no deadline effects and provider batching is semantically stable.

An **approximation-changing intervention** changes allowed ranking error, such as exact-to-HNSW search, quantization, or reduced candidate depth. It requires oracle-relative evaluation and workload/scale testing.

A **relevance-changing intervention** changes retrieval ordering without changing source knowledge: fusion weights, reranker, query expansion, or context admission.

A **knowledge-changing intervention** changes what information is represented: source admission, normalization, chunking, generated representations, or embedding model.

A **policy/security intervention** changes authorization, disclosure, redaction, or evidence-kind rules. It requires security review and cannot be approved solely by relevance gains.

A **user-outcome intervention** changes answer, tool, or presentation behavior. It requires answer/session/frontend evaluation.

One candidate can belong to several classes. The strictest required gate wins.

21.4 Dependency closure

Represent parameters and artifacts as a directed acyclic dependency graph. If node a changes, every reachable derived node is invalid unless a verified semantic cache key proves reuse. For example:

```
normalization -> documents -> chunks -> representations -> embeddings -> vector index
                          \-> lexical index
query rewrite -> channel rankings -> fusion -> rerank -> context -> answer -> presentation
index artifacts --------------------^                          ^
structured fact DB -------------------------------------------|
```

The closure determines build cost and evaluation reuse. A fusion-weight candidate can reuse channel rankings only if every upstream query input is frozen. A chunk-size candidate cannot reuse retrieval rankings or answer contexts. A reranker-pool candidate can reuse fused rankings and hydrated candidates only if the pool's maximum depth was retained.

This graph should be machine-readable. `ragopt` can use it to validate candidate snapshots and schedule shared intermediate work without learning RAG semantics.

21.5 Objectives and constraints

Let metric vector be:

$$M(R)=\left(Q_{ret},Q_{ans},Q_{ground},Q_{ux},F,L,C,K,U,S\right),$$

where retrieval, answer, grounding, and UX quality are followed by freshness, latency, monetary cost, capacity/resource use, reliability/uptime, and security/privacy. Some coordinates are maximized, some minimized, and some are predicates.

Security, lineage integrity, and release consistency are hard constraints. Reliability and latency usually have service objectives. Quality is compared only among candidates satisfying those constraints. A single weighted scalar hides unacceptable trade-offs and changes meaning when metric scales drift.

Use lexicographic gates and Pareto analysis:

1. invariants and security;
2. release/build integrity;
3. reliability and freshness floors;
4. latency, cost, memory, and capacity budgets;
5. non-inferiority on protected quality strata;
6. improvement on target metric with uncertainty;
7. human/product review of the Pareto trade-off.

21.6 Candidate object

A RAG-specific candidate layer on `ragopt` can be:

```
type Intervention struct {
    ID                string
    Class             []InterventionClass
    Target            ParameterRef
    Before            artifact.Ref
    After             artifact.Ref
    Hypothesis        string
    Invalidates       []NodeID
    Preserves         []PropertyID
    RequiredFidelity  Fidelity
    Constraints       []ConstraintID
}

type Candidate struct {
    ParentRelease release.ID
    SourceBarrier corpus.BarrierVector
    Intervention Intervention
    Spec          release.Spec
}
```

`Candidate` is immutable and behavior-complete. It does not contain an already-built index unless construction has completed; the build output references the candidate ID.

22. Optimizing the indexing phase

22.1 Index optimization questions

Indexing optimization asks at least five distinct questions:

1. What source material and normalized form should be retrievable?
2. What retrieval units and derived views best expose relevant evidence?
3. What vector and lexical representations preserve useful similarity and exact matching?
4. What backend and physical parameters meet quality, capacity, freshness, and cost objectives?
5. How can the index be maintained as sources change without violating release semantics?

The first three are semantic and relevance questions. The fourth mixes approximation and operations. The fifth is temporal and systems behavior. Treating all five as one grid search produces uninterpretable results.

22.2 Corpus diagnostics before model evaluation

Cheap corpus diagnostics should precede expensive query evaluation. `ragkit/indexbundle/stats` already measures chunk distributions and signals. Extend this with:

- admitted/excluded/quarantined source counts by connector, role, scope, and content class;
- document and chunk size distributions;
- near-duplicate and template-furniture ratios;
- empty or low-information chunks;
- orphan lineage and metadata completeness;
- representation count and cost per source class;
- embedding norm and duplicate-vector diagnostics;
- access-filter selectivity;
- source update and deletion frequency;
- expected delta and compaction sizes.

Diagnostics are not quality metrics, but they reject obviously invalid candidates and explain downstream changes. A chunker that creates 20% empty chunks should not proceed to LLM judging.

22.3 Chunking experiments

Chunking experiments need labels at a compatible granularity. Document-level relevance can hide a wrong-chunk failure. Evaluation sets should contain relevant chunk spans or evidence requirements where possible, and should distinguish “any chunk from document” from “the chunk that contains the answer.”

Useful chunking metrics include:

- answer-bearing span containment;
- boundary loss and context fragmentation;
- retrieval recall at chunk and document levels;
- redundant chunks per query;
- context-token efficiency;
- citation specificity;
- update amplification: chunks invalidated per source edit;
- stable-identity retention across revisions;

- build and storage cost.

Chunk size interacts with representation and context policy. Larger chunks can improve answer-span containment but reduce lexical specificity and context capacity. Generated summaries can compensate for large raw chunks in retrieval while evidence still hydrates the full source chunk. Experiments should test these interactions rather than choosing chunk size in isolation.

22.4 Representation experiments

Representations should be evaluated as retrieval views, not as additional evidence. A candidate adds or changes one kind and measures:

- incremental relevant-chunk recall;
- rank changes by query intent;
- false-positive patterns;
- collapse behavior when several views hit one chunk;
- generation and embedding cost;
- storage and index fan-out;
- source-update rebuild amplification;
- provider reproducibility and cacheability.

RAG-TTC's raw, summary, contextual, question, and entity forms provide a ready experimental field. A factorial or sequential design can estimate interactions between representation kind and query class. Generated questions may help natural-language factual queries but harm product SKU lookups; entity forms may do the reverse.

Prompts and model versions are first-class candidate assets. A generated representation corpus should be retained so candidate differences can be inspected and replayed without another provider call.

22.5 Embedding experiments

Embedding comparison requires the same corpus snapshot and evaluation cases. Metrics include vector-channel recall, hybrid marginal gain over lexical, memory, index build time, query embedding latency, provider cost, and drift under source classes. Dimensions affect storage and ANN performance.

A model change invalidates every vector. Multi-fidelity evaluation can sample source/query strata before full embedding. However, a sampled candidate cannot be promoted without a complete build and full integrity verification.

For provider-hosted embeddings, rate limits and batching affect build duration. Batch size may change outputs for poorly behaved APIs; equivalence tests should confirm vector stability. Retained vectors are material artifacts even when semantic model identity is uncertain.

22.6 Lexical index experiments

Lexical optimization covers analyzers, tokenization, stemming, synonyms, field structure, and boosts. Exact SKU, product, schema, and administrative terms often make lexical search indispensable. Query-time synonyms, as in GEC, are easy to iterate and do not require rebuilds. Index-time synonyms can improve analysis consistency but create a larger invalidation scope.

Evaluation should separate exact-identifier, terminology, natural-language, and long-question strata. A title boost can improve navigational queries while harming body-specific evidence. Keep channel rankings and contributions for diagnosis.

Access-scope filter pushdown is both a security and relevance feature. Backend experiments should include authorized top-k completeness and latency at realistic filter selectivity.

22.7 ANN optimization

The RAG-TTC HNSW bakeoff provides the base pattern: exact oracle, recall threshold, latency gate, and reproducibility check. A production bakeoff extends it with:

- build time and peak memory;
- steady-state memory and disk;
- query concurrency and tail latency;
- filter selectivity;
- insert/delete/update sequence behavior;
- recall by query and source strata;
- recall after delta accumulation and compaction;
- crash/reopen behavior;
- deterministic or bounded-nondeterministic rebuild;
- capacity projection at expected corpus growth.

Search parameters can be serving configuration rather than build configuration if the backend supports them dynamically. They still belong to release query identity when they alter rankings.

22.8 Maintenance optimization

Refresh strategy is itself optimizable. Candidate policies change delta threshold, compaction cadence, stage concurrency, or connector polling. Objectives include freshness lag, build cost, query fan-out, tail latency, and operational reliability.

A maintenance-policy experiment must preserve logical source view and release consistency. It can replay a historical change stream through candidate policies and compare activation timelines. This is a discrete-event simulation before production canary.

The outcome is not one index score. It is a time series of active-release watermarks, build states, costs, and query performance under changing overlay size.

23. Optimizing the query phase

23.1 Query rewriting and routing

A query rewrite can expand terms, generate variants, infer metadata filters, produce a hypothetical answer, or classify intent. Its evaluation must retain original query and generated artifacts. Failure policy matters: `ragkit` can fall back from multi-query or HyDE to the original query. The evaluator should report both intended-route and degraded-route performance.

Routing candidates change which channels, indexes, structured tools, or connected sources run. Evaluate routing accuracy and end-to-end outcome, not classifier accuracy alone. A route can be "wrong" by label yet produce the best answer; conversely, a correct intent label can select a weak retrieval plan.

GEC synonyms are a bounded lexical rewrite and fit a cheap paired trial. Garden's intent routes affect representation filters, source roles, connected augmentation, and structured-first behavior; they require session and presentation evaluation.

23.2 Candidate depth and overfetch

Per-channel k, fusion depth, rerank pool, and postfilter overfetch interact. Increasing depth can improve recall while increasing latency, hydration, remote disclosure, and reranker cost. Postfilter overfetch is not a substitute for authorized top-k semantics unless completeness is established.

Evaluate recall curves as a function of depth, not one point. Retain channel rankings to estimate whether additional depth has marginal value. For remote reranking, compute cost and latency per pool item and measure the head/tail transition.

A query plan can allocate depth adaptively based on confidence or intent. Adaptive policies require trace-level evaluation because average k hides expensive subgroups.

23.3 Fusion optimization

Fusion parameters are low-cost to sweep when channel rankings are frozen. GEC's current grid is therefore a sensible first tool. A stronger design adds:

- paired per-query metric differences;
- confidence intervals;
- query-stratum effects;
- sensitivity surfaces, not only the winning point;
- stability under evaluation-set bootstrap;
- constraints on channel disappearance or dominance;
- holdout confirmation after parameter selection.

RRF constant and weights can interact with channel depth and collapse policy. A candidate generated from frozen top-k rankings cannot estimate behavior at a larger channel depth unless deeper rankings were retained.

Learned fusion is possible, but the training/evaluation split and feature lineage become more complex. A deterministic parameterized fusion remains easier to audit for administrative products.

23.4 Reranker optimization

A reranker candidate includes model, document-text composition, candidate pool, score/rank blend, timeout, provider data policy, and failure behavior. Evaluate:

- reranked nDCG and MRR;
- answer-bearing evidence movement;
- regressions by source role and query class;
- provider latency, cost, and failure rate;
- fallback quality and frequency;
- remote disclosure volume;
- calibration of score gaps if used adaptively;
- security filtering before provider invocation.

GEC title-prefixes rerank text because searchable representations may carry breadcrumbs. This composition should be a named, versioned function. Otherwise a title formatting change silently invalidates reranker caches and behavior.

Fail-open behavior must be visible to the evaluation arm. Offline tests should inject provider failures and verify the declared fallback. Production metrics should distinguish `reranked` from `fused_fallback` outcomes.

23.5 Diversity and evidence selection

Top-ranked chunks can be redundant. Evidence selection can enforce document, source, topic, or product diversity. Metrics include answer coverage, unique sources, redundancy, context tokens, and citation precision. Diversity is not always beneficial: multiple chunks from one long source may be necessary for a multi-part answer.

A useful candidate is a constrained selection algorithm over a retained candidate pool. Because it changes only admission, it can reuse retrieval and reranker outputs. Answer evaluation is required because retrieval metrics over the full pool do not measure the context actually shown to the model.

23.6 Context budget

The current `ragkit` policy admits whole chunks under count and rune limits. Optimize count, token budget, ordering, and optional compaction. Whole-chunk admission preserves exact evidence but can waste context when chunks are large. Extractive snippets can improve density but create new evidence lineage obligations.

Evaluation should measure:

- relevant evidence included;
- unsupported distractor volume;
- answer correctness and faithfulness;
- citation specificity;
- model input tokens and latency;
- truncation/abstention rate;
- sensitivity to chunk order.

A context candidate must be paired with the same retrieved pool. Otherwise retrieval variation confounds admission effects.

23.7 Answer and agent policy

Prompts, grounding contracts, model providers, tool descriptions, and iteration limits are query-phase behavior. They are not retrieval parameters, but they determine whether retrieved evidence becomes a useful product outcome.

Agent optimization should count tool calls, repeated searches, route changes, evidence reuse, structured-query success, time to terminal result, and invalid final attempts. A candidate that improves final judge score by doubling tool calls may violate latency and cost constraints. Garden's multi-turn calibration and RAG-TTC's tool-loop artifacts are the correct evaluation level.

Tool descriptions are security- and behavior-relevant. A self-optimizer may propose text changes, but the candidate must be immutable, evaluated against held-out sessions, and promoted through the same release path.

24. Joint index-query optimization

24.1 Why independent tuning fails

Index and query parameters interact. A contextual representation has no effect if the route filters it out. A smaller chunk size can require larger channel depth and context diversity. An HNSW recall loss may be masked by lexical fusion on one workload and exposed on another. A reranker can compensate for noisy high-recall retrieval, changing the optimal first-stage k. Query expansion can reduce the value of generated question representations.

Sequentially optimizing each layer at a fixed setting can converge to a poor local design. A full Cartesian search is usually too expensive. The solution is structured experimental design guided by the dependency graph and diagnostics.

24.2 Hierarchical search strategy

A practical strategy has five levels.

1. **Feasibility screening.** Reject candidates violating type, capability, security, storage, or cost bounds.
2. **Upstream representation screening.** Evaluate source/chunk/representation/embedding candidates with cheap retrieval tests and a fixed robust downstream policy.
3. **Downstream policy tuning.** For promising index candidates, tune route, depth, fusion, rerank, and context using retained intermediate rankings.
4. **Interaction trials.** Test selected cross-layer combinations where diagnostics predict interaction.
5. **End-to-end confirmation.** Build complete releases and run answer, session, shadow, and canary gates.

This resembles nested optimization rather than one flat loop. `ragopt` campaigns can represent each level as a set of immutable candidates and linked runs.

24.3 Shared intermediates

The dependency DAG enables common-subexpression reuse across candidates. Candidates sharing normalized corpus and chunking can share chunk artifacts. Candidates sharing representation text can share embeddings. Fusion sweeps share channel rankings. Context-policy candidates share reranker outputs.

Reuse must be semantic, not path-based. Each intermediate is keyed by operation spec and canonical inputs. The campaign records which candidate uses which artifact. This reduces cost without weakening isolation.

24.4 Factorial and sequential designs

For a small number of interacting factors, a factorial or fractional-factorial design estimates main effects and interactions more efficiently than one-factor-at-a-time tuning. Example factors might be chunk size, contextual representation, vector depth, and rerank pool.

For larger spaces, sequential model-based optimization can propose candidates, but its surrogate model must respect categorical parameters, constraints, and heterogeneous fidelity. The proposer must not see the final holdout labels used for promotion. It receives training-run summaries and can request bounded interventions.

The system should retain negative trajectories. They reveal interaction surfaces and prevent repeated failed ideas.

24.5 Multi-task and stratum effects

RAG workloads are mixtures: exact product lookup, policy question, schema documentation, comparative shopping, procedural guidance, and conversational follow-up. Aggregate optimization can sacrifice a small critical stratum for a large easy one.

Define protected strata and minimum sample coverage. Report per-stratum paired effects and worst-case regressions. Security and negative-policy cases should not be mixed into positive retrieval hit rate. They have different outcome semantics.

Candidate selection can seek Pareto improvement across strata or impose non-inferiority margins. Product owners decide acceptable trade-offs; the optimizer supplies evidence.

24.6 Source drift and robust optimization

The query distribution and corpus both drift. A candidate overfit to one snapshot may fail after content changes. Robust evaluation uses multiple historical source barriers and temporal query windows. Index candidates are rebuilt or replayed at each barrier; query-only candidates can often reuse historical rankings.

Optimize expected performance plus tail-risk terms, such as worst-decile source class or conditional value at risk over query strata. Avoid claiming universal superiority from one static benchmark.

25. Evaluation, statistics, and evidence

25.1 Evaluation units

The unit must match the semantic level.

- **Document/chunk unit:** corpus and derivation diagnostics.
- **Query unit:** ranked retrieval and evidence sufficiency.
- **Answer unit:** generated answer, citations, abstention, and cost.
- **Turn unit:** tool trajectory and terminal projection.
- **Conversation unit:** multi-turn consistency, choices, and session release behavior.
- **Release-time unit:** freshness, build reliability, activation, and rollback.
- **Frontend session unit:** snapshot/reconnect/stream convergence and user interaction.

Do not aggregate across incompatible units without preserving denominators.

25.2 Retrieval metrics

Use recall, precision, MRR, nDCG, hit rate, and coverage under explicit target granularity. Add evidence-oriented metrics:

- answer-bearing span recall;
- source-role and scope coverage;
- redundant evidence ratio;
- authorized top-k completeness;
- first relevant latency when streaming retrieval;
- stale or superseded evidence rate;
- representation contribution and collapse diagnostics.

A query with no positive relevance target should not be counted as a retrieval miss. Negative-policy cases measure false disclosure or false answer behavior under separate metrics.

25.3 Answer and grounding metrics

Answer evaluation includes correctness, completeness, faithfulness, citation precision, citation coverage, abstention appropriateness, style/format contract, and structured choice/widget correctness. Structural validators run before model judges.

LLM judges are measurements with error, not ground truth. RAGAS decomposes context relevance, faithfulness, and answer relevance; ARES uses synthetic training and a small human-labeled set for statistically corrected evaluation. These frameworks support decomposition, but product-specific labels and audits remain necessary.

Judge failures must remain in denominators. Invalid output, timeout, empty statements, or parse failure is an explicit missing/failed measurement, not a zero or silent success. Pair candidate and incumbent judge calls and repeat stochastic evaluations.

25.4 Paired uncertainty

For metric m, compute per-case paired differences δ_i. Report mean/median, confidence interval, win/tie/loss counts, and stratum effects. A nonparametric paired bootstrap is appropriate for many IR and answer metrics. For bounded binary outcomes, exact or Wilson-style intervals can supplement.

Do not infer improvement from aggregate means alone when a few cases dominate. Retain per-case artifacts. Multiple stochastic repeats can be nested within case; analysis should distinguish within-case model variance from between-case workload variance.

A gate can require:

$$Pr\left(\Delta m > -\epsilon \vee data\right) \geq 1 - \alpha$$

for non-inferiority, and a separate target improvement criterion. The implementation can use bootstrap confidence rather than a Bayesian model; the semantic statement is the same.

25.5 Multiple comparisons

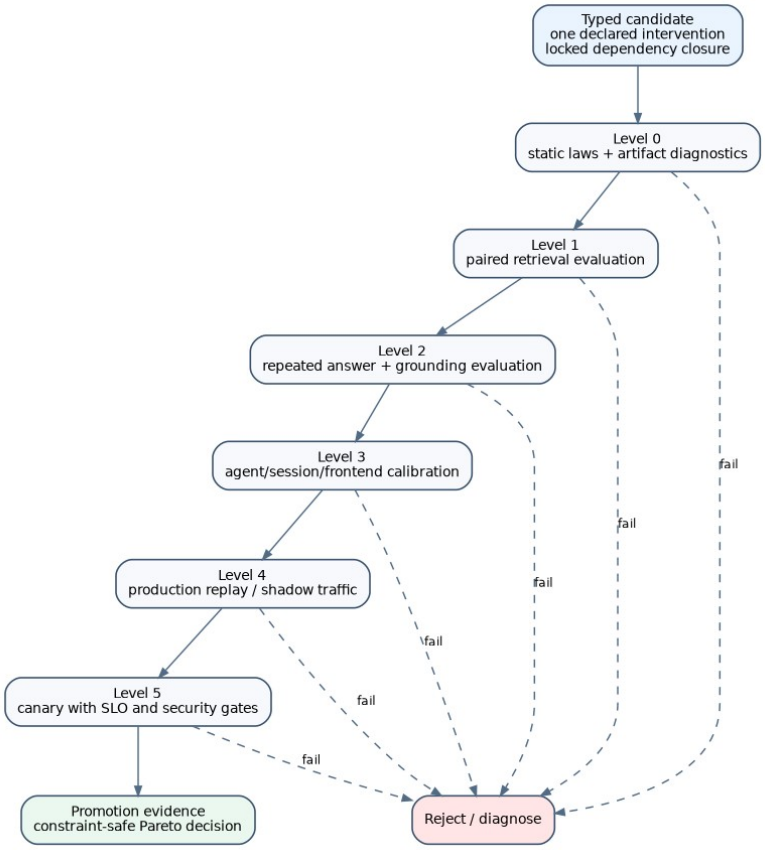
Large sweeps create selection bias. The best of many noisy candidates is likely overestimated. Use a development set for exploration, a validation set for selection, and a final holdout for promotion. Sequential online tests require alpha-spending or equivalent error control.

A candidate proposer must not have unrestricted access to final holdout results. `ragopt` can enforce roles: exploratory runs return detailed metrics; promotion runs return gate decisions and limited diagnostics until the campaign closes.

25.6 Multi-fidelity evaluation

Evaluation should proceed from cheap to expensive:

1. static schema, lineage, capability, and security laws;
2. corpus/index diagnostics;
3. paired retrieval evaluation;
4. repeated answer and grounding evaluation;
5. agent/session/frontend calibration;
6. production replay or shadow traffic;
7. canary release with SLO and security monitoring.



A candidate advances through increasing evaluation fidelity and cost.

A candidate that fails at one level is diagnosed rather than automatically discarded forever; a revised intervention becomes a new candidate. The run artifacts remain immutable.

25.7 Operational metrics

Measure distributions, not only means:

- end-to-end latency and stage latency p50/p95/p99;
- queue and resource-admission time;
- provider timeout and fallback rates;
- cache hit rate by semantic stage;
- request cancellation and partial-output rate;
- build duration, retry, quarantine, and activation lag;
- memory, disk, CPU, and provider cost;
- release lease count and drain duration;
- frontend hydration, reconnect, duplicate, and gap rate.

Latency must specify boundaries. RAG-TTC's ANN bakeoff excludes query-embedding latency, which is correct for backend isolation but not for end-to-end claims.

25.8 Security evaluation

Security gates include:

- no unauthorized candidate returned;
- no unauthorized text disclosed to remote stages;

- source and structured-data policy respected;
- prompt-injection and content hygiene tests;
- deleted/revoked content inaccessible under declared policy;
- tool arguments cannot set server-owned scopes;
- release and evidence lineage complete;
- logs and traces do not contain prohibited payloads;
- frontend customer mode does not expose developer-only provenance.

These are mostly invariant tests and adversarial cases, not scalar optimization metrics.

25.9 Promotion evidence

A promotion report contains:

- candidate and parent release IDs;
- exact intervention and dependency closure;
- source barriers and evaluation-set versions;
- all run IDs and native artifacts;
- gate results in order;
- paired effects with uncertainty and strata;
- operational and security results;
- known limitations and unmeasured risks;
- rollout cohort and monitoring plan;
- rollback release and trigger thresholds;
- human approvals where required.

The report proposes activation through the registry. It does not mutate product configuration files in place.

26. Online evaluation and safe evolution

26.1 Offline is necessary, not sufficient

Offline evaluation provides labels and counterfactual control but cannot reproduce production load, provider incidents, user distribution, frontend behavior, or live source drift. Online evidence is required for serving changes and high-impact quality candidates.

Online evaluation begins with replay or shadowing. The candidate receives copies of production requests under the same source/privacy policy, but its outputs are not shown to users. This measures latency, failures, traces, and judgeable outcomes. It does not measure user interaction effects.

26.2 Shadow execution

Shadow execution must not double prohibited remote disclosure or cost without policy. A shadow plan can reuse incumbent authorized evidence for downstream-only candidates, but not when the candidate changes retrieval. The dependency graph determines valid sharing.

Shadow traces link incumbent and candidate by comparison ID. Candidate failures never affect the production response. Resource isolation prevents shadow load from degrading incumbent SLOs.

26.3 Canary releases

A canary assigns a stable cohort to candidate release. Hard monitors include security violations, error rate, p95/p99 latency, provider spend, stale-answer rate, and frontend terminal completion. Soft monitors include answer judges and user signals.

Automated rollback is another compare-and-swap activation to the prior release when hard thresholds trigger. The canary release remains immutable and retained for diagnosis. Rollback does not erase its events.

26.4 A/B and interleaving

For user-facing ranking, interleaving can compare preferences with fewer samples, but RAG answers and agents complicate attribution. A/B at conversation level is usually simpler. Assignment must remain stable within conversation to avoid mixed policy.

User signals are noisy and can reward verbosity or presentation rather than correctness. Combine them with grounding and safety gates. Garden's choices and widgets provide product-specific interaction outcomes that can be measured directly, such as choice completion or product-card engagement, subject to privacy policy.

26.5 Continual optimization without self-mutation

A safe continual loop is:

1. detect a diagnostic opportunity from traces;
2. construct a bounded immutable intervention;
3. build a candidate release against a frozen source barrier;
4. evaluate through required fidelities;
5. produce promotion evidence;

6. activate through independent authority;
7. monitor and retain rollback path;
8. incorporate results into future proposals.

The system never lets a running model edit the active prompt, synonym file, tool description, or index in place. “Self-optimization” means automated proposal and experiment scheduling under fixed boundaries, not self-modifying production behavior.

26.6 Content refresh versus optimization

A content refresh changes source barrier and should freeze behavior policy. Its question is whether the new source state can be represented and served without regression and within freshness objectives.

An optimization changes behavior policy and freezes source barrier. Its question is causal improvement. After promotion, the optimized behavior can be applied to newer source barriers through normal refresh.

Combining both changes in one candidate prevents attribution. Emergency cases may require it, but the report must state that quality differences cannot be assigned to one cause.

26.7 Learning from production failures

Production traces can generate evaluation cases, but selection must avoid leaking the same case into final promotion verdicts. A failure-review pipeline should:

- identify candidate cases from immutable traces;
- redact and authorize them;
- assign labels through product review;
- add them to a development or future evaluation version;
- record which releases and campaigns had already seen them.

This yields a growing diagnostic suite without rewriting historical results.

Part V. RAG in production

27. Serving APIs and frontend contracts

27.1 The service boundary

The shared production boundary should expose semantic operations rather than an HTTP framework. Product servers already have mature conversation, authentication, persistence, tool, and WebSocket stacks. `ragkit` should fit beneath them.

A minimal service surface is:

```
type Service interface {
    Search(context.Context, SearchRequest) (SearchOutcome, error)
    Answer(context.Context, AnswerRequest, EventSink) (AnswerOutcome, error)
    StartAgentTurn(context.Context, AgentRequest, EventSink) (TurnOutcome, error)
}
```

The methods are separate because their terminal conditions differ. Each request contains product scope, subject context, conversation/turn IDs, query input, deadline, and optional release-selection constraints. The caller cannot pass arbitrary access scopes as model-controlled fields; product authorization constructs `SubjectContext`.

The service internally acquires a release lease or accepts a lease supplied by a conversation runtime. The outcome always includes release ID and trace summary. An ordinary Go error represents failure to perform the API contract itself; domain failures are typed outcomes so they remain observable and evaluable.

27.2 Search outcome

A direct search outcome should contain:

```
type SearchOutcome struct {
    Status       SearchStatus // complete, degraded, failed, cancelled
    ReleaseID    release.ID
    QueryPlanID  query.PlanID
    Hits         []EvidenceHit
    Warnings     []Warning
    Trace         trace.Ref
    Timing        Timing
}
```

`EvidenceHit` carries stable chunk/source revision, rank, finite score, channel contributions, evidence kind, authorization certificate reference, and presentation metadata. It does not copy every source payload by default; hydration policy determines what is returned to the product.

A degraded status distinguishes fallback from intended behavior. This lets GEC preserve its fail-open reranker while making it measurable.

27.3 Answer outcome

An answer outcome contains admitted evidence and a validated projection:

```
type AnswerOutcome struct {
    Status          AnswerStatus
    ReleaseID       release.ID
    EvidenceSession EvidenceSessionID
    Answer           ValidatedAnswer
    Evidence         []EvidenceRef
    Warnings         []Warning
    Trace             trace.Ref
    Usage             Usage
}
```

The answer can be text, a typed JSON contract, choices, or a product-specific envelope. Shared code owns generic grounding and citation relationships. Product code owns widget schemas and domain fields.

The `EventSink` receives lifecycle and presentation events. It should be idempotent by event ID and return an error when durable emission fails. The product decides whether a failure to record an observation is fatal; the shared plan can label the requirement.

27.4 Agent-turn boundary

An agent request includes conversation snapshot, allowed tool capabilities, release lease policy, model profile, and bounded budgets. Product code supplies tool implementations and server-owned authorization. The shared RAG runtime supplies search tools that are release- and evidence-session-aware.

The agent result retains the complete typed trajectory or a reference to it. Tool results carry causation and release IDs. A model-generated tool argument cannot override product scope, release, or provider data policy.

RAG-TTC's current `SearchTool` can implement this interface with little semantic loss. Its route table becomes a release query asset; its evidence ledger becomes a shared evidence session; connected augmentation remains a product/provider adapter.

27.5 Release resolution API

```go
type Resolver interface {
    Acquire(context.Context, AcquireRequest) (*Lease, error)
}

type AcquireRequest struct {
    ProductScope   string
    Subject        SubjectContext
    ConversationID string
    TurnID         string
    CohortKey      string
    Pinning        PinningPolicy
}

type Lease struct {
    ID() release.ID
    Manifest() release.Manifest
    Runtime() RuntimeResources
    Close() error
}
```

`RuntimeResources` exposes typed searchers, evidence stores, validators, structured snapshots, and policy only through immutable interfaces. It should not expose artifact paths for product code to reopen independently, which would permit mixed releases.

27.6 Transport adapters

HTTP, gRPC, CLI, model tools, and WebSockets are adapters. A direct REST endpoint can return `SearchOutcome`. A chat server can translate lifecycle events to its timeline. A model tool can serialize a bounded subset of evidence. A CLI can print trace details.

Adapters must preserve IDs and status. They should not convert degraded success into ordinary success or drop release lineage. Public/customer adapters may redact internal trace fields while preserving a safe trace reference.

27.7 Frontend event contract

A RAG-aware frontend event should include:

- conversation and turn identity;
- event and entity versions;
- release ID;
- semantic kind: retrieval started/completed, source admitted, answer delta, answer terminal, choices, widget, warning, cancellation;
- customer-safe payload;
- optional developer projection reference;
- causation and correlation IDs.

The server can expose sources before answer completion or only at terminal validation. That is a product policy and should be consistent. Source cards must reference the turn's release, not current active release.

27.8 Snapshot API

A snapshot response contains:

```
type Snapshot struct {
    StreamID       string
    ThroughOrdinal uint64
    Entities       []EntitySnapshot
    SchemaVersion  string
}
```

The subscribe request supplies the last applied ordinal. The server either sends a suffix or a new snapshot plus suffix. The client discards duplicate event IDs and events at or below the snapshot boundary.

GEC's current manager already sends the last snapshot ordinal and buffers events around hydration. The new contract formalizes entity versions and deduplication rather than replacing the stack.

27.9 Product-specific projection boundary

Garden's evidence-bound widgets should remain product-owned because product fields, comparisons, and conflict rules are domain semantics. The shared package can provide:

- evidence and structured-fact references;
- projection session bound to a release/evidence session;
- validators that every payload field cites admitted provenance;
- generic source-card types;
- event envelope and replay laws.

GEC can project administrative sources and developer traces differently. RAG-TTC can expose tool-loop diagnostics. One universal widget schema would erase useful distinctions.

28. Reliability, deadlines, caching, and backpressure

28.1 Reliability is stage-specific

A RAG request has multiple failure domains: release resolution, local index, query embedding, connected retrieval, reranking, hydration, generation, validation, event persistence, and frontend transport. Treating all as one `500` loses the ability to degrade safely.

Each stage declares:

- whether it is required or optional;
- retry policy and idempotency;
- fallback policy;
- deadline allocation;
- whether partial output is valid;
- whether failure affects release health;
- what trace and metric it emits.

A local index corruption is not equivalent to a reranker timeout. The former should quarantine a release. The latter may degrade one request and trip a provider circuit breaker.

28.2 Deadline algebra

Let total deadline be D. A plan allocates stage budgets d_i and reserve r:

$$\sum_i d_i + r \le D.$$

Parallel stages use maximum elapsed time rather than sum, but shared provider and queue budgets still matter. Static allocation is simple; adaptive allocation can transfer unused budget. The plan records allocation policy.

A stage starting without enough remaining budget should fail fast or choose a cheaper fallback. For example, skip remote reranking when only validation reserve remains. This changes outcomes and belongs to release query policy.

Timeout errors should identify queue, connection, provider, or response phase where possible. End-to-end latency metrics alone cannot diagnose budget misuse.

28.3 Retry semantics

Retries are safe only for idempotent operations or with idempotency keys. Local search and immutable hydration are safe. Provider generation may produce a new sample and incur cost; a retry is a new trace branch. Tool calls to structured systems may have side effects and require product-specific guarantees.

Retry policy includes maximum attempts, backoff, retryable error classes, and remaining-budget check. The final outcome records attempts. A successful retry is not trace-equivalent to first-attempt success and can be an early incident signal.

28.4 Circuit breakers and health

Provider health is tracked per endpoint/model/data class. A circuit breaker can prevent repeated slow failures and select a declared fallback. It must not silently route protected data to a different provider with a different policy.

Release health combines artifact health and dependency health. An index-open failure quarantines the release on that replica and should stop new leases if widespread. A reranker outage can mark a route degraded without invalidating the index release, but the release behavior remains the same because fallback policy is declared.

28.5 Cache layers

RAG systems have several caches with different semantics:

- source payload cache;
- normalized document and derivation cache;
- generated representation cache;
- embedding cache;
- opened release/artifact cache;
- query embedding cache;
- channel result cache;
- reranker result cache;
- final answer or session replay cache;
- frontend snapshot cache.

Every cache key includes all semantic inputs. A query-result cache must include release ID, subject-policy partition, query plan, route, filters, locale, and query text. Caching authorized results under only text and bundle ID is unsafe.

Final answer caching is particularly difficult because conversation context, subject, stochastic policy, and presentation change. Prefer caching deterministic intermediate results unless product semantics make final answers explicitly reusable.

28.6 Cache soundness laws

For cache C_f of operation f:

1. equal keys imply inputs equivalent under f's denotation;
2. a cached value verifies against the key and output schema;
3. cache hit and fresh execution are observationally equivalent under declared properties;
4. protected subject partitions cannot collide;
5. cache invalidation follows semantic identity rather than time alone;
6. provider outputs retained as material remain linked to original policy and model identity.

Property tests can mutate each input field and confirm whether key changes according to the operation spec.

28.7 Backpressure

Under load, the system should reject or queue work before it exhausts providers or memory. Separate pools for query embedding, reranking, generation, connected retrieval, and build work prevent a large refresh from starving interactive queries.

Admission considers deadline, estimated cost, tenant quota, and current queue. A request rejected before disclosure returns a typed overloaded outcome. An accepted request receives a budget reservation.

Build workloads use lower-priority or separately provisioned resources. Semantic caches can be shared, but resource queues should not.

28.8 Load shedding and quality degradation

A declared degradation ladder may be:

1. full hybrid plus reranker;
2. hybrid without reranker;
3. lexical plus vector with lower depth;
4. lexical-only local search;
5. safe abstention.

The ladder must respect product and query class. A vector-only corpus cannot use lexical fallback. A security-sensitive query may prefer abstention over connected search. Every step has its own plan ID and outcome warning.

Quality evaluation must include degraded plans because production traffic will use them under stress. Reliability is the weighted behavior over healthy and degraded states, not ideal-path quality alone.

28.9 Cancellation and resource reclamation

Context cancellation should stop queued work, propagate to providers, and prevent nonessential post-processing. Provider APIs may not cancel immediately; traces distinguish requested from confirmed cancellation. Results arriving after cancellation can populate safe semantic caches but not emit customer events.

Release leases close after terminal persistence, not merely after model completion, if frontend/source projection still needs release artifacts. Long-running streams must renew process-level lease heartbeats.

28.10 Graceful shutdown

A server shutdown stops acquiring new leases, marks the replica draining, waits for bounded in-flight work, persists terminal cancellation for forced exits, closes release handles, and flushes trace/event stores. Startup preloads active release and verifies dependencies before accepting traffic.

This lifecycle should be tested with active agent turns and snapshot streams. Current startup-bound services already close bundles; the shared release manager generalizes the behavior across hot activations.

29. Security, privacy, and trust boundaries

29.1 Trust zones

The system has at least five trust zones:

1. source systems and connector credentials;
2. local corpus and index artifact plane;
3. product server and authorization policy;
4. remote model, embedding, reranker, or connected-search providers;
5. customer and developer frontends.

Data can cross a boundary only through a typed operation with policy. Retrieval correctness is subordinate to authorization and provider disclosure rules.

29.2 Authorization before ranking effects

Authorization should be applied as early as possible. Source admission can exclude globally prohibited data. Index partitioning and metadata filters constrain channel search. Local authorization verifies candidates before text hydration or remote stages. Final presentation rechecks evidence references.

GEC's current post-rerank filter violates the remote non-disclosure property. The immediate fix is to prefilter candidate IDs using local chunk/document metadata before `rerank`. The complete fix is authorized top-k retrieval through filter pushdown or scope-partitioned indexes, followed by an authorization certificate.

A regression test should use a reranker spy that fails the test if it receives unauthorized marker text. Returned-result tests alone will not catch the disclosure.

29.3 Noninterference goal

A strong ideal is that unauthorized documents do not influence observable results, including ranks and timing. Full noninterference is difficult because global index statistics or shared ANN topology can create indirect effects. The production requirement should at least guarantee no content disclosure and no unauthorized result; products can decide whether rank-position influence is acceptable.

For high-assurance administrative corpora, per-scope indexes or cryptographically separate partitions provide a clearer model. The route selects partitions authorized for the subject and fuses only those rankings.

29.4 Provider data policy

Each remote provider configuration declares allowed data classes, regions, retention, training use, and logging policy. A provider call includes a policy decision reference. Rerankers receive source text; embedding providers receive corpus or query text; generators receive evidence and conversation. These are distinct disclosure categories.

Fallback cannot change provider policy implicitly. If primary provider is unavailable and backup is not permitted for the data class, the correct outcome is local fallback or abstention.

29.5 Prompt injection in sources

Source normalization should mark or remove active content such as scripts and hidden instructions. Retrieval should treat source text as evidence, not system instructions. Context formatting clearly delimits source material and tells the model not to follow embedded commands.

Tests should include documents that ask the model to ignore policy, reveal secrets, call tools, or fabricate citations. Grounding and tool policy should prevent source text from granting capabilities.

Source content remains untrusted even when authorized. Structured facts from trusted databases have different authority and should be typed separately.

29.6 Tool argument authority

Model-visible tool schemas expose query terms, requested limits, or route hints only when safe. Access scopes, tenant, release, provider policy, and database permissions come from server context. GEC already follows this principle in its knowledge tool.

The tool runtime validates every argument and caps limits. Unknown routes fall back or fail according to release policy. Tool descriptions are versioned release assets because they influence model behavior but do not confer authority.

29.7 Evidence provenance

Every evidence item should support a chain:

```
release -> source barrier -> source revision -> normalized document
        -> chunk span -> representation contribution -> ranked hit
        -> authorization certificate -> admitted evidence -> citation/widget field
```

Structured facts add database snapshot/query/item lineage. Connected retrieval adds provider request and response artifact identity with weaker reproducibility classification.

This chain enables audits, deletion impact analysis, stale-answer diagnosis, and source-card rendering. Missing links are verification failures, not optional metadata.

29.8 Logging and traces

Trace payload policy uses references and digests by default. Sensitive query/evidence text can be stored in an authorized encrypted artifact store with retention, not duplicated across logs. Customer frontend receives only safe projection. Developer mode still enforces subject authorization.

A trace redaction bug can be more serious than a retrieval bug because logs have broader access and retention. Static trace-schema review and payload scanners belong in release gates.

29.9 Multi-tenant isolation

Release scope, indexes, caches, and query keys include tenant where required. Shared public corpus can be referenced by several tenant releases, but private delta overlays and subject filters remain isolated. Provider rate and cost quotas are tenant-aware.

Cross-tenant cache reuse is permitted only for public immutable inputs and operations whose output is independent of tenant policy. The cache artifact records its sharing class.

29.10 Security under optimization

The candidate system cannot relax security constraints to improve relevance. Policy assets are immutable and reviewed. Proposers operate within a capability sandbox. Security test suites are hidden from unrestricted automated proposers when exposure would make gaming easy.

Online shadow execution uses the same authorization and provider policy as production. Candidate traces are subject to the same retention and redaction rules.

30. Observability, SLOs, and runtime diagnosis

30.1 Observability model

Observability should answer four questions:

1. What source and release state existed?
2. What plan and transitions executed?
3. What outcome and evidence reached the user?
4. Why did this differ from an expected or comparison run?

Metrics alone answer the second question poorly. Traces, release manifests, evidence lineage, build events, and frontend event logs must be linked.

30.2 Build telemetry

Build metrics include:

- source items captured, admitted, excluded, deleted, and late;
- source watermark and capture lag;
- affected versus reused documents/chunks/representations/vectors;
- cache hits and provider calls;
- stage throughput, queue, retries, and quarantines;
- cost and resource usage;
- delta size, tombstone ratio, and compaction trigger;
- verification and evaluation results;
- time from observation to verified and active release;
- activation conflicts and rollback.

Metrics carry build intent, source barrier, and release labels. Avoid unbounded document IDs as metric labels; detailed item data belongs in traces/artifacts.

30.3 Query telemetry

Query metrics include:

- requests by interpreter and route;
- release and cohort;
- channel success/timeout/failure;
- candidate and evidence counts;
- filter selectivity and authorized starvation indicators;
- reranker invocation/fallback;
- context tokens and source diversity;
- generation and validation outcome;
- tool calls and iterations;
- end-to-end and stage latency;
- cancellation and partial output;
- provider usage and cost;
- frontend terminal delivery.

A `complete` counter without `degraded` dimension hides provider incidents. A quality dashboard should stratify by path actually executed.

30.4 Freshness SLOs

Example objectives are:

- 99% of source revisions observed within 15 minutes for class A;
- 99% of admitted revisions active within 60 minutes of observation;
- active release watermark no older than 24 hours for documentation;
- security tombstones block new acquisitions within 5 minutes;
- cross-source release skew below a declared bound.

The exact numbers are product decisions. The architecture exposes the clocks required to measure them.

30.5 Query SLOs

Define SLOs by interpreter and route. Direct search can have a lower latency target than agent turns. A hybrid-rerank route has different budget from lexical fallback. Report p50/p95/p99 and success/degraded/abstain separately.

A terminal outcome is not enough if the frontend never applies it. End-to-end completion spans submission reservation, query runtime, durable event persistence, WebSocket delivery, and client reduction where measurable.

30.6 Quality monitoring

Production has sparse labels. Use a mix of:

- sampled human review;
- structural grounding validators;
- calibrated model judges;
- user corrections and follow-up signals;
- retrieval contribution and empty-evidence alerts;
- canary paired shadow judgments;
- regression cases mined from incidents.

Judge scores are not SLOs unless calibrated and audited. They are indicators. Safety invariants and operational objectives remain directly measurable.

30.7 Trace-based diagnosis

A retrieval miss investigation should reconstruct:

1. source revision and active release watermark;
2. whether the expected document was admitted and indexed;
3. chunk/representation identities and index membership;
4. query rewrite and route;
5. per-channel ranks and filters;
6. fusion and reranker movement;
7. evidence admission and context budget;
8. answer/tool behavior;
9. frontend projection.

The trace schema and artifact references should make this possible without rerunning live providers. A replay can test counterfactual policy changes against retained intermediate artifacts.

30.8 Release health and automatic action

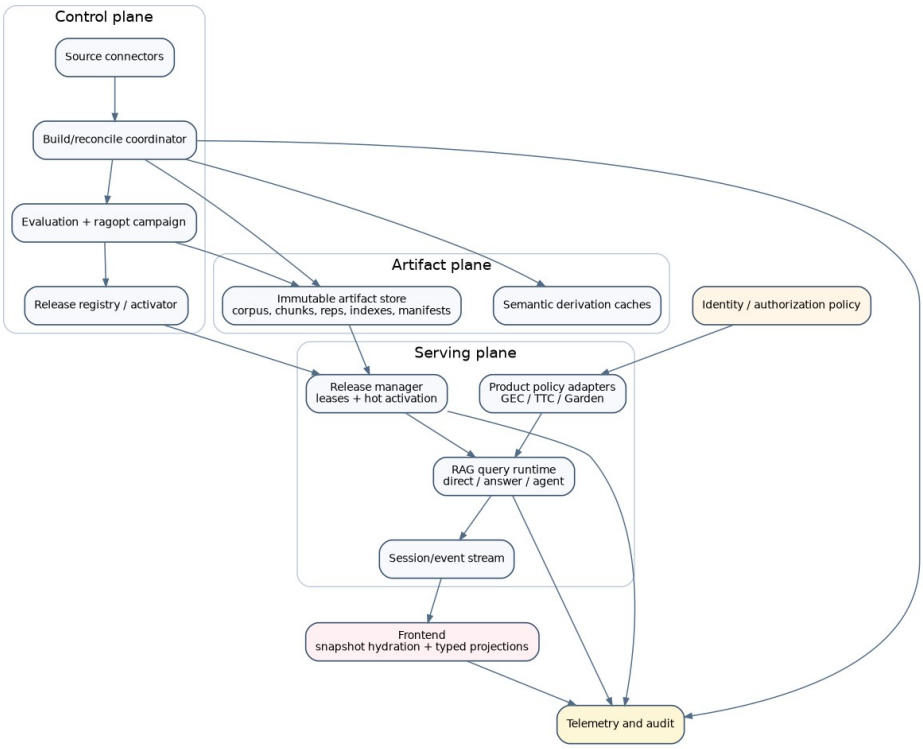
The release manager aggregates hard health signals. Examples:

- index open/verification failures;
- query error rate above threshold;
- security invariant failure;
- deleted-content exposure;
- severe latency or provider-cost regression;
- frontend terminal-event failure.

A canary can auto-rollback on hard signals. An active stable release can be quarantined and rolled back under incident policy. Soft quality drift opens an investigation rather than immediate automation.

30.9 Service-level evidence

SLO evaluation itself should be reproducible. Retain the time window, metric query/version, release cohorts, exclusions, and decision result. Promotion reports link to this evidence. This applies `ragopt`-style custody to operational decisions without moving metric semantics into `ragopt`.



A production topology separates control, artifact, serving, and projection planes.

Part VI. Shared architecture and migration

31. Proposed package architecture

31.1 Design rule

The shared package boundary follows semantic ownership. `ragkit` owns general RAG-domain meaning and runtime laws. Product repositories own source-specific normalization, authorization policy, structured-domain meaning, agent prompts, and presentation schemas. `ragopt` owns experiment custody and campaign mechanics. Transport frameworks and deployment schedulers remain separate.

The design does not require all packages to be extracted immediately. Types can begin under internal paths and move only after two applied systems use matching semantics. The release, source-change, query-interpreter, and frontend-law types are exceptions where one shared definition prevents high-risk divergence.

31.2 `ragkit/corpus`

Responsibilities:

- source keys, revisions, tombstones, cursors, barriers, and watermarks;
- connector capability descriptions;
- normalized document envelopes and admission records;
- snapshot and change-stream validation;
- source-lineage and temporal consistency laws;
- canonical snapshot manifests.

It does not know SQL schemas, product fields, Git admission rules, or web crawler details. Those are adapters.

Proposed surface:

```
package corpus

type Connector interface {
    ID() ConnectorID
    Capabilities() Capabilities
}

type Snapshotter interface {
    Connector
    Snapshot(context.Context, SnapshotRequest, ChangeSink) (Barrier, error)
}

type ChangeSource interface {
    Connector
    Changes(context.Context, Cursor, ChangeSink) error
}

type Normalizer interface {
    Spec() NormalizeSpec
    Normalize(context.Context, DocumentRevision) ([]NormalizedDocument, Admission, error)
}
```

A `ChangeSink` is backpressured and idempotent by connector/cursor/revision identity. It does not require loading the whole corpus into memory.

31.3 `ragkit/derive`

Responsibilities:

- versioned stage specifications;

- semantic derivation keys;
- impact graph and invalidation closure;
- content-addressed cache contracts;
- deterministic and retained-stochastic derivations;
- lineage graph from documents to index entries;
- incremental equivalence helpers.

Existing chunking, representations, embedding, and flow packages remain domain implementations. `derive` connects them without becoming another workflow framework.

```
type NodeSpec interface {
    Kind() NodeKind
    SemanticID() digest.Digest
    Dependencies() []NodeKind
    Incrementality() Incrementality
}

type Planner interface {
    Plan(context.Context, BaseLineage, corpus.Snapshot, Graph) (ImpactPlan, error)
}
```

31.4 `ragkit/index`

Responsibilities:

- backend capabilities and semantic contracts;
- full and delta build interfaces;
- immutable snapshot searchers;
- base/delta/tombstone views;
- compaction;
- exact-oracle and incremental-equivalence certification;
- common filters and result-completeness declarations.

Existing Bleve and SQLite implementations can adapt behind this package. `indexbundle` can become an implementation of one full immutable release artifact rather than the top-level production unit.

31.5 `ragkit/build`

Responsibilities:

- build intent and plan identity;
- durable event vocabulary and reducer;
- checkpoint manifest;
- local coordinator implementation;
- stage scheduling over `flow`;
- verification and release-registration handoff;
- status and progress projections.

This package is justified once both GEC and TTC use the same refresh lifecycle. Until then it can live in `ragkit/experimental/build` or one product with shared interfaces fixed in `corpus`, `derive`, and `release`.

It must remain a fixed RAG coordinator. Generic distributed workflow concerns belong to an external scheduler.

31.6 `ragkit/release`

Responsibilities:

- behavior-complete release manifest and canonical ID;
- lifecycle state and activation events;
- registry, compare-and-swap activation, cohort mapping;
- resolver and reference-counted/epoch leases;
- release verification hooks and retirement;
- rollback and emergency revocation.

This package is the synchronization boundary between maintenance and querying. It should be adopted early even while builds remain full and activation still requires process-local reload.

31.7 `ragkit/query`

Responsibilities:

- retrieval operation signature and typed plans;
- rewrite, channel, collapse, fusion, filtering, rerank, hydrate, and admit specifications;
- direct-search interpreter;
- common outcomes, warnings, and trace events;
- authorization-certificate and provider-disclosure contracts;
- deterministic reference interpreter and production concurrent interpreter.

Existing `rag/retrieval` kernels move under or are used by this package. Existing `rag/answering` becomes one interpreter rather than the only complete query abstraction.

31.8 `ragkit/answer` and `ragkit/agent`

`ragkit/answer` owns context construction, generation request formation, grounded contract validation, abstention, and answer outcomes. Product prompts and structured response schemas are inputs.

`ragkit/agent` should remain small. It defines a bounded RAG tool-transition host, idempotent call IDs, evidence-session integration, and terminal validation hooks. It does not replace Geppetto/Pinocchio or product chat frameworks. It adapts shared search semantics into them.

31.9 `ragkit/evidence`

Responsibilities:

- typed source, structured, generated, and connected evidence references;
- evidence-session scopes and release coherence;
- stable citation labels;
- admission certificates;
- generic source projections;
- laws for boundedness and stable reuse.

Product-specific ledgers can wrap it. The package must not assume every evidence kind can be cited as source authority.

31.10 `ragkit/stream`

Responsibilities:

- event envelope, entity version, ordinal, operation, and patch modes;
- snapshot type;
- pure reducer interfaces and conformance tests;

- deduplication and stale-event laws;
- generic RAG lifecycle events.

Product timeline schemas remain outside. GEC and Garden map their entities to this envelope.

31.11 `ragkit/eval`

Responsibilities:

- retrieval and evidence metric schemas;
- answer/grounding result envelopes;
- temporal and operational metric coordinates;
- per-case artifacts and stratification;
- paired-difference helpers;
- exact-oracle and release comparison contracts.

It does not implement a universal LLM judge or product utility. Those are arms that emit native artifacts and shared metric projections.

31.12 `ragopt/ragspace`

A thin adapter package can define RAG parameter references, intervention classes, dependency closures, and fidelity requirements. It imports `ragkit` types. Core `ragopt` remains domain-neutral.

```
type Arm struct {
    Builder   ragbuild.Client
    Resolver  release.Resolver
    Executor  rageval.Executor
    Projector MetricProjector
}
```

The arm builds or resolves candidate releases and evaluates them at product-native levels. `ragopt` schedules paired cells and gates their projections.

31.13 Dependency rules

- `ragkit/corpus`, `derive`, `index`, `release`, `query`, `evidence`, `stream`, and `eval` do not import product code.
- `ragkit/query` depends on `release`, `index`, and `evidence`, not on HTTP or chat frameworks.
- `ragkit/build` depends on corpus/derive/index/release and uses flow as inner execution.
- `ragopt` core does not import `ragkit`.
- `ragopt/ragspace` may import both and is optional.
- GEC, RAG-TTC, and Garden depend on published `ragkit`, never copied source.
- Product presentation depends on shared evidence/stream types but retains domain schemas.

32. API blueprint and executable laws

32.1 Release specification

```
type Spec struct {
    SchemaVersion string
    Product       string
    Corpus        corpus.SnapshotRef
    Build         build.SpecRef
    Indexes       []index.ViewSpec
    Query         query.Spec
    Answer        answer.Spec
    Agent         *agent.Spec
    Evidence      evidence.Policy
    Structured    []StructuredAsset
    Presentation  []artifact.Ref
    Validators    []artifact.Ref
    DataPolicy    policy.Ref
}

type Manifest struct {
    ID        ID
    Spec      Spec
    Artifacts []artifact.VerifiedRef
    BuiltAt   time.Time
    VerifiedAt time.Time
}
```

Canonical validation rejects paths without material digests where behavior depends on content, mutable provider aliases without retained identity classification, duplicate artifact roles, and missing dependency links.

32.2 Query plan

```
type Spec struct {
    PlanID          PlanID
    Rewrite         RewriteSpec
    Channels        []ChannelSpec
    Collapse        CollapseSpec
    Authorization   AuthorizationSpec
    Fusion          FusionSpec
    Rerank          *RerankSpec
    Admission       AdmissionSpec
    Deadline        DeadlineSpec
    Fallbacks       []FallbackRule
    TracePolicy     trace.Policy
}
```

A plan validator checks:

- unique channel names and deterministic order;
- finite weights and positive depths;
- authorization domination of remote text operations;
- capability match with release indexes;
- bounded remote candidate and agent loops;
- fallback plan existence and policy compatibility;
- evidence admission after release-bound hydration;
- complete semantic identity.

32.3 Search interface

```
type Interpreter interface {
    Execute(context.Context, *release.Lease, SubjectContext, Request) Outcome
}
```

```
type Outcome struct {
    Status    Status
    Release   release.ID
    Plan      PlanID
    Rankings  []ChannelRanking
    Fused     []Candidate
    Evidence  []evidence.Admitted
    Warnings  []Warning
    Trace     trace.Ref
}
```

The deterministic reference interpreter executes channels in canonical sequence with no deadlines and retained provider outputs. The production interpreter may execute concurrently but must refine protected semantics.

32.4 Build events

```
type Event interface{ isBuildEvent() }

type Started struct{ Intent IntentID; Fence uint64 }
type BarrierCaptured struct{ Barrier corpus.BarrierVector }
type PlanCommitted struct{ Plan artifact.Ref }
type ItemCommitted struct{ Stage StageID; Key derive.Key; Artifact artifact.Ref }
type ItemQuarantined struct{ Stage StageID; Key derive.Key; Reason ErrorCode }
type StageSealed struct{ Stage StageID; Summary StageSummary }
type VerificationCompleted struct{ Report artifact.Ref; Passed bool }
type ReleaseRegistered struct{ Release release.ID }
type BuildFailed struct{ Stage StageID; Error ErrorCode }
type BuildCancelled struct{ Reason string }
```

A pure reducer rejects impossible transitions. The event store appends with expected position and fence token. The reducer is a small model-checking target.

32.5 Activation API

```
type Activator interface {
    CompareAndSwap(context.Context, ActivationRequest) (Activation, error)
}

type ActivationRequest struct {
    Scope          string
    Expected       ID
    Desired        ID
    IdempotencyKey string
    Actor          string
    Reason         string
    GateReport     artifact.Ref
    Cohort         *CohortPolicy
}
```

The activator verifies desired release state, gate report policy, and expected head. It emits one immutable activation event. It does not rewrite a config file and hope all replicas reload consistently.

32.6 Stream reducer

```
type Event struct {
    ID            string
    StreamID      string
    StreamOrdinal uint64
    EntityID      string
    EntityVersion uint64
    Operation     Operation
    PatchMode     PatchMode
    Payload       json.RawMessage
    ReleaseID     release.ID
    CausationID   string
}
```

```
type Reducer[S any] interface {
    Apply(S, Event) (S, error)
}
```

The conformance suite generates snapshots, suffixes, duplicates, stale events, and legal reorderings. Product reducers must pass before adopting the envelope.

32.7 Law suite

The shared law suite contains:

Release laws. Canonical manifest round trip; ID sensitivity to material behavior fields; insensitivity to explicitly operational deployment fields; artifact role uniqueness.

Lease laws. No new lease after draining; old lease remains valid; close idempotence; retirement only after zero leases; activation does not mutate existing lease.

Query laws. deterministic fusion and tie order; authorization before remote disclosure; all evidence from lease release; fallback warning preservation; context budget and stable prefix where applicable.

Build laws. event reducer transition validity; duplicate item commit idempotence; resume equivalence; incremental/full equivalence; no publication before verification; activation absent from build success.

Evidence laws. stable labels, boundedness, no mixed release, evidence-kind preservation, source projection lineage.

Stream laws. snapshot-suffix equivalence, duplicate idempotence, stale rejection, deterministic display, terminal immutability.

Optimization laws. dependency closure complete; candidate exactly one declared intervention; paired coordinates exact; missing outcomes retained; gates ordered and fail closed.

33. GEC migration

33.1 Current strengths to retain

GEC already has product-correct boundaries: server-owned scopes and roles, separate knowledge and structured tools, strict synonym loading, deterministic weighted RRF, explicit forced routes for evaluation, run-scoped evidence labels, persistent chat/timeline infrastructure, and customer/developer distinctions. These should not be generalized away.

Its immediate production issues are release identity, authorization order, startup-only activation, and fragmented optimization custody.

33.2 Phase G0: behavioral fixture

Before changing the query path, capture fixture cases containing:

- lexical, hybrid, no-rerank, and synonym-expanded rankings;
- reranker success and failure fallback;
- access scope and source role filtering;
- evidence labels and tool serialization;
- empty/invalid queries;
- representative admin conversations and frontend snapshots.

Use retained local search/reranker outputs where remote providers are unavailable. These fixtures define current intentional behavior and expose intentional changes such as secure prefiltering.

33.3 Phase G1: behavior-complete release

Wrap the existing startup resources in a `release.Manifest` without changing execution. Include:

- bundle and corpus identity;
- synonym file content digest and expansion spec;
- reranker provider/model, pool, document-text composer, blend, timeout, and fallback;
- query route/defaults and search-depth behavior;
- evidence policy and tool schema/description;
- answer prompt/contract and inference profile where used;
- structured SQL/tool policy references;
- frontend source projection schema.

Every query trace and tool output adds release ID. This immediately makes current behavior auditable.

33.4 Phase G2: authorization before reranking

Change retrieval to return metadata-bearing candidate IDs, prefilter them locally by access scope and source role, and hydrate only authorized candidates for reranking. Add the remote spy regression test.

Then replace heuristic postfilter completeness with index filter pushdown or scope-partitioned search. The query outcome reports whether authorized top-k completeness is guaranteed. Remove comments that rely on current two-scope corpus size as a general safety argument.

33.5 Phase G3: release manager

Replace the single `*knowledge.Service` with a resolver-backed runtime. A release loader opens bundle, documents, synonyms, reranker adapter, prompts, and validators. The chat runtime acquires one lease per run/turn. The evidence ledger records the lease ID.

Begin with manual activation and one process. Add compare-and-swap registry, preload, and draining. Later add replica-wide registry watch. A configuration reload that fails leaves the old release active.

33.6 Phase G4: corpus connector and refresh

Because `internal/knowledgebuild` is absent from the supplied source, the migration must begin with a source-contract audit when that package is available. Based on design records, product/category/schema-doc connectors become `corpus` adapters. The existing committed manifest and normalization become versioned specs. The first shared coordinator can still perform a full nightly build with content-addressed representation/embedding reuse.

Add source snapshot identity and watermarks to the release. Only after full-refresh operation is stable should GEC add delta overlays. Product and category rows are suitable for logical-key upsert/delete; embedded schema docs may use package/library version barriers.

33.7 Phase G5: optimization integration

Map current RRF sweep into a `ragopt` campaign whose candidate intervention is fusion spec. Retain frozen channel rankings as native artifacts and add paired confidence/holdout. Map reranker and synonym experiments to separate intervention classes.

Answer-quality judges and evaluation sets remain GEC-owned arms. They emit common answer/grounding metric projections and retain failures. Promotion creates a new release spec and activation report rather than environment-variable instructions.

33.8 Phase G6: frontend event hardening

Add event IDs and entity versions to SessionStream/UI events. Update `wsManager` to discard events at or below snapshot ordinal and deduplicate IDs. Update `timelineSlice` to reject stale versions before merging. Append patches carry offsets or are converted to full replace values at reconnection boundaries.

Property tests generate snapshot/event schedules. Existing UI mapping and display sorting remain.

34. RAG-TTC migration

34.1 Current strengths to retain

RAG-TTC contains the richest experimental surface: multiple representations, caches and budgets, committed-Git snapshots, workspace artifacts, exact and ANN search, connected retrieval, search routes, agent tools, evidence ledgers, answer/tool evaluation, persistent chat server, and diagnostics. It should be the first proving ground for joint index-query optimization.

It also contains copied `ragkit` source. The copied substrate must be removed so runtime fixes and semantics have one owner.

34.2 Phase T0: hard cutover to `ragkit`

Use package-level differential fixtures to compare current copied code and `ragkit` for chunking, representations, bundle opening, retrieval, fusion, context, and evaluation. Resolve intentional differences. Change imports to `github.com/go-go-golems/ragkit` and delete copied packages rather than maintaining adapters indefinitely.

Product-specific packages such as `ttcrag`, product catalog, connected retrieval, knowledge database, tool answer, HNSW candidate, application chat, and experiment commands remain in RAG-TTC.

34.3 Phase T1: source connector

Adapt `gochunk.LoadCommitted` to `corpus.Snapshotter`. Git commit/tree becomes snapshot token. Admission records map directly. Snapshot digest and per-file document revisions enter release lineage.

A future Git change source can diff commits and emit upsert/delete changes. The clean committed-tree snapshot remains the oracle. Working-directory indexing, if needed for developer tools, is a separate weaker connector with explicit non-repeatable semantics.

34.4 Phase T2: build coordinator

Adapt the current indexes build and workspace index commands to one build intent and stage DAG. Existing representation and embedding caches become derivation caches. CLI progress subscribes to build events. Dry run prints impact plan and budgets.

The first implementation still produces full `indexbundle` artifacts. Later it adds delta views. Experiment builds and production refresh use the same build semantics but different source/parameter locks and registries.

34.5 Phase T3: query interpreters

Extract shared retrieval plan construction from workspace search, ask, and `ttcrag.SearchTool`. Direct CLI search uses direct interpreter. Ask uses answer interpreter. The model tool uses agent interpreter with the same channel/fusion kernels.

Route observation, connected augmentation, product filters, and structured routes remain RAG-TTC-owned extensions. Every tool call runs under the turn lease and evidence session.

34.6 Phase T4: ANN backend certification

Move HNSW candidate behind `ragkit/index` capability interface only after it passes:

- static exact-oracle recall/latency gate;
- incremental insertion/deletion sequence tests;
- base-plus-delta query view tests;
- filter behavior;
- build/reopen/reproducibility tests;
- memory, build time, and concurrency benchmarks;
- compaction and source-refresh scenarios.

The existing bakeoff becomes one fidelity in a broader backend campaign.

34.7 Phase T5: production serving

The simple serve command and canonical chat server resolve releases through shared manager rather than opening one bundle forever. Per-conversation runtime construction receives a turn or conversation lease. Submission idempotency, timeline persistence, and auth remain application-owned.

Expose release and evidence IDs in outcomes and SessionStream events. Add shadow arm support that executes a candidate release without emitting customer events.

34.8 Phase T6: optimization campaigns

Define typed RAG spaces for representation kind, chunking, embedding, ANN, route, channel depth, fusion, reranking, context, and agent/tool policy. Use dependency-aware shared artifacts. Current manual arm contracts become candidate specs. Tool-answer and session evaluations remain native arms.

RAG-TTC can then serve as the integration test for `ragopt/ragspace` without making `ragopt` own RAG behavior.

35. Garden migration and presentation semantics

35.1 Current strengths to retain

Garden's distinctive value is product interpretation: intent-specific routes, structured product facts, connected fallback, exact fact augmentation, evidence-admitted widgets, field alignment, conflict suppression, customer/developer projections, and real multi-turn calibration. These are not generic retrieval utilities.

The migration should replace infrastructure beneath them while preserving the product semantic layer.

35.2 Phase A: replace copied substrate transitively

Garden currently depends on RAG-TTC, which contains copied RAG core. After RAG-TTC cuts over, Garden receives shared `ragkit` behavior without direct large changes. Add fixture tests around current route outputs, citations, structured-first responses, and grounded widgets.

35.3 Phase B: Garden release manifest

Create one release spec containing:

- TTC index view and source corpus;
- embedding/query provider identities;
- fact database content digest and schema/query adapters;
- tool configuration content;
- intent route definitions and connected-retrieval policy;
- evidence-session limits;
- prompt/profile/tool schema assets;
- grounded widget projection schemas and conflict policy.

Paths are deployment locators only. The manifest uses content identities. The per-conversation session search is created from a release lease.

35.4 Phase C: evidence epochs

Garden currently creates fresh search state per conversation. Choose one of two explicit policies:

- pin the entire conversation to one release with a maximum conversation lease duration; or
- pin each turn, create evidence epoch per turn, and allow widgets only from the current epoch unless historical evidence is explicitly imported with original release lineage.

Per-turn pinning is preferable for freshness. Follow-up interactions can reference prior visible content, but new widget generation should not silently treat old evidence as current.

35.5 Phase D: typed projection integration

Keep `grounded_widgets.go` and `evidenceview` product-owned. Replace internal citation identity plumbing with `ragkit/evidence` references. Each widget field carries source chunk or structured fact lineage and release ID. The generic stream envelope transports the typed widget payload.

Conflicting fact suppression remains a Garden validator. It can emit structured diagnostics for calibration and optimization.

35.6 Phase E: calibration as a high-fidelity arm

Garden calibration becomes a `ragopt` arm at conversation level. Each case has stable idempotency keys and source/release constraints. The runner records every snapshot poll and terminal-settle decision as native artifacts. Metrics include:

- terminal completion and latency;
- answer and choice assertions;
- source/evidence kinds;
- widget grounding and field lineage;
- tool calls and route decisions;
- word/token budget;
- release consistency across turn;
- customer and developer projection correctness.

Run incumbent and candidate on paired conversation cases and repeat stochastic profiles. Preserve current real-server execution rather than replacing it with a unit-level simulator.

35.7 Phase F: frontend law conformance

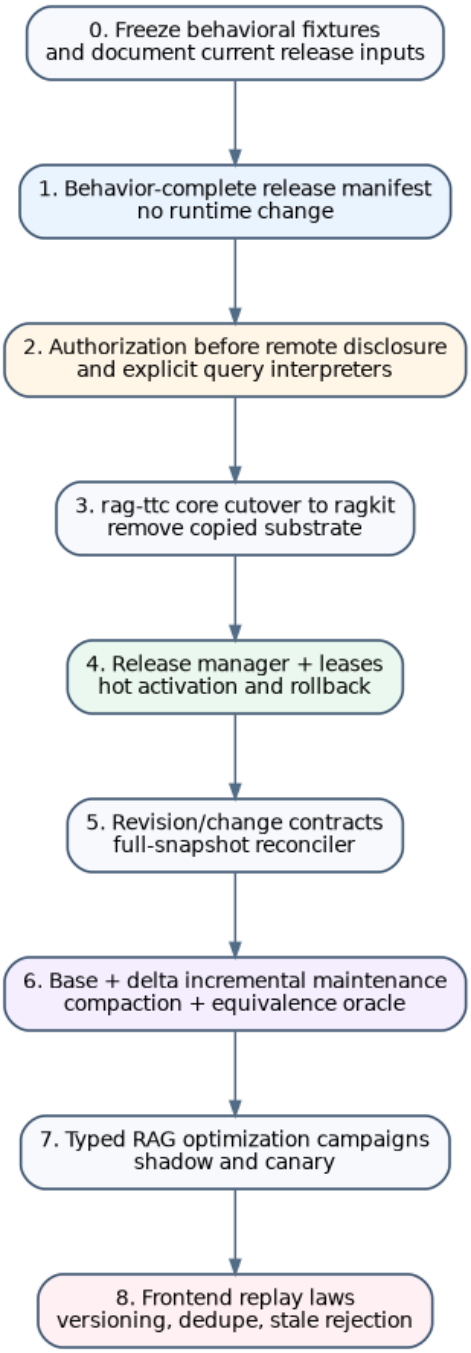
Garden's shared chat provider and widget frontend should adopt event IDs, entity versions, release lineage, and snapshot-suffix conformance tests. Zod schemas continue validating product payloads. Customer mode must never render developer-only lineage even when it is present in the authoritative event artifact.

36. Migration program, verification, and conclusion

36.1 Dependency-ordered program

The migration is ordered to reduce semantic risk:

1. freeze behavioral fixtures and current runtime identities;
2. introduce behavior-complete release manifests without changing query output;
3. fix authorization before remote disclosure;
4. cut RAG-TTC copied core over to `ragkit`;
5. separate direct, answer, and agent interpreters;
6. add release manager, leases, compare-and-swap activation, and rollback;
7. introduce source revisions, barriers, and a full-snapshot reconciler;
8. add durable build events/checkpoints over existing full builds;
9. add base-plus-delta views, tombstones, and compaction with full-build oracle;
10. add typed RAG optimization spaces and multi-fidelity campaigns;
11. add shadow/canary resolution and operational gates;
12. harden frontend snapshot/event semantics.



Dependency-ordered migration from current fixed bundles to dynamic production RAG.

36.2 Why release identity comes first

Without complete release identity, every later comparison is ambiguous. Hot activation cannot say what it activated. A query trace cannot prove which synonyms, fact DB, prompt, or reranker it used. An optimization candidate cannot be reproduced. Therefore the first implementation change is a manifest and trace field, not incremental indexing.

This can be introduced around current startup-bound services and creates immediate value with low behavioral risk.

36.3 Why security precedes relevance work

GEC's remote rerank ordering is a concrete trust-boundary defect. Fixing it may change rankings because authorized prefiltering changes the candidate pool. That change should be accepted as a security correction and then rebaselined, not delayed to preserve benchmark continuity.

The new authorization certificate and plan law prevent recurrence across products.

36.4 Why full refresh precedes deltas

A durable full-refresh machine establishes source barriers, build intent, checkpoints, verification, release registration, activation, and observability. Incremental maintenance reuses all of these and changes the impact plan/index view. Implementing deltas first would mix source, build, and activation bugs.

Full rebuild remains the oracle, so this work is never discarded.

36.5 Acceptance criteria by milestone

Release milestone. Every turn records one complete release ID; changing any material query asset changes release ID; fixtures remain stable.

Security milestone. No unauthorized marker text reaches remote reranker/generator in adversarial tests; authorized top-k semantics are declared and measured.

Activation milestone. New queries switch atomically; in-flight old queries complete; rollback is one CAS; no mixed-release evidence.

Refresh milestone. Source barrier and watermark are visible; duplicate connector events are idempotent; failed build resumes; publication never activates implicitly.

Incremental milestone. Random change sequences produce views equivalent to clean full builds; deletes disappear; compaction preserves query behavior within backend tolerance.

Optimization milestone. Candidates declare intervention/dependency/fidelity; paired runs retain failures; holdout and uncertainty gates are enforced; promotion references immutable release.

Frontend milestone. Duplicate and stale events cannot corrupt state; snapshot plus suffix equals full replay; terminal answer/source/widget provenance retains release ID.

36.6 Model checking targets

Small state machines merit exhaustive exploration:

- release activation with two concurrent activators and leases;
- build event reducer with retry, cancellation, lease loss, and resume;
- query cancellation across channel/generation/event stages;
- evidence session under repeated calls and release changes;
- frontend snapshot, duplicate, stale, and reordered events.

TLA+ or a small explicit-state model can find interleavings that ordinary tests miss. The production Go reducers remain the executable authority; model traces become test fixtures.

36.7 Formal-proof targets

Machine-checked proof is most valuable for stable pure kernels:

- total ordering and finite-score ranking;

- weighted RRF determinism;
- authorization domination in typed plans;
- incremental differential laws for finite maps and document-local stages;
- event-reducer invariants;
- snapshot-suffix reducer equivalence;
- release-ID canonical encoding.

Provider behavior and complete product correctness remain empirical. The architecture concentrates proof effort where it can establish lasting guarantees.

36.8 Operational adoption strategy

Run old and new paths in differential mode. For release manager, resolve the same current bundle through a lease and compare outputs. For query interpreters, execute both against retained searchers. For frontend changes, replay captured snapshots/events through both reducers. For incremental builds, compare every candidate delta view to full build before activation.

Compatibility adapters have deletion dates. The objective is one semantic path, not permanent double execution.

36.9 Final conclusion

The field of RAG begins before chunking and ends after the frontend has reconstructed a grounded user outcome. It includes source revision semantics, temporal capture, derived views, physical indexes, authorization, retrieval algebra, stochastic providers, agent trajectories, release activation, event streams, and optimization under uncertainty.

The supplied systems already contain most of the necessary local mechanisms. `ragkit` has deterministic retrieval and verified immutable artifacts. RAG-TTC has source snapshots, experimental builds, ANN comparison, agentic search, and persistent chat. GEC has administrative policy and practical reranking/evaluation. Garden has evidence-to-product presentation and multi-turn calibration. `ragopt` has disciplined experiment custody. The architectural task is to connect these mechanisms under a domain model that explains time and state rather than hiding them.

The proposed semantics makes three distinctions decisive. Denotational outcomes explain what a release means to a user. Intensional traces retain facts that outcomes erase. Operational transition systems explain how partial, concurrent, and failureful execution produces those traces. Together they give optimization a correct object: not a score over a static pipeline, but a constrained comparison of evolving RAG releases and their runtime behavior.

The resulting architecture remains pragmatic. It does not require theorem proving before shipping, an all-purpose workflow engine, or a universal product schema. It requires explicit source barriers, immutable behavior-complete releases, lease-pinned query interpreters, typed evidence and events, durable build transitions, and dependency-aware experiments. These are implementable from the current code and strong enough to support incremental indexing, hot activation, safe frontend serving, and joint retrieval optimization.

Appendix A. Structural operational semantics

This appendix collects a compact rule set suitable for an executable reference model. The notation is schematic; production types carry additional IDs and policy.

A.1 Build-machine domains

A build configuration is:

$$B = \langle i, f, c, \tau, p, s, W, A, Q, O \rangle$$

with intent i, fence f, source cursor c, barrier τ, impact plan p, stage state s, pending/in-flight work W, committed artifacts A, quarantine set Q, and event log O.

Start

$$\frac{LeaseBuild(i) = f}{\langle i, \bot, \bot, \bot, \bot, idle, \varnothing, \varnothing, \varnothing, O \rangle \xrightarrow{Started(i,f)} \langle i, f, c_0, \bot, \bot, capture, \varnothing, \varnothing, \varnothing, O' \rangle}$$

Source upsert

$$\frac{Next(c) = Upsert(r, c') \quad RevisionConsistent(r)}{\langle i, f, c, \bot, \bot, capture, W, A, Q, O \rangle \xrightarrow{SourceUpsert(r)} \langle i, f, c', \bot, \bot, capture, W \uplus \{r\}, A, Q, O' \rangle}$$

A duplicate revision with equal digest is a no-op event or retained duplicate observation. A duplicate revision with unequal digest transitions to source-integrity failure.

Source delete

$$\frac{Next(c) = Delete(t, c')}{\langle i, f, c, \bot, \bot, capture, W, A, Q, O \rangle \xrightarrow{SourceDelete(t)} \langle i, f, c', \bot, \bot, capture, W \uplus \{t\}, A, Q, O' \rangle}$$

Barrier

$$\frac{Next(c) = Barrier(\tau, c') \quad SnapshotValid(W, \tau)}{\langle i, f, c, \bot, \bot, capture, W, A, Q, O \rangle \xrightarrow{BarrierCaptured(\tau)} \langle i, f, c', \tau, \bot, plan, W, A, Q, O' \rangle}$$

Plan

$$\frac{p = ImpactPlan(i, \tau, W, A_{base}) \quad PlanValid(p)}{\langle i, f, c, \tau, \bot, plan, W, A, Q, O \rangle \xrightarrow{PlanCommitted(p)} \langle i, f, c, \tau, p, derive, Ready(p), A, Q, O' \rangle}$$

Cache hit

$$\frac{x \in Wk = Key(x) \quad C[k] = a \quad Verify(k, a)}{\langle \ldots, W, A, Q, O \rangle \xrightarrow{CacheHit(k,a)} \langle \ldots, W x\}, A[k \mapsto a], Q, O' \rangle}$$

Execute and commit

$$\frac{x \in Wk = Key(x) \quad Execute(x) = a \quad Verify(k, a) \quad FenceCurrent(f)}{\langle \ldots, W, A, Q, O \rangle \xrightarrow{ItemCommitted(k,a)} \langle \ldots, W x\}, A[k \mapsto a], Q, O' \rangle}$$

Duplicate commit

$$\frac{A[k]=a_0 \quad Equivalent(a_0,a_1)}{\langle \ldots, A, O\rangle \xrightarrow{DuplicateCommit(k)} \langle \ldots, A, O'\rangle}$$

If equivalence fails, the build transitions to nondeterminism quarantine.

Retry

$$\frac{Execute(x)=e \quad Retryable(e,n,d)}{\langle \ldots, x^n \in W, O\rangle \xrightarrow{RetryScheduled(x,n+1,e)} \langle \ldots, x^{n+1} \in W, O'\rangle}$$

Quarantine

$$\frac{Execute(x)=e \quad Quarantinable(e,p)}{\langle \ldots, W, A, Q, O\rangle \xrightarrow{ItemQuarantined(x,e)} \langle \ldots, W\,x\}, A, Q \cup \{x \mapsto e\}, O'\rangle}$$

Stage seal

$$\frac{W_s=\varnothing \quad DependenciesSealed(s) \quad StageInvariant(s,A,Q)}{\langle \ldots, s, W, A, Q, O\rangle \xrightarrow{StageSealed(s)} \langle \ldots, NextStage(s), ReadyNext(p,A), A, Q, O'\rangle}$$

Verify

$$\frac{AllBuildStagesSealed \quad v=VerifyReleaseMaterial(i,\tau,A,Q)}{B \xrightarrow{VerificationCompleted(v)} B'}$$

If v fails, state is `failed` or `quarantined`; no publication rule applies.

Register release

$$\frac{v=pass \quad R=MakeRelease(i,\tau,A) \quad Register(R)}{B \xrightarrow{ReleaseRegistered(R)} B'}$$

There is intentionally no activation conclusion in this rule.

Cancel

$$\frac{Cancelable(s)}{\langle \ldots, s, W, A, Q, O\rangle \xrightarrow{BuildCancelled(reason)} \langle \ldots, cancelled, \varnothing, A, Q, O'\rangle}$$

Committed content-addressed artifacts remain reusable.

A.2 Activation-machine rules

Registry state maps scope to an active release and release IDs to lifecycle/lease counts.

Stage

$$\frac{Verified(R) \quad Preload(R)=ok}{\Sigma \xrightarrow{Stage(R)} \Sigma[R.state := Staged)}$$

Compare-and-swap activation

$$\frac{\Sigma.active(scope)=R_0 \quad R_1.state=StagedGateEligible(R_1)}{\Sigma \xrightarrow{Activate(scope,R_0,R_1)} \Sigma[active(scope):=R_1, R_1.state:=Active, R_0.state:=Draining]}$$

If the expected release is not current, the operation returns conflict without state change.

Acquire lease

$$\frac{R=\Sigma.active(scope) \quad R.state=Active}{\Sigma \xrightarrow{Acquire(R,\lambda)} \Sigma[R.leases:=R.leases+1]}$$

Release lease

$$\frac{\lambda.release=R \quad \neg\lambda.closed}{\Sigma \xrightarrow{Close(\lambda)} \Sigma[R.leases:=R.leases-1, \lambda.closed:=true]}$$

Retire

$$\frac{R.state=Draining \quad R.leases=0 \quad RetentionSatisfied(R)}{\Sigma \xrightarrow{Retire(R)} \Sigma[R.state:=Retired]}$$

A.3 Query-machine rules

A query state is:

$$Q=\langle x,u,\lambda,p,s,H,E,G,T,d\rangle.$$

Acquire

$$\frac{AcquireRelease(x,u)=\lambda}{\langle x,u,\perp,p,received,\ldots\rangle \xrightarrow{ReleaseAcquired(\lambda.R)} \langle x,u,\lambda,p,rewrite,\ldots\rangle}$$

Rewrite

$$\frac{q'=Rewrite_p(x.query)}{Q[stage=rewrite] \xrightarrow{QueryRewritten(q')} Q'[stage=channels]}$$

Provider rewrite failure applies a declared fallback rule or transitions to failed; it never silently returns an empty query.

Search channel

$$\frac{AuthorizedFilter(u,\lambda.R)=f \quad h=Search(\lambda,index_i,q',f,k_i)}{Q \xrightarrow{ChannelCompleted(i,h)} Q[H_i:=h]}$$

Channel timeout

$$\frac{elapsed_i \geq d_i}{Q \xrightarrow{ChannelTimedOut(i)} FallbackOrFail(Q,i)}$$

Fuse

$$\frac{h = Fuse_p(Collapse_p(H))}{Q[stage = rank) \xrightarrow{Fused(h)} Q'[H := h)}$$

Authorize candidate pool

$$\frac{\gamma = AuthorizeCandidates(u, H, providerPolicy) Valid(\gamma)}{Q \xrightarrow{CandidatesAuthorized(\gamma)} Q'[certificate := \gamma)}$$

Remote rerank

$$\frac{Valid(\gamma, \lambda.R, p_r) e = HydrateAuthorized(H, \gamma, \lambda) h' = Rerank_{p_r}(q, e)}{Q \xrightarrow{Reranked(h')} Q'[H := h')}$$

No remote rerank rule exists without γ.

Admit evidence

$$\frac{(E', \eta) = Admit(H, u, \lambda, p.budget)}{Q \xrightarrow{EvidenceAdmitted(\eta)} Q'[E := E', stage := generate)}$$

Generate

$$\frac{g = Generate_p(x, E)}{Q \xrightarrow{Generated(g)} Q'[G := g, stage := validate)}$$

Validate answer

$$\frac{Validate_p(G, E) = a}{Q \xrightarrow{AnswerValidated(a)} Q'[stage := emit)}$$

Invalid output applies repair, abstention, or fail policy.

Emit and complete

$$\frac{ev = TerminalEvent(a, E, \lambda.R) Persist(ev)}{Q \xrightarrow{TerminalEmitted(ev)} Q'[stage := complete)}$$

The lease closes after required terminal persistence/projection work.

A.4 Agent rules

Model chooses tool call

$$\frac{n > 0 \Pi(a) = ToolCall(id, t, args)}{a \xrightarrow{ToolRequested(id, t)} a'[pending := call, n := n - 1)}$$

Idempotent tool replay

$$\frac{L_{calls}[id]=(args,result)}{a \xrightarrow{ToolReplayed(id)} a'[messages:=messages\cdot result]}$$

A repeated ID with different args is a conflict terminal error.

Search tool

$$\frac{t=searcho=SearchInterpreter(\lambda,u,args,E)}{a \xrightarrow{SearchToolCompleted(id,o)} a'[E:=o.E,messages:=messages\cdot o]}$$

Final candidate

$$\frac{\Pi(a)=Final(g)Validate(g,E)=v}{a \xrightarrow{AgentFinal(v)} terminal(v)}$$

Iteration exhaustion

$$\frac{n=0\ \neg terminal(a)}{a \xrightarrow{BudgetExhausted} abstainOrFail(a)}$$

A.5 Frontend rules

Client state is $F=\langle S,n,V,D\rangle$, where S is entity state, n last stream ordinal, V entity versions, and D seen event IDs.

Apply fresh event

$$\frac{e.id\notin De.ordinal>ne.version>V[e.entity)}{F \xrightarrow{Apply(e)} \langle \rho(S,e),e.ordinal,V[e.entity:=e.version),D\cup\{e.id\}\rangle}$$

Duplicate

$$\frac{e.id\in D}{F \xrightarrow{IgnoreDuplicate(e)} F}$$

Stale entity update

$$\frac{e.version\leq V[e.entity)}{F \xrightarrow{IgnoreStale(e)} F}$$

Snapshot hydrate

$$\frac{SnapshotValid(S_n,n)}{F \xrightarrow{Hydrate(S_n,n)} \langle S_n,n,Versions(S_n),SeenThrough(n)\rangle}$$

Buffered events with ordinal greater than n are then applied through the ordinary rules.

Appendix B. Semantic laws and proof obligations

B.1 Source and corpus laws

Revision consistency. A connector cannot emit the same `(connector,key,revision)` with two content digests.

Snapshot determinism. Given the same barrier and retained payloads, normalization produces the same canonical document map.

Delete dominance. A tombstone at revision r_d removes every prior normalized/derived item for the logical key until a later valid upsert.

Barrier closure. A release contains only revisions admitted at its barrier vector; later changes cannot mutate it.

Policy totality. Every source revision has exactly one admission result. Missing policy metadata is rejection, not implicit public access.

B.2 Derivation laws

Lineage totality. Every chunk points to one normalized document revision; every representation points to one chunk; every vector points to one representation.

Representation non-authority. Generated retrieval representations cannot be projected as source evidence without an explicit evidence-kind conversion policy and validator.

Semantic cache soundness. Equal derivation keys imply operation inputs are equivalent for the operation's denotation.

Incremental correctness. Maintained output after a change sequence equals clean output from the integrated source state, exactly or under a declared backend tolerance.

Resume equivalence. Any valid checkpoint prefix resumed under the same intent and retained provider outputs reaches the same terminal release as uninterrupted execution.

B.3 Index laws

Finite total order. Every returned ranking has positive consecutive ranks, unique IDs under the result policy, finite scores, and a deterministic tie-break.

Filter meaning. A backend declares whether filters are pre-search complete, post-search heuristic, or partition-selecting. The interface cannot conflate them.

Snapshot isolation. A searcher opened from one release view does not observe later delta publication or activation.

Tombstone invisibility. No result references lineage dominated by a tombstone in the view.

Exact oracle. An exact backend's output is the reference for approximation tests at the same embedding/query/filter state.

B.4 Release laws

Behavior completeness. Every asset or policy capable of changing protected query outcomes is represented in release identity.

Immutability. No artifact referenced by a release changes bytes or semantic version after registration.

Atomic head change. Activation changes the resolver head in one compare-and-swap event.

Lease coherence. Every operation under a lease resolves all RAG artifacts from that release.

Drain safety. Activation never invalidates existing leases; retirement never occurs while leases remain.

Rollback purity. Rollback selects an immutable prior release and does not edit the failed release.

B.5 Query laws

Authorization domination. Every remote operation receiving evidence text is preceded on all plan paths by authorization for the subject, evidence IDs, provider, and release.

Authorized ranking contract. A result advertised as complete top-k is top-k within the authorized subcorpus.

Release coherence. Channel search, hydration, rerank, evidence admission, context, validation, citations, and presentation use the same release.

Fallback visibility. Any route deviation caused by failure or load shedding changes status/trace; it cannot be represented as normal intended success.

Evidence boundedness. Context and ledger limits are enforced before generator/presentation consumption.

Grounding closure. Every citation or grounded field references admitted evidence in the same evidence session and release.

Cancellation terminality. After terminal cancellation, no customer-visible nonterminal event is emitted for the turn; late results may only enter allowed caches/diagnostics.

B.6 Agent laws

Bounded progress. Every tool transition consumes an iteration or explicit resource budget; infinite internal loops are impossible.

Call idempotency. A repeated tool call ID with equal arguments returns the retained result; unequal arguments conflict.

Ledger stability. Repeated evidence retains label and lineage.

Tool authority. Model arguments cannot alter subject, release, data policy, or server-owned authorization.

Terminal validation. A model stop is not terminal success until product answer/presentation validators pass.

B.7 Stream laws

Snapshot-suffix equivalence. Snapshot through n plus suffix $n+1..m$ equals full replay through m.

Duplicate idempotence. Applying an event twice is equivalent to once.

Stale rejection. Lower/equal entity version cannot overwrite higher state.

Append exactness. An append patch applies once at its declared offset; gaps and overlaps fail or request resynchronization.

Terminal immutability. Terminal answer/citation/widget state cannot be overwritten except by an explicit higher-version correction event with product policy.

Release provenance. Source and widget entities retain the turn release ID through snapshot and live events.

B.8 Optimization laws

Frozen causal baseline. Candidate and incumbent share source barrier for behavior optimization; content refresh freezes behavior policy.

Dependency closure. No upstream-changed candidate reuses downstream artifacts whose semantic key is invalidated.

Exact pairing. Every comparison coordinate has explicit incumbent and candidate outcomes; missing or failed cells are not dropped.

Holdout integrity. Promotion evidence is computed on data not exposed to unrestricted candidate proposal.

Constraint precedence. Security and integrity failure cannot be compensated by relevance or cost gains.

No in-place promotion. A successful candidate produces a new release and activation request.

Appendix C. Extended Go API blueprint

The following interfaces are intentionally explicit. They are a design target, not a claim that all types should be added in one change.

C.1 Corpus capture

```go
package corpus

type ConnectorID string

type Capabilities struct {
    RepeatableSnapshot bool
    DurableCursor      bool
    OrderedChanges     bool
    EmitsDeletes       bool
    EmitsWatermarks    bool
    MaxReordering      time.Duration
}

type SnapshotRequest struct {
    Scope       string
    AsOf        *time.Time
    Prior       *SnapshotRef
    DataPolicy  policy.Ref
}

type ChangeSink interface {
    Put(context.Context, Change) error
}

type Snapshotter interface {
    ID() ConnectorID
    Capabilities() Capabilities
    Snapshot(context.Context, SnapshotRequest, ChangeSink) (Barrier, error)
}

type ChangeSource interface {
    ID() ConnectorID
    Capabilities() Capabilities
    Changes(context.Context, Cursor, ChangeSink) error
}
```

C.2 Derivation graph

```go
package derive

type Determinism uint8
const (
    Deterministic Determinism = iota
    DeterministicWithTranscript
    Stochastic
)

type Incrementality uint8
const (
    FullOnly Incrementality = iota
    PartitionLocal
    Pointwise
    BackendManaged
)

type Spec struct {
    Kind           string
    Version        string
    SemanticConfig json.RawMessage
    Determinism    Determinism
    Incremental    Incrementality
```

```
    Capabilities   []string
}

type Key struct {
    Operation digest.Digest
    Input     digest.Digest
    Deps      []digest.Digest
}

type ImpactPlan struct {
    ID          digest.Digest
    Base        release.ID
    Barrier     corpus.BarrierVector
    Add         []WorkItem
    Remove      []LineageRef
    Reuse       []artifact.Ref
    Expected    map[string]uint64
    Budget      resource.Budget
}
```

C.3 Index view

```
package index

type ResultCompleteness uint8
const (
    CompleteWithinFilter ResultCompleteness = iota
    PostFilteredHeuristic
    ApproximateWithinFilter
)

type SearchRequest struct {
    Query       rag.Query
    Vector      *rag.Vector
    Filter      Filter
    Limit       int
    SearchParam json.RawMessage
}

type SearchResult struct {
    Hits         []rag.Hit
    Completeness ResultCompleteness
    Trace         trace.SpanRef
}

type SnapshotSearcher interface {
    Search(context.Context, SearchRequest) (SearchResult, error)
    ReleaseID() release.ID
    Close() error
}

type ViewSpec struct {
    Backend    string
    Base       artifact.Ref
    Deltas     []artifact.Ref
    Tombstones artifact.Ref
    Spec       artifact.Ref
}
```

C.4 Build coordinator

```
package ragbuild

type Coordinator interface {
    Register(context.Context, Intent) (RunID, error)
    Start(context.Context, RunID) error
    Cancel(context.Context, RunID, string) error
    Resume(context.Context, RunID) error
    Status(context.Context, RunID) (Status, error)
    Events(context.Context, RunID, uint64, EventSink) error
}
```

```
type Intent struct {
    ID               IntentID
    Product          string
    Capture          corpus.SnapshotRequest
    BaseRelease      release.ID
    ReleaseSpec      release.Spec
    EvaluationPolicy rageval.PolicyID
}

type Status struct {
    Run        RunID
    State      State
    Barrier    *corpus.BarrierVector
    Plan       *artifact.Ref
    Progress   []StageProgress
    Warnings   []Warning
    Release    *release.ID
    LastEvent  uint64
}
```

C.5 Release registry

```
package release

type Registry interface {
    Register(context.Context, Manifest) error
    Get(context.Context, ID) (Manifest, error)
    Head(context.Context, string) (ID, error)
    Stage(context.Context, ID) error
    CompareAndSwap(context.Context, ActivationRequest) (Activation, error)
    Resolve(context.Context, ResolveRequest) (ID, error)
    Revoke(context.Context, RevokeRequest) error
}

type Loader interface {
    Open(context.Context, Manifest) (RuntimeResources, error)
}

type Manager interface {
    Acquire(context.Context, ResolveRequest) (*Lease, error)
    Preload(context.Context, ID) error
    Drain(context.Context, ID) error
}
```

C.6 Subject and authorization

```
package authz

type Subject struct {
    ID       string
    Tenant   string
    Claims   map[string][]string
    DataUse  []string
}

type Candidate struct {
    EvidenceID evidence.ID
    Metadata   map[string]string
    DataClass  string
}

type Certificate struct {
    ID          digest.Digest
    Release     release.ID
    SubjectHash digest.Digest
    Provider    string
    Allowed     []evidence.ID
    Policy      policy.Ref
}

type Authorizer interface {
```

```
    Authorize(context.Context, Subject, []Candidate, ProviderUse) (Certificate, error)
}
```

C.7 Query plans and interpreters

```
package query

type Request struct {
    ID             string
    ConversationID string
    TurnID         string
    Text           string
    RouteHint      string
    Locale         string
    Deadline       time.Time
}

type Plan struct {
    ID            PlanID
    Rewrite       []Operation
    Channels      []Channel
    Collapse      Operation
    Fusion        Operation
    Authorization Operation
    Rerank        *Operation
    Admission     Operation
    Fallbacks     []Fallback
}

type SearchInterpreter interface {
    Execute(context.Context, *release.Lease, authz.Subject, Request, Plan) Outcome
}
```

C.8 Evidence sessions

```
package evidence

type Scope uint8
const (
    OperationScope Scope = iota
    TurnScope
    ConversationScope
)

type Session interface {
    ID() SessionID
    ReleaseID() release.ID
    Add(context.Context, Candidate) (Admitted, bool, error)
    Snapshot() Snapshot
    Close() error
}

type Policy struct {
    Scope             Scope
    MaxDistinct       int
    MaxRunes          int
    AllowedKinds      []Kind
    StableLabelPrefix string
}
```

C.9 Answer interpreter

```
package answer

type Interpreter interface {
    Answer(context.Context, *release.Lease, authz.Subject, Request, EventSink) Outcome
}

type Outcome struct {
    Status      Status
    Release     release.ID
```

```
    Answer      *Validated
    Evidence    evidence.Snapshot
    Warnings    []query.Warning
    Usage       Usage
    Trace       trace.Ref
}
```

C.10 Agent adapter

```
package ragagent

type Host interface {
    RunTurn(context.Context, TurnRequest, EventSink) TurnOutcome
}

type TurnRequest struct {
    Lease        *release.Lease
    Subject      authz.Subject
    Conversation ConversationSnapshot
    Tools        ToolRegistry
    Policy       Policy
    Evidence     evidence.Session
}

type Policy struct {
    MaxIterations int
    Deadline      time.Duration
    ToolBudget    int
    FinalContract artifact.Ref
}
```

C.11 Stream envelope

```
package ragstream

type Operation string
const (
    Upsert    Operation = "upsert"
    Patch     Operation = "patch"
    Tombstone Operation = "tombstone"
)

type PatchMode string
const (
    Replace PatchMode = "replace"
    Append  PatchMode = "append"
    Merge   PatchMode = "merge"
)

type Event struct {
    SchemaVersion string
    ID            string
    StreamID      string
    Ordinal       uint64
    EntityID      string
    EntityVersion uint64
    Operation     Operation
    PatchMode     PatchMode
    Offset        *uint64
    Payload       json.RawMessage
    Release       release.ID
    Conversation  string
    Turn          string
    Causation     string
}
```

C.12 RAG optimization adapter

```
package ragspace

type ParameterRef struct {
```

```
    Layer string
    Name  string
    Path  string
}

type Space interface {
    ValidateIntervention(release.Spec, Intervention) error
    DependencyClosure(Intervention) []derive.NodeKind
    RequiredFidelity(Intervention) rageval.Fidelity
}

type Evaluator interface {
    Evaluate(context.Context, release.ID, rageval.Case, rageval.Repeat) (ragopt.CellOutcome, error)
}
```

Appendix D. State and transition tables

D.1 Release lifecycle

Current state	Event	Preconditions	Next state	Query acquisition
Registered	Verify	Manifest and artifacts available	Verified or Quarantined	No
Verified	Stage	Loader opens all required resources	Staged	No
Staged	Activate CAS	Expected head matches; gates eligible	Active	Yes
Active	Superseded	Another release activated	Draining	No new leases
Draining	Last lease closed	Retention conditions met	Retired	No
Any non-retired	Revoke	Emergency authority	Revoked/ Quarantined	No; in-flight policy-specific
Retired	Purge	Retention and audit permit	Purged metadata/material as allowed	No

D.2 Build lifecycle

State	Allowed events	Terminal?	Resume behavior
Pending	start, cancel	No	Start from intent
Capturing	source change, barrier, retry, cancel, fail	No	Resume connector cursor
Planning	plan commit, cancel, fail	No	Recompute and verify same plan ID
Deriving	cache hit, item commit, retry, quarantine, stage seal, cancel, fail	No	Reconstruct pending items from plan/events
Indexing	item commit, checkpoint, stage seal, cancel, fail	No	Open verified partial artifacts
Verifying	verification result, fail	No	Repeat pure verification
Evaluating	evaluation cells, gate result, cancel, fail	No	Resume native evaluator/runstore
Published	release registered	Yes	Return release
Failed	retry/resume under policy	Yes for attempt	New attempt or resumed same intent
Cancelled	none except explicit new resume command	Yes	Committed cache remains reusable

D.3 Query lifecycle

State	Principal events	Failure/fallback
Received	acquire release	no release -> failed
Rewrite	local/provider rewrite	original-query fallback or fail
Channels	start/finish/timeout channel	declared partial/fallback/fail
Rank	collapse/fuse/filter	invariant failure -> fail
Rerank	authorize/hydrate/call/blend	fused fallback or fail
Admit	budget/diversity/lineage	empty -> abstain or continue policy
Generate	provider stream/result	retry, partial evidence, fail
Validate	parse/ground/repair	repair, abstain, fail
Emit	persist UI/terminal event	product policy: fatal or retry
Complete	close lease	terminal
Cancelled	persist cancellation, close lease	terminal

D.4 Evidence-session scopes

Scope	Typical adopter	Release policy	Reuse behavior
Operation	`ragkit/answering`	one request lease	no cross-call reuse
Run/turn	GEC and RAG-TTC tool turn	one turn lease	stable labels across tool calls
Conversation	current Garden session pattern	conversation lease or explicit epochs	requires stale/release policy

D.5 Frontend patch modes

Mode	Semantics	Idempotence requirement	Recommended use
Replace	payload replaces field/entity value	naturally idempotent by version	snapshots, reconnect, terminal values
Merge	typed field merge	idempotent only for overwrite/set fields	sparse terminal metadata
Append	append at exact offset	dedupe event ID and verify offset	live text streaming only
Tombstone	remove/hide entity at version	monotone until valid recreation	deletions/retractions

Appendix E. Optimization campaign catalog

This appendix turns the optimization semantics of Part IV into executable study designs. It is deliberately more prescriptive than the main text. Each campaign identifies the intervention, the affected dependency closure, the required evaluation unit, the validity threats, and the promotion gates. The purpose is not to prescribe permanent threshold values. It is to prevent incomparable experiments from being presented as one undifferentiated tuning loop.

E.1 A campaign is a typed intervention, not a parameter sweep

Let a production release specification be a value R in a typed configuration space. A candidate is not merely a partial map from string names to numbers. It is an intervention

$$I : R \rightsquigarrow R'$$

with five declarations:

1. **target nodes** - the components directly changed;
2. **dependency closure** - the artifacts and evaluators invalidated by the change;
3. **semantic class** - operational, approximation, relevance, knowledge, policy, interaction, or presentation changing;
4. **claimed invariants** - properties asserted to remain equal or within a tolerance;
5. **required evidence** - the lowest fidelity at which the claim can be decided.

A campaign is then a finite or adaptive family of interventions evaluated under a common protocol:

$$C = \left(R_0, I, W, M, G, A\right),$$

where R_0 is the baseline release, I is the candidate generator, W is the workload and sampling design, M is the measurement plan, G is an ordered gate program, and A is the allocation policy. The allocation policy may be a grid, random search, Bayesian optimization, successive halving, racing, or a manually curated set. The semantics do not depend on the search algorithm. They depend on whether the candidates, cells, and gates retain their identities and causal scopes.

Every candidate should have a machine-readable declaration resembling:

```
type Intervention struct {
    ID               InterventionID
    Baseline         release.ID
    Patch            release.SpecPatch
    Targets          []derive.NodeKind
    Closure          []derive.NodeKind
    SemanticClass    []SemanticClass
    ClaimedInvariants []InvariantClaim
    Workload         eval.WorkloadID
    Fidelity         eval.Fidelity
    Repeats          eval.RepeatPolicy
}
```

The closure must be computed from the release dependency graph, not entered by hand. A change to the embedding model invalidates vector representations, vector indexes, release manifests, retrieval evaluation, and all answer/session evaluations that depend on retrieval. It need not invalidate lexical-index bytes when lexical analysis is unchanged. A change to `efSearch` invalidates no corpus derivation, but it invalidates vector-search observations and every downstream outcome that may depend on the candidate set. A change to a frontend widget projection invalidates session and presentation evaluation while leaving retrieval metrics unchanged.

E.2 Parameter families and minimum evaluation levels

The following catalog is a starting schema for `ragopt/ragspace`. "Minimum fidelity" means the first level at which a candidate can possibly be accepted. Lower levels can still reject it cheaply.

Layer	Parameter family	Typical semantic class	Minimum fidelity
Source	connector and capture protocol	knowledge, reliability	refresh simulation
Source	inclusion, exclusion, access policy	policy, security, knowledge	policy suite plus retrieval
Normalization	canonical text and metadata	knowledge, relevance	retrieval plus lineage
Chunking	algorithm, size, overlap, boundaries	relevance, knowledge projection	retrieval and answer
Representation	raw, context, summary, questions, entities	relevance, knowledge projection	retrieval and answer
Representation generation	model, prompt, decoding, reuse	knowledge, stochastic	repeated answer/retrieval
Embedding	model, dimension, normalization	relevance	retrieval and answer
Lexical index	analyzer, fields, boosts	relevance	retrieval
Vector index	exact or ANN backend	approximation, operational	oracle plus load
Vector construction	graph, quantization, partitions	approximation, operational	oracle plus scale
Rewrite	synonyms, variants, HyDE, route	relevance, stochastic	retrieval and answer
Candidate generation	channel depth and overfetch	relevance, operational	retrieval plus policy
Filtering	subject, source, role, kind	policy, security, relevance	security plus retrieval
Fusion	rank constant, weights, tie rules	relevance	retrieval
Reranking	model, pool, blend, fallback	relevance, disclosure, reliability	retrieval, answer, failure
Context	token/rune budget, diversity, ordering	answer	answer
Answer	model, prompt, contract, repair	answer, stochastic	repeated answer
Agent	tools, routing, iteration and retry	interaction	session
Serving	deadlines, concurrency, cache	operational, sometimes outcome	load and failure
Release	rollout, pinning, epoch scope	operational, experimental	concurrency and canary
Presentation	event and widget policy	user outcome	frontend/session

A useful rule is that a candidate must be evaluated at the highest fidelity required by any changed node in its dependency closure. This prevents a retriever-only metric from promoting a prompt change, and prevents an exact-oracle ANN test from promoting a new chunking scheme whose relevance semantics changed.

E.3 Invalidation examples

The dependency graph should produce explicit invalidation records. Examples follow.

E.3.1 Chunk-size intervention

Changing fixed chunks from 1,200 to 800 runes directly changes chunk derivation. The closure normally includes:

- chunk records and chunk identities;
- every representation attached to changed chunks;
- generated-representation cache keys when the source chunk is part of the prompt input;
- embeddings of changed representations;
- lexical and vector index partitions for changed representations;
- release manifest and aggregate digests;
- retrieval judgments whose evidence unit is a chunk ID;
- answer and session fixtures whose accepted citations name old chunks.

The source snapshot and normalized document IDs remain stable. A correct impact planner records this distinction so that source capture and document normalization can be reused.

E.3.2 Fusion-weight intervention

Changing a vector-channel weight affects only query policy and downstream observations. No index needs rebuilding. The closure includes ranked retrieval, evidence admission, answer generation, agent behavior, and frontend outputs. Because the candidate can change which evidence is disclosed to a generator, cached generated answers from the baseline are not valid candidate observations.

E.3.3 Reranker-provider intervention

Changing a reranker model or endpoint changes ranking, latency, cost, disclosure, failure behavior, and possibly jurisdiction. The intervention therefore cannot be classified as relevance-only. It requires provider identity in release material, a disclosure-policy check, repeated failure testing, and a fail-open/fail-closed declaration. The candidate is invalid if its remote stage receives text that the subject is not authorized to disclose.

E.3.4 Deadline intervention

Increasing concurrency or reducing a stage timeout may be operational under an idealized no-timeout semantics, but it is outcome-changing in a real service whenever the timeout can trigger partial retrieval, reranker fallback, answer truncation, or cancellation. The intervention may claim answer-distribution equivalence only after a workload test shows the altered deadline is not reached in the declared operating envelope. Otherwise, timeout behavior is part of the denotation and requires answer/session evaluation.

E.4 Campaign 1: GEC fusion, synonyms, and reranking

E.4.1 Question

The practical question is not “what is the best RRF constant?” It is:

> For each authorized administrative query class, which combination of lexical rewriting, channel allocation, fusion, and reranking improves evidence and answer quality without increasing unauthorized disclosure, tail latency, cost, or failure severity?

The current GEC sweep varies a small RRF/vector-weight space against a fixed corpus. That is an appropriate first retrieval experiment, but it omits three coupled effects: synonyms alter only lexical query behavior; reranking can compensate for or amplify first-stage changes; and post-retrieval authorization changes effective candidate depth.

E.4.2 Candidate factors

A first complete campaign should vary a bounded, reviewable set:

- synonym-set revision and whether expansion is disjunctive or weighted;
- lexical and vector channel depths;
- preauthorization overfetch strategy;
- RRF rank constant and channel weights;
- representation kinds admitted to each channel;
- rerank pool size;
- rerank/fused blending policy;
- reranker model and local versus remote execution;
- fail-open or fail-closed behavior by query class;
- context evidence count and diversity constraints.

Authorization policy is not a tunable quality factor. It is a hard constraint and release input. The campaign may compare *implementations* of the same policy, such as pushdown filters versus local pre-rerank filtering, but must prove policy equivalence.

E.4.3 Workload design

The workload should be stratified by:

- subject role and source scope;
- query intent: exact policy lookup, broad procedure, troubleshooting, cross-document synthesis, and unsupported question;
- lexical characteristics: exact terminology, synonym-dependent wording, acronym, misspelling, and paraphrase;
- corpus density: one authoritative source, many near-duplicates, conflicting revisions, and no relevant source;
- reranker sensitivity: cases where lexical and vector channels disagree;
- temporal status: current, superseded, recently changed, and tombstoned material.

Every test case carries an authorized evidence set, a forbidden-disclosure set, graded relevance judgments, expected source authority, and answer/abstention expectations. Forbidden evidence is necessary: a system can improve conventional recall by retrieving material the subject must not see.

E.4.4 Measurements

Retrieve-level measurements include graded nDCG, recall at evidence budget, reciprocal rank of first authoritative chunk, duplicate-source concentration, and authorized-candidate survival. Intensional measurements include candidate counts before and after policy, remote bytes disclosed, fallback path, rewrite expansion, reranker calls, and release lineage. Answer measurements include citation support, contradiction, authority selection, completeness, abstention appropriateness, latency, and cost.

E.4.5 Ordered gates

1. **Security gate.** Zero forbidden chunks disclosed to any remote provider and zero returned to the caller.

2. **Integrity gate.** Every evidence and answer citation belongs to the pinned release and authorized candidate set.
3. **Reliability gate.** Declared fallback behavior holds under reranker timeout, provider error, malformed score, and partial cancellation.
4. **Retrieval gate.** Noninferior authoritative recall and nDCG on every protected stratum; aggregate improvement alone is insufficient.
5. **Answer gate.** Noninferior grounding and abstention; material completeness improvement on targeted classes.
6. **Operational gate.** Tail-latency, provider-call, and cost budgets.
7. **Canary gate.** No protected-stratum regression under production traffic and no unexplained trace-distribution shift.

A winning candidate may be Pareto-superior rather than scalar-best. For example, a local reranker may have slightly lower nDCG but eliminate remote disclosure and substantially reduce latency variance. The promotion report should show that trade rather than hiding it in one score.

E.5 Campaign 2: exact-to-ANN certification for RAG-TTC

E.5.1 Semantic claim

An ANN backend does not preserve the exact ranked-list semantics. Its claim is relative to an exact oracle over a declared workload and operating envelope:

$$Pr\left[Recall@k\left(A(q), E(q)\right) \geq \rho\right] \geq 1 - \alpha,$$

where A is the approximate backend, E the exact backend, ρ the required recall threshold, and α the tolerated violation probability. This claim must be conditioned on corpus size, vector distribution, filter selectivity, concurrency, update state, and hardware class.

E.5.2 Factors

- backend implementation and version;
- distance metric and vector normalization;
- graph construction parameters such as M and `efConstruction`;
- query parameter such as `efSearch`;
- quantization or compression mode;
- shard/partition count;
- filter pushdown strategy;
- base-plus-delta overlay sizes;
- deleted-vector fraction and compaction state;
- concurrency and memory residency.

E.5.3 Workloads

Use at least four workload families:

1. **Natural queries** sampled from evaluated TTC tasks and production-like traces after privacy processing.
2. **Adversarial nearest-neighbor probes** around dense clusters, duplicate representations, ties, and near-boundary vectors.
3. **Filter-selective queries** for narrow source, representation, product, or policy filters.
4. **Dynamic-state queries** after controlled upserts, deletes, tombstones, and compaction.

Measure oracle recall at multiple k values, score/rank distortion, deterministic tie behavior, p50/p95/p99 latency, throughput, memory, build duration, update cost, compaction cost, and failure recovery. Repeat the build under identical inputs and compare the declared reproducibility class. Bit-identical graph bytes may be unnecessary, but ranked results under fixed seeds and environment should satisfy a stated tolerance.

E.5.4 Gates

- exact metric and normalization compatibility;
- no deleted or unauthorized item returned;
- recall lower confidence bound above the required threshold for every critical stratum;
- bounded tail latency at target concurrency;
- bounded memory and build time;
- successful checkpoint, reopen, and crash recovery;
- full-build and base-plus-delta query equivalence within the declared ANN tolerance;
- no degradation of downstream answer grounding beyond its own noninferiority margin.

The last gate is essential. A one-point recall loss can be irrelevant when lexical fusion recovers the evidence, or catastrophic when the lost neighbor is the only authoritative source. ANN promotion therefore requires both oracle-relative and task-relative evidence.

E.6 Campaign 3: chunking and representation design

E.6.1 Why this is a joint experiment

Chunking and representation generation determine the searchable knowledge projection. They cannot be optimized independently of channel depth, context admission, and answer policy. Smaller chunks may improve localization but fragment definitions. Larger chunks may preserve context but waste evidence budget. Generated questions may raise recall for paraphrases while adding cost, noise, and stale derived claims. Contextual representations may improve retrieval while never being suitable as direct evidence.

E.6.2 Candidate families

A disciplined campaign should use a small factorial structure rather than an unconstrained Cartesian product:

- baseline Markdown-heading chunks;
- smaller heading-aware chunks with overlap;
- hierarchical parent/child chunks;
- raw only;
- raw plus breadcrumb/context representation;
- raw plus generated questions;
- raw plus context and questions;
- retrieval at chunk, representation, or parent-child level;
- evidence admission at source chunk or parent section level.

Each generated representation must retain its producing prompt, model, decoding parameters, source chunk, and generation transcript or retained deterministic output. It is searchable derived data, not authority. A final answer cites the source evidence from which the representation was derived.

E.6.3 Evaluation units

Use three linked labels:

- **source-span relevance**, identifying authoritative byte or structural spans;
- **retrieval-unit relevance**, identifying which chunks/representations make those spans discoverable;
- **answer support**, identifying which admitted evidence actually supports claims.

This avoids freezing the benchmark to baseline chunk IDs. When chunk boundaries change, source-span labels can be projected onto the candidate's chunk graph. The projection rule must be versioned, for example by overlap threshold, structural containment, or explicit assessor mapping.

E.6.4 Cost model

Report storage, representation-generation calls, embedding calls, index build duration, index size, query channel operations, reranking tokens, context tokens, and refresh amplification. Refresh amplification is particularly important:

$$A_{refresh} = \frac{\text{derived items recomputed}}{\text{source items materially changed}}.$$

A chunking scheme that improves frozen-corpus nDCG but turns a one-line edit into thousands of regenerated representations may be unacceptable for a frequently changing corpus.

E.6.5 Promotion rule

A candidate must improve a declared relevance or answer stratum, remain noninferior on protected strata, satisfy freshness and cost envelopes, and pass incremental/full-build equivalence. The campaign should not promote an opaque "best chunk size." It should produce a release-specific design choice with a workload and change-rate envelope.

E.7 Campaign 4: refresh, overlay, and compaction policy

Retrieval quality experiments usually freeze the corpus. A production maintenance campaign varies the *trajectory* of the corpus and system load.

E.7.1 Input process

Generate or replay a timestamped revision stream containing:

- new documents;
- small edits within existing documents;
- large replacements;
- metadata-only policy changes;
- deletions and later recreations;
- source reordering and duplicate delivery;
- delayed and out-of-order changes;
- barriers and connector restarts;
- bursts followed by idle periods.

The simulation should preserve source-specific revision identities and event-time versus observation-time distinctions. A delete must name the object and revision relationship it supersedes; "file absent in one poll" is not always a deletion.

E.7.2 Policy factors

- polling or subscription cadence;
- debounce/coalescing window;
- maximum change batch;
- impact-plan granularity;

- delta-overlay activation threshold;
- maximum overlay depth or tombstone ratio;
- compaction schedule;
- full-rebuild cadence;
- failure retry and quarantine policy;
- staleness budget by source class;
- activation gate strictness.

E.7.3 Correctness oracle

At every barrier b, compare the activated maintained view with a clean rebuild from the source snapshot at b. Exact backends should be observationally equal for normalized documents, chunks, representations, filters, and ranked output under a deterministic query set. Approximate backends should compare through an exact logical oracle plus their declared approximation relation.

The simulation must also check absence. Deleted material must not be retrievable, disclosed, cited, or projected. Tombstone correctness is a first-class property, not the complement of recall.

E.7.4 Operational measurements

- source-to-captured, captured-to-built, and built-to-active lag;
- percent of time inside freshness SLO;
- activation frequency and skipped/coalesced revisions;
- work amplification and cache hit rate;
- build queue depth and age;
- overlay size, tombstone ratio, and compaction pause;
- query latency under concurrent maintenance;
- recovery point after injected crashes;
- time and work to converge after connector outage;
- number of releases retained and storage pressure.

E.7.5 Failure injections

Crash after every durable build event, duplicate every source change, reorder changes inside the connector's allowed window, lose a worker lease, corrupt a staged artifact, fail one provider batch, activate concurrently from two coordinators, revoke the active release, and reconnect a frontend during activation. The expected behavior is expressed in machine invariants, not in log-message matching.

E.8 Campaign 5: Garden agent and widget calibration

Garden demonstrates a user-level RAG system whose output includes choices, facts, citations, and widgets across multiple turns. Retrieval-only evaluation is insufficient.

E.8.1 Experimental unit

The unit is a scripted or assessor-driven conversation with:

- an initial user goal;
- hidden constraints revealed over turns;
- expected intent transitions;
- authoritative structured facts;
- acceptable source evidence;
- expected clarifying questions or abstention;

- admissible widgets and fields;
- terminal user outcome.

Each conversation runs against one pinned release or against explicitly modeled evidence epochs. Repeats use controlled provider seeds when supported and always retain transcripts and traces.

E.8.2 Factors

- intent classifier/router;
- structured-first versus retrieve-first ordering;
- connected-retrieval policy;
- search tool description and agent prompt;
- maximum tool iterations;
- evidence novelty threshold;
- structured-fact augmentation rules;
- grounded-widget admission and conflict suppression;
- answer model and decoding policy;
- conversation release scope.

E.8.3 Session metrics

Measure task completion, number of turns, unnecessary tool calls, unsupported claims, fact conflicts, widget eligibility, field-level provenance, stale evidence reuse, clarification quality, abandonment proxy, total latency, and cost. Inspect the trajectory: two sessions may end with identical text while one leaked stale facts, retried a tool repeatedly, or crossed release epochs.

E.8.4 Gates

- no widget field without admissible provenance;
- no conflict hidden by projection;
- no structured fact or chunk from a different release epoch unless the product explicitly marks it;
- noninferior task completion and grounding;
- bounded turn/tool count and latency;
- no increase in unsafe or misleading terminal outcomes;
- frontend snapshot-plus-suffix convergence for every calibrated session.

E.9 Multi-fidelity allocation

Evaluation cost grows sharply from static laws to canary traffic. A candidate should advance only when the current fidelity can no longer decide its eligibility.

A practical ladder is:

1. **Schema and dependency validation.** Is the intervention well typed, and is its invalidation closure complete?
2. **Deterministic laws.** Can artifacts open, lineage reconcile, filters preserve policy, and reducers converge?
3. **Retrieval cells.** Does the candidate improve or preserve ranked evidence on a stratified suite?
4. **Repeated answer cells.** Does the distribution of answers, grounding, latency, and cost satisfy gates?
5. **Session calibration.** Do agent trajectories and frontend projections remain valid?
6. **Refresh and load simulation.** Does the candidate operate under corpus evolution and concurrency?
7. **Shadow.** What traces would production requests produce without affecting users?
8. **Canary.** Does a small eligible subject/request population satisfy online gates?

9. **Promotion.** Does CAS activation change the intended release pointer with rollback ready?

Successive halving is appropriate only within comparable candidates. It is unsafe to race a cheap retrieval-only candidate against an agent-policy candidate using one early scalar. The allocator should group candidates by minimum fidelity and semantic class.

E.10 Statistical decision rules

Exact paired cells should remain the primitive `ragopt` unit. For each case i, repeat r, baseline b, and candidate c, retain the paired difference

$$\Delta_{i,r,m} = m(c,i,r) - m(b,i,r)$$

for every metric m. Pairing controls case difficulty and, where provider control permits, shared randomness. Aggregate reports should preserve strata and uncertainty rather than only pooled means.

Recommended rules include:

- exact or permutation tests for deterministic paired rankings;
- paired bootstrap intervals over cases for nDCG, recall, latency, and cost;
- cluster bootstrap at conversation or source level when observations are dependent;
- noninferiority margins for protected quality and safety metrics;
- lower confidence bounds for recall and success rates;
- upper confidence bounds for latency, cost, failure, and disclosure;
- sequential confidence methods for canaries when repeated peeking is expected;
- explicit multiplicity control or hierarchical gate ordering for large candidate families.

The gate program should evaluate hard constraints before preferences. A useful partial order is:

$$\text{security} \prec \text{integrity} \prec \text{reliability} \prec \text{quality} \prec \text{latency/cost} \prec \text{preference}.$$

A failure in an earlier class makes later aggregate gains irrelevant. Among surviving candidates, report the Pareto frontier. A human or product policy then chooses a release; `ragopt` should not conceal that policy inside an unexplained weighted sum.

E.11 Temporal holdouts and corpus leakage

RAG evaluation is especially vulnerable to temporal leakage. A benchmark built from the same corpus snapshot used for candidate engineering may reward memorized source structure, stale aliases, or answers that are no longer authoritative. Maintain at least:

- a development workload tied to a known source snapshot;
- a hidden static holdout;
- a future-revision holdout consisting of changes captured after the candidate design began;
- a deletion/supersession suite;
- a production shadow sample with policy-safe retention.

Generated evaluation questions must retain their source revision and generation procedure. They should not be treated as independent evidence of quality when the same model/prompt family generated candidate representations. Correlated synthetic artifacts can make a representation scheme appear better than it is.

E.12 Promotion report schema

A promotion report should be sufficient to reconstruct the decision without rerunning the campaign. It contains:

- baseline and candidate release specifications;
- intervention and dependency closure;
- source snapshot and workload identities;
- evaluator and judge identities;
- exact paired-cell coverage and missingness;
- metric distributions by protected stratum;
- every gate result with evidence references;
- failure and fallback distributions;
- refresh/load envelope;
- security/disclosure attestations;
- Pareto comparison;
- canary routing and rollback plan;
- decision authority and timestamp.

The report does not claim universal optimality. It states that a candidate is eligible, under a declared workload and operating envelope, to become the next active release.

Appendix F. Verification, testing, and model checking

The runtime semantics are useful only if they become executable obligations. This appendix maps each obligation to the cheapest verification technique that can detect its violation. Ordinary unit tests remain important, but concurrency, replay, corpus evolution, and stochastic providers require a layered strategy.

F.1 Verification hierarchy

Technique	Best target	Typical failure found
Example unit test	local branch or transformation	wrong field, score, or event
Golden/differential fixture	compatibility during migration	changed rank, citation, or trace
Property test	algebraic law over many inputs	non-idempotent reducer, unstable identity
Fuzz test	parser and state-machine boundary	malformed manifest, panic, invalid transition
State-machine model test	concurrent lifecycle	double activation, leaked lease, lost update
Fault-injection test	durable workflow	non-resumable build, duplicate side effect
Load/soak test	resource and deadline envelope	queue collapse, tail amplification
Statistical test	stochastic provider behavior	quality or latency regression
Formal model check	finite concurrency protocol	safety/liveness counterexample
Proof or proof assistant	small mathematical kernel	invalid law or incomplete precondition

The architecture should expose pure reducers and transition validators so that most state behavior can be tested without starting servers or providers. Integration tests then verify that storage and transport adapters implement the same events.

F.2 Source and snapshot test matrix

F.2.1 Example cases

- first observation of a source object produces one upsert;
- identical repeated observation produces no semantic change;
- a newer content revision produces one replacement;
- metadata-only policy revision changes the policy projection without silently retaining an old authorization certificate;
- delete removes the object from the next complete snapshot;
- delete followed by recreate produces a distinct revision lineage according to source semantics;
- a barrier closes exactly the prefix promised by the connector;
- a restarted connector resumes from a committed cursor without dropping or inventing changes;
- an out-of-order stale revision cannot overwrite a newer accepted revision;
- two aliases for one source object normalize to the declared canonical identity.

F.2.2 Properties

For a connector whose delivery contract permits duplicates:

$$reduce(S, e, e) = reduce(S, e).$$

For any permutation that preserves the connector's causal order and barrier rules, reducing the event multiset yields the same captured snapshot. For a source snapshot digest d, repeated serialization and capture produce the same digest. A barrier identity binds the exact source frontier, admission-policy revision, and connector state used by the build.

F.2.3 Generators

Generate small source worlds of documents with random content, metadata, roles, and revisions. Generate event traces containing duplicates, delayed events, deletes, recreations, and barriers. Shrinking should preserve the failing causal relation; otherwise the minimal counterexample may become an invalid trace. Store every discovered seed as a regression fixture.

F.3 Derivation and incremental-maintenance tests

F.3.1 Local deterministic laws

For normalization, chunking, representation projection, and embedding-cache keys:

- determinism under repeated execution;
- stable identity under irrelevant input ordering;
- sensitivity to every behaviorally material input;
- locality: an unchanged document does not change another document's derived IDs;
- range validity and exact source-span reconstruction;
- representation-to-source lineage totality;
- no representation may be admitted as authoritative evidence without resolving to a source evidence object.

F.3.2 Incremental/full equivalence

For randomly generated source state D and valid change batch ΔD:

$$normalize\left(F_{inc}\left(F\left(D\right),\Delta D\right)\right)=normalize\left(F\left(D\oplus\Delta D\right)\right).$$

Normalization removes irrelevant backend ordering and serial-format variation while retaining all observable content, identity, policy, and search semantics. Run the property after every barrier and after arbitrary compaction points. Include deletes, boundary-shifting edits, generated representations, failed/quarantined derivations, and cache hits.

F.3.3 Crash-point enumeration

Instrument the build coordinator so that a test can terminate it immediately after every durable event:

- source cursor commit;
- impact-plan commit;
- work-item lease;
- provider response receipt;
- artifact write;
- artifact digest verification;
- item commit;
- stage seal;
- index checkpoint;
- evaluation cell;
- release registration;
- activation CAS.

Restart from the durable ledger and verify that the final registered release and visible activation effect equal an uninterrupted execution. Provider calls may be repeated; publication and activation must not produce duplicate semantic effects.

F.4 Index-backend conformance suite

Every backend implements a capability-specific suite.

F.4.1 Common exact semantics

- inserted items are searchable according to the metric;
- deleted/tombstoned items are never returned;
- filters are sound: every result satisfies the filter;
- a stable total order resolves equal scores;
- scores are finite and normalized as declared;
- opening a checkpoint reproduces the same observable index;
- concurrent snapshot readers see one committed view;
- compaction preserves logical contents;
- malformed or incompatible manifests fail closed.

F.4.2 Exact backend oracle

For small generated indexes, compare against a pure in-memory implementation. Exhaustively enumerate queries from a finite vector/term domain where feasible. This oracle should be intentionally simple, not optimized.

F.4.3 Approximate backend relation

An ANN backend is not tested for equality with the exact oracle. It is tested for:

- sound membership and filters;
- recall/rank-distance relation under declared parameters;
- deterministic or distributional reproducibility class;
- monotonicity expectations where valid, such as nondecreasing search effort not materially reducing recall;
- behavior under insert/delete/overlay/compaction;
- no catastrophic stratum with hidden zero recall.

Performance assertions are separated from logical conformance so a slow CI host cannot make correctness flaky.

F.5 Query algebra property tests

Generate finite channel rankings with ties, duplicate chunks through multiple representations, policy labels, and finite scores. Then verify:

- collapse returns at most one candidate per collapse identity;
- fusion is deterministic under map/input iteration permutation;
- total-order comparator is antisymmetric, transitive, and total;
- every fused contribution refers to an input rank;
- policy filtering is monotone: tightening policy never introduces a candidate;
- authorization precedes every remote text-bearing event;
- evidence admission never exceeds declared count/token/rune budgets;

- admitted evidence is a subsequence of the policy-valid ranked candidates unless diversification explicitly documents a reorder;
- citations resolve only to admitted evidence;
- fallback paths are explicit in the trace and cannot masquerade as the intended stage.

RRF deserves a direct reference implementation with rational or high-precision arithmetic for small cases. Production floating-point output can then be compared after the declared rounding and tie policy.

F.6 Remote-disclosure spy tests

Implement provider spies for rewrite, embedding, reranking, and generation. Each spy records the exact text, metadata, subject certificate, release, and purpose presented to it. The test corpus contains uniquely marked secrets by role and source scope. For every generated subject/query pair:

1. run the query or agent turn;
2. collect all remote disclosures;
3. assert that every disclosed item is authorized for that subject, provider, purpose, and jurisdiction;
4. assert that returned evidence is a subset of authorized, admitted material;
5. assert that trace redaction does not itself leak the marked secret.

This suite should run against GEC before any relevance migration. It converts a subtle ordering defect into a mechanically testable security property.

F.7 Release and activation state-machine tests

Model commands such as register, verify, stage, activate, acquire, release, revoke, retire, and purge. Generate command sequences against both a pure model and the real registry implementation.

Key invariants:

- at most one active release per routing key;
- activation succeeds only from an eligible staged release;
- a failed compare-and-swap does not change the head;
- a lease returns exactly the release that was active at acquisition linearization;
- all evidence and events under a lease carry that release;
- no new lease is granted to draining, retired, revoked, or quarantined releases;
- retirement cannot purge resources while leases remain;
- rollback is another validated activation, not mutation of an old release;
- revocation behavior for in-flight leases follows the declared policy;
- registry replay reconstructs the same head and lease-independent state.

Run concurrent histories with randomized scheduling. Record invocation and response times and check linearizability for the active-head register and lease acquisition. A lightweight model checker can enumerate short histories; a stress harness can explore longer randomized histories.

F.8 Query-machine transition tests

The query interpreter should expose a pure transition function or test seam over stage outcomes. Generate combinations of:

- rewrite success, timeout, malformed result, and fallback;
- lexical/vector/connected channel success, partial timeout, and empty result;
- reranker success, timeout, bad scores, and provider denial;
- evidence empty, over budget, duplicate, or stale;

- generator success, invalid contract, repair success/failure, and stream interruption;
- client cancellation at every transition;
- terminal event persistence success/failure.

Verify that every run reaches one terminal class within the model's assumptions, closes its release lease exactly once, records every fallback, and never emits a final answer before validation. Deadline allocation must be monotone in elapsed time and never produce a negative child budget.

F.9 Agent and tool-loop tests

Agentic RAG adds replay and nontermination hazards. Test:

- one logical tool call ID executes at most one semantic search effect despite transport replay;
- repeated tool calls can reuse the turn evidence ledger without relabeling existing evidence;
- a tool result belongs to the turn release;
- zero-search completion is represented distinctly from search failure;
- maximum-iteration exhaustion produces a terminal, inspectable result;
- cancellation interrupts model and tool work and persists a terminal event;
- malformed tool arguments do not mutate evidence state;
- connected retrieval failures follow route policy;
- final citations resolve to evidence actually accumulated during the turn;
- conversation-scoped reuse across a release change creates a declared new evidence epoch or is rejected.

A small-step reference interpreter can generate the expected trace from scripted model choices. Product adapters are differential-tested against it while retaining product-specific tools and projections.

F.10 Frontend reducer tests

The frontend projection contract should be testable in Go/TypeScript against the same event schema.

F.10.1 Core convergence law

For a snapshot S_n at ordinal n and suffix events $e_{n+1}, \ldots, e_m$:

$$reduce\left(S_n, \left[e_{n+1}, \ldots, e_m\right)\right) = S_m.$$

The result must remain equal under duplicate delivery and any reordering permitted by the transport buffer rules.

F.10.2 Required cases

- live events arrive before snapshot hydration;
- duplicate event ID before and after hydration;
- stale entity version with larger global ordinal;
- fresh entity version delivered out of order;
- append patch replayed twice;
- append patch with wrong offset;
- snapshot truncation followed by a suffix beyond retained history;
- reconnect during release activation;
- cancellation and error terminal events;
- widget retraction or tombstone;
- server replay from an event cursor;
- client with unsupported schema version.

Append patches should carry an expected offset or segment identity. The reducer either applies exactly once or requests resynchronization; it must not silently duplicate text.

F.10.3 Cross-language fixture

Serialize event traces and expected terminal state into a language-neutral fixture. Run the authoritative reducer in Go and the browser reducer in TypeScript. The normalized states must be equal. This catches drift between backend semantics and frontend convenience code.

F.11 Differential migration fixtures

Before deleting duplicate implementations, capture behavior from the current products. A fixture contains input corpus, bundle/release inputs, subject, query or conversation, provider stubs, expected ranked candidates, evidence labels, answer contract result, observations, and frontend events.

Run current and target implementations side by side. Classify differences:

- intended security correction;
- intended deterministic-order correction;
- intended release-lineage addition;
- acceptable trace enrichment;
- unacceptable behavior regression;
- nondeterministic provider variation requiring repeated comparison.

Do not require byte equality where the migration intentionally changes identities or event envelopes. Define a normalization that preserves the semantic layer being claimed. For example, GEC may intentionally filter before reranking; its remote-disclosure trace must differ, while its authorized result ranking should remain compatible or be reevaluated.

F.12 Load, soak, and chaos verification

A production RAG system has coupled queues: source capture, derivation, embedding/reranking provider calls, build publication, query channels, generation streams, event persistence, and WebSocket delivery. Test the system under realistic joint load rather than isolated microbenchmarks.

Scenarios include:

- steady query load during a large corpus refresh;
- burst of source changes while a compaction runs;
- provider rate-limit reduction;
- slow frontend consumers;
- registry/storage latency;
- repeated failed candidate builds;
- release activation during long agent turns;
- old-release drain with new-release canary load;
- connector outage and catch-up;
- mass cancellation.

Measure queue age, admission rejections, deadline exhaustion by stage, release lease duration, memory, file descriptors, provider concurrency, event lag, and freshness. Soak tests should cover retention and compaction cycles; otherwise resource leaks and ever-growing ledgers remain invisible.

F.13 TLA+ model outline

The release/build protocol is small enough for a bounded TLA+ model. Suggested variables:

```
active            routing key -> release or None
releaseState      release -> Registered | Verified | Staged | Active |
                             Draining | Retired | Revoked | Quarantined
leases            query -> release or None
buildState        build -> lifecycle state
registeredByBuild build -> release or None
frontier          connector -> cursor/barrier
published         artifact digest set
```

Actions include `Register`, `Verify`, `Stage`, `ActivateCAS`, `Acquire`, `Release`, `Supersede`, `Retire`, `Revoke`, `BuildCommit`, `Resume`, and `Cancel`. Safety invariants:

```
OneActive == each routing key has at most one active head
LeaseEligible == every lease references a release active at acquisition
NoPrematurePurge == leased releases are not purged
CASLinear == successful activation observes expected predecessor
OneReleasePerBuild == a build registers at most one semantic release
NoActiveQuarantine == active heads are never quarantined
```

Liveness assumptions should be modest and explicit: fair storage, eventual worker retry, and eventual lease closure for nonfaulty clients. Candidate liveness properties include eventual build terminality and eventual retirement of a superseded release after all leases close. Model cancellation and revocation separately because forced termination of in-flight queries is a product policy, not a universal law.

The frontend protocol can use a second small model with snapshot ordinal, delivered event set, dedupe set, entity versions, and append offsets. Its invariant is convergence with the authoritative event prefix or an explicit resync state.

F.14 Proof targets

Formal proof effort should focus on kernels with high reuse and small state.

1. **Total ranking.** Prove the comparator yields a total order over all finite valid scores and stable IDs.
2. **Incremental algebra.** Prove local delta rules for deterministic document-local derivations; use full-build differential testing for provider-backed stages.
3. **Overlay semantics.** Prove lookup/search membership of base plus ordered deltas with tombstones equals the integrated logical multiset, before approximation.
4. **Authorization noninterference.** Prove that no remote-text action is enabled without a valid authorization certificate for the candidate set.
5. **Activation linearizability.** Prove or model-check the CAS head and lease-acquisition protocol.
6. **Reducer convergence.** Prove idempotence and stale-update rejection for replace/merge/tombstone events and exact-once offset behavior for append events.
7. **Gate monotonicity.** Prove that adding a failed earlier hard gate cannot make a candidate eligible through later scores.

Provider quality, natural-language correctness, and empirical latency are not suitable proof targets. They require measurement under retained uncertainty.

F.15 CI and release verification tiers

A workable pipeline separates fast deterministic checks from expensive campaigns.

- **Per commit:** unit, property, fuzz corpus, manifest/schema, pure reducer, dependency-closure, and small differential fixtures.
- **Per merge:** backend conformance, cross-language reducer, state-machine randomized tests, provider-spy security tests, and representative retrieval cells.

- **Nightly:** larger fuzzing, build crash matrix, ANN oracle suite, refresh simulation, session calibration subset, and moderate load.
- **Candidate release:** complete paired evaluation, security attestation, full refresh/compaction simulation, load envelope, and reproducible promotion report.
- **Canary:** online SLO/gate monitor with automatic stop/rollback authority for hard constraints.
- **Periodic:** full rebuild audit against maintained state, disaster-recovery exercise, retention/purge audit, and model-check update when protocol changes.

A test result is itself an immutable artifact referenced by the release or promotion report. “Tests passed” without evaluator identity, input snapshot, seed, and artifact digest is not sufficient evidence.

Appendix G. Empirical source map, limitations, and current-to-target mapping

G.1 Scope of the supplied snapshot

The review covers five development scopes. Counts are static measurements of the supplied archive and are included to characterize the evidence base, not to compare team productivity.

Scope	Files	Go files	Nonblank Go lines	Go test functions
`ragkit`	176	173	17,743	273
`ragopt`	120	45	5,925	42
RAG-TTC	1,302	515	76,705	905
GEC RAG	1,114	200	28,668	252
TTC Garden	940	70	8,485	108
Total	**3,652**	**1,003**	**137,526**	**1,580**

The RAG-TTC `pkg` tree substantially overlaps `ragkit`: 165 matching relative Go paths were found, with 50 byte-identical files and 114 pairs at token-set Jaccard similarity of at least 0.95. This supports a hard shared-package migration rather than continued synchronization of copied substrates. The overlap statistic does not imply all same-path files are semantically interchangeable; product-specific forks require fixture-backed review.

G.2 `ragkit` source map

Source area	Evidence used in this volume
`rag/answering/service.go`	channel execution, fusion/rerank flow, fallback and observation behavior
`rag/answering/context.go`	whole-chunk context admission, ordering, count/rune budgets
`rag/answering/contract.go`	grounded answer schema, citation and supplied-evidence validation
`rag/indexbundle/build.go`	immutable full-bundle construction and atomic publication
`rag/indexbundle/open.go`	manifest, backend, and query-embedder compatibility verification
`rag/indexbundle/verified_documents.go`	source-root confinement and corpus digest checks
`rag/flow`	stage-local caching, retries, resource admission, and budgets
document/chunk/representation packages	deterministic identity and derivation substrate

`ragkit` is therefore already a substantial batch RAG library. The target architecture extends it into corpus, release, index-view, interpreter, trace, and stream semantics; it does not replace its current deterministic kernels.

G.3 RAG-TTC source map

Source area	Evidence used in this volume
`cmd/rag-ttc/cmds/indexes/build.go`	applied complete-corpus representation/embedding build

Source area	Evidence used in this volume
`cmd/rag-ttc/cmds/indexes/ann_bakeoff.go`	exact-oracle HNSW quality/latency gate and rebuild check
`cmd/rag-ttc/cmds/workspace/index.go`	committed source snapshot to workspace artifacts
`pkg/gochunk/snapshot.go`	Git-tree capture, tracked-file admission, snapshot digest
`pkg/ttcrag/search.go`	model-invoked retrieval routes and turn evidence ledger
`pkg/app/chat/controller.go`	active-turn custody, cancellation, observation collection
`pkg/app/chatserver`	persistent submissions, timelines, WebSocket streaming, runtime composition

RAG-TTC provides the richest applied bridge between index construction and agentic serving. Its principal architectural liability is the copied common substrate and the absence of native corpus reconciliation and release activation.

G.4 GEC source map

Source area	Evidence used in this volume
`internal/knowledge/service.go`	startup bundle, query-time synonyms/reranker, post-ranking scope filtering
`internal/knowledge/sweep.go`	offline fusion/vector-weight grid sweep
`internal/knowledge/tool.go`	server-controlled scopes and run-scoped evidence labels
`web/src/ws/wsManager.ts`	snapshot hydration, pre-hydration buffering, ordering, truncation
`web/src/store/timelineSlice.ts`	entity upserts and append/replace stream patches

The imported `internal/knowledgebuild` package is absent from the supplied snapshot. Design records and call sites describe its role, but its implementation was not directly inspected. Assertions about exact GEC build internals are therefore intentionally limited. The query and frontend findings are based on present source.

G.5 Garden source map

Source area	Evidence used in this volume
`backend/internal/ragsearch/ragsearch.go`	intent routes and per-conversation session resources
`backend/internal/ragsearch/searchtool.go`	structured-first facts, routed retrieval, and observations
`backend/internal/ragsearch/grounded_widgets.go`	evidence-bound typed frontend projections
`backend/internal/calibration/runner.go`	multi-turn calibration and stable terminal polling

Garden is the clearest evidence that production RAG semantics extend beyond ranked text and answer strings. Structured facts, typed widgets, conversation state, and field-level provenance are user-visible semantics and belong in release/evaluation scope even though their domain meaning remains in the Garden application.

G.6 `ragopt` source map

`ragopt` was reviewed as a whole because its responsibilities cut across packages: candidate construction, exact baseline/candidate pairing, resumable run custody, comparison, ordered gates, and reporting. No

native corpus revision, index build, query trace, release activation, or frontend projection model was found. This is a deliberate boundary, not a deficiency. The proposed `ragopt/ragspace` adapter supplies RAG-specific intervention spaces and evaluators while preserving the generic experiment kernel.

G.7 High-priority findings

G.7.1 P0: authorization before remote disclosure

GEC's observed ordering allows candidate hydration and optional remote reranking before final source-scope filtering. A candidate that will later be removed can therefore cross a provider boundary. The target order is policy-constrained candidate generation or a local authorization filter before any remote text-bearing stage, with an auditable certificate carried through the trace.

G.7.2 P0: behavior-complete release identity

GEC's reranker and synonyms are query-time configuration outside the opened bundle identity. Garden composes an index bundle, structured fact database, tool policy, prompts, and projections without one material release root. The target release manifest binds every input that can change observable behavior or disclosure.

G.7.3 P0: atomic activation and pinning

The applied services primarily open fixed resources at startup. There is no shared hot-stage, compare-and-swap activation, draining, rollback, or lease protocol. The target registry gives each query/turn/session one release epoch and permits safe replacement without mixed evidence.

G.7.4 P1: corpus-change semantics

The shared builder consumes complete corpus material and emits immutable bundles. RAG-TTC adds strong Git snapshot capture and caches, but not revision reconciliation. The target introduces source changes, barriers, cursors, impact plans, delta overlays, compaction, and full-rebuild equivalence.

G.7.5 P1: query-mode separation

Direct search, retrieve-generate, and agentic search are implemented through related primitives but have different state and terminal semantics. The target exposes separate interpreters over one retrieval plan/algebra.

G.7.6 P1: frontend replay laws

GEC has a practical hydration buffer, but stale entity versions and duplicate append patches are not rejected by a complete shared law. The target uses versioned event envelopes, event-ID deduplication, entity versions, exact append offsets, and resynchronization.

G.7.7 P1: joint optimization

Current applied experiments are valuable but narrow: fusion-weight sweeps and ANN parameter bakeoffs. The target uses typed intervention spaces, dependency closure, temporal holdouts, paired statistics, multiple fidelities, and constraint-first Pareto gates.

G.7.8 P2: durable build custody

`ragkit/flow` provides stage execution controls, but no durable cross-process production build state machine. The target adds append-only build events, resumable work items, leases, verification, quarantine, cancellation, and release registration.

G.8 Current-to-target package mapping

Current capability	Current owner	Target owner	Migration relation
documents, chunks, representations	`ragkit` and copy	`ragkit/corpus`, `ragkit/derive`	retain and generalize
full immutable bundle	`ragkit/indexbundle`	`ragkit/index` plus `ragkit/release`	wrap, then extend
stage cache/retry/budget	`ragkit/flow`	`ragkit/flow` used by build/query runtimes	retain; do not overpromote
full RAG-TTC index command	RAG-TTC	product connector + `ragkit/build`	refactor orchestration
Git committed snapshot	RAG-TTC `gochunk`	connector implementation of `ragkit/corpus`	extract interface, retain policy
exact/ANN bakeoff	RAG-TTC command	`ragkit/eval` + `ragopt/ragspace`	turn into reusable campaign
GEC query service	GEC	GEC facade over `ragkit/query`	preserve product policy
GEC scopes/roles	GEC	GEC policy adapter + shared authorization certificate	retain domain meaning
GEC synonyms/reranker	GEC config	behavior-complete release query policy	materialize and identify
turn evidence ledger	GEC/RAG-TTC	`ragkit/evidence` with scoped adapters	share laws, retain presentation
Garden structured facts	Garden	Garden evidence adapter	retain in product
grounded widgets	Garden	Garden projection over `ragkit/stream`	retain product semantics
chat timelines/WebSocket	applied apps	product server over shared event envelope/reducer law	share protocol, not UI domain
experiment run/gates	`ragopt`	`ragopt`	retain generic kernel
RAG candidate space	ad hoc commands	`ragopt/ragspace`	add thin domain adapter

G.9 Proposed repository/package shape

A practical target layout is:

```
ragkit/
  corpus/          revision, change, connector, cursor, barrier, snapshot
  derive/          impact graph, deterministic stages, lineage, cache keys
  index/           logical view, exact/ANN capabilities, overlay, compaction
  release/         behavior-complete spec, registry, activation, leases
  query/           plans, stages, direct/answer/agent interpreters
  evidence/        typed evidence, admission, scoped sessions, provenance
  trace/           versioned intensional and operational observations
  stream/          event envelope, snapshot, reducer laws, replay cursors
  eval/            RAG workloads, metrics, refresh and session harnesses
  flow/            bounded local execution primitives

ragopt/
  ...              existing generic experiment kernel
  ragspace/         interventions, dependency closure, fidelities, gates

products/
  gec/              authorization, source roles, admin tools, judges, UI policy
  rag-ttc/          TTC sources, connected retrieval, agent tools, providers
  garden/           intent, structured facts, catalog semantics, widgets
```

`ragkit/build` may be introduced only when at least two products share the same durable build-state semantics. Until then, the transition types and event laws can live in `corpus`, `derive`, `index`, and `release`, while each product composes them with its existing workflow/runtime infrastructure. This avoids creating an orchestration framework before operational commonality is demonstrated.

G.10 Product migration acceptance map

G.10.1 GEC

- current authorized outputs captured as differential fixtures;
- authorization enforced before all remote text disclosure;
- synonyms, reranker, prompts, and policy bound into release identity;
- query/turn pinned to a release lease;
- full refresh and activation available before incremental refresh;
- frontend reducer satisfies duplicate/stale/append laws;
- fusion/rerank campaign uses protected role/scope strata.

G.10.2 RAG-TTC

- copied common packages deleted after compatibility classification;
- committed Git capture implements shared source revision/snapshot interface;
- index command emits a behavior-complete staged release;
- direct search, answer, and agent tool use separate interpreters/traces;
- HNSW backend passes common capability and dynamic-state certification;
- chat server acquires/releases one release per turn;
- optimization commands become reproducible `ragopt/ragspace` campaigns.

G.10.3 Garden

- all transitive copied retrieval substrate replaced by shared packages;
- structured fact snapshot and presentation policy included in Garden release material;
- conversation evidence reuse has explicit release-epoch rules;
- grounded widget fields carry evidence and release provenance;
- calibration runner records complete session traces and paired candidate cells;
- browser/server projection reducers share fixtures and convergence laws.

G.11 Analysis limitations

1. **Static snapshot.** The archive is a development snapshot, not necessarily the deployed system or latest branch.
2. **Missing GEC build source.** `internal/knowledgebuild` is referenced but absent; detailed build claims rely only on call sites and design material.
3. **Toolchain mismatch.** The repositories require Go 1.26.x; the environment provides Go 1.23.2 and cannot download a newer toolchain. The snapshot was not compiled or executed.
4. **No production telemetry.** Latency, cost, failure, traffic, corpus-change rate, and user-outcome claims are design requirements, not measurements of a live deployment.
5. **No provider experiment.** Remote model behavior was not benchmarked. Stochastic semantics and gates are proposed from interfaces and runtime paths.
6. **No security penetration test.** The disclosure issue is a source-ordering finding, not a claim that a specific provider received unauthorized production content.
7. **Code-count limitations.** File/line/test counts are descriptive and depend on the extracted archive and simple static counting rules.

8. **Formalization boundary.** The operational rules abstract storage, scheduler, network, and model internals. Implementations must refine them and state any additional failure modes.

These limitations do not weaken the central architectural conclusion: the supplied implementations already exhibit corpus capture, immutable indexes, retrieval pipelines, agentic tool use, persistent streaming, and optimization experiments, but the shared packages do not yet model their coupled runtime semantics as one production RAG domain.

Appendix H. Selected bibliography

This bibliography emphasizes primary sources that motivate the formal and empirical structures used in the volume. It is not a survey of every RAG framework or vector database.

H.1 Retrieval-augmented generation and retrieval

Lewis, Patrick, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, et al. 2020. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks." *Advances in Neural Information Processing Systems* 33.

Karpukhin, Vladimir, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. "Dense Passage Retrieval for Open-Domain Question Answering." In *Proceedings of EMNLP 2020*.

Khattab, Omar, and Matei Zaharia. 2020. "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT." In *Proceedings of SIGIR 2020*.

Thakur, Nandan, Nils Reimers, Andreas Rucklé, Abhishek Srivastava, and Iryna Gurevych. 2021. "BEIR: A Heterogeneous Benchmark for Zero-Shot Evaluation of Information Retrieval Models." In *NeurIPS Datasets and Benchmarks*.

Petroni, Fabio, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, et al. 2021. "KILT: A Benchmark for Knowledge Intensive Language Tasks." In *Proceedings of NAACL-HLT 2021*.

Robertson, Stephen, and Hugo Zaragoza. 2009. "The Probabilistic Relevance Framework: BM25 and Beyond." *Foundations and Trends in Information Retrieval* 3 (4): 333-389.

Cormack, Gordon V., Charles L. A. Clarke, and Stefan Buettcher. 2009. "Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods." In *Proceedings of SIGIR 2009*.

Järvelin, Kalervo, and Jaana Kekäläinen. 2002. "Cumulated Gain-Based Evaluation of IR Techniques." *ACM Transactions on Information Systems* 20 (4): 422-446.

Malkov, Yu. A., and D. A. Yashunin. 2020. "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs." *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (4): 824-836.

H.2 RAG evaluation

Es, Shahul, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. "RAGAs: Automated Evaluation of Retrieval Augmented Generation." In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, 150-158.

Saad-Falcon, Jon, Omar Khattab, Christopher Potts, and Matei Zaharia. 2024. "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems." In *Proceedings of NAACL-HLT 2024*, 338-354.

The evaluation design in this volume uses these works as evidence for decomposed RAG measurements, while adding release lineage, authorization, freshness, failure, agent trajectories, and frontend projection as first-class dimensions.

H.3 Denotational, operational, and effectful semantics

Scott, Dana, and Christopher Strachey. 1971. “Toward a Mathematical Semantics for Computer Languages.” Programming Research Group Technical Monograph PRG-6, Oxford University Computing Laboratory.

Kahn, Gilles. 1974. “The Semantics of a Simple Language for Parallel Programming.” In *Information Processing 74: Proceedings of IFIP Congress 74*.

Plotkin, Gordon D. 1981. “A Structural Approach to Operational Semantics.” DAIMI FN-19, Aarhus University. Reprinted with revisions in *The Journal of Logic and Algebraic Programming* 60-61 (2004): 17-139.

Moggi, Eugenio. 1991. “Notions of Computation and Monads.” *Information and Computation* 93 (1): 55-92.

Fritz, Tobias. 2020. “A Synthetic Approach to Markov Kernels, Conditional Independence and theorems on Sufficient Statistics.” *Advances in Mathematics* 370: 107239.

These sources motivate the distinction between extensional denotations, small-step labelled transitions, stream/process meanings, and effectful or probabilistic composition. The APIs proposed here use ordinary Go types rather than exposing the mathematical machinery directly.

H.4 Incremental computation, replicated state, and concurrency

Budiu, Mihai, Tej Chajed, Frank McSherry, Leonid Ryzhyk, and Val Tannen. 2023. “DBSP: Automatic Incremental View Maintenance for Rich Query Languages.” *Proceedings of the VLDB Endowment* 16 (7): 1601-1614.

Shapiro, Marc, Nuno Preguiça, Carlos Baquero, and Marek Zawirski. 2011. “Conflict-Free Replicated Data Types.” In *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*.

Lamport, Leslie. 1994. “The Temporal Logic of Actions.” *ACM Transactions on Programming Languages and Systems* 16 (3): 872-923.

Lamport, Leslie. 2002. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley.

DBSP informs the full-versus-incremental equivalence law and signed-change perspective. CRDT work informs the analysis of duplicate/out-of-order frontend events, while also clarifying why unrestricted text append is not automatically a convergent replicated datatype. TLA/TLA+ motivates protocol-level safety and liveness models for activation, leasing, and replay.

H.5 Property-based verification

Claessen, Koen, and John Hughes. 2000. “QuickCheck: A Lightweight Tool for Random Testing of Haskell Programs.” In *Proceedings of ICFP 2000*.

Property-based testing is used throughout this volume for identities, ranking orders, incremental/full equivalence, state-machine traces, and frontend convergence. The method complements, rather than replaces, product fixtures and stochastic evaluation.

H.6 Empirical system under study

Supplied repository snapshot. 2026. `ragkit`, `ragopt`, RAG-TTC, GEC RAG Chat, TTC Garden Assistant, associated tests, design records, and frontend code, provided for this architecture study. All

implementation-specific findings in the volume derive from that snapshot subject to the limitations in Appendix G.

Appendix I. Glossary of operational RAG terms

Activated release. The immutable behavior-complete release currently selected by a routing key for new leases.

Activation. A compare-and-swap transition that changes release resolution. It is distinct from building, publishing, staging, and warming.

Agent interpreter. A query interpreter whose execution includes model decisions and zero or more tool calls before a terminal projection.

Authorization certificate. An auditable value proving which subject, policy revision, release, candidate set, provider, and purpose authorize a disclosure or stage transition.

Barrier. A source event asserting that a finite prefix or snapshot frontier is complete enough to build and identify.

Base-plus-delta index. A logical searchable view composed from an immutable base index, one or more change layers, and tombstones, later compacted into a new base.

Behavior-complete release. An immutable manifest over all inputs capable of changing observable RAG behavior: corpus, derivations, indexes, query and answer policy, providers, structured stores, and presentation.

Build intent. A durable request to derive a candidate release from a source frontier and release specification.

Candidate. A proposed release or release patch evaluated against a baseline under an explicit intervention declaration.

Change. A typed source revision event such as upsert, delete, or barrier. Delivery may be duplicated; semantic reduction must be idempotent.

Collapse. Mapping multiple searchable representations or duplicate candidates to one logical evidence identity under a defined score/contribution rule.

Compaction. A semantics-preserving maintenance operation that integrates delta layers and tombstones into a new base representation.

Conversation epoch. A declared interval in which conversation-scoped evidence belongs to one release. A release change either starts a new epoch or is rejected under the product policy.

Corpus snapshot. A finite, identified source state at a barrier, including admission policy and source lineage.

Denotational semantics. The mathematical meaning of a RAG release/service as a mapping from subject, state, and request to outcomes and traces, abstracting from particular execution steps.

Direct interpreter. A query interpreter that terminates with ranked evidence and trace rather than generating an answer.

Disclosure. Transmission of source-derived content or metadata across a trust boundary, including to embedding, reranking, generation, logging, or telemetry systems.

Evidence. Authoritative or explicitly typed material admitted for use in an answer or presentation. A searchable representation is not automatically evidence.

Evidence session. Scoped mutable custody of admitted evidence and stable labels over an operation, turn, run, or conversation epoch.

Extensional outcome. The externally observable result class and content, such as ranked evidence, answer, abstention, failure, or presentation state.

Fallback. A declared alternate transition after stage failure or deadline, retained in the trace because it can change outcomes and reliability semantics.

Freshness. The relation between source event time/frontier and the release serving a query. It is not synonymous with build recency.

Full-rebuild oracle. A clean derivation from the complete source snapshot used to validate incremental maintenance.

Gate. A typed decision predicate over experiment evidence. Hard gates are evaluated before preference comparisons.

Impact plan. The dependency-closure computation that identifies which derived items and evaluations a source change or intervention invalidates.

Incremental view maintenance. Updating derived state from source changes while preserving equivalence to recomputation from the integrated source state.

Intensional trace. The lineage, decisions, disclosures, fallbacks, latency, cost, and iteration history by which an outcome was produced.

Interpreter. A runtime that executes a typed query plan under a release lease. Direct, answer, and agent interpreters have different terminal rules.

Lease. A reference to one release acquired for a query, turn, or session epoch; it prevents mixed-release execution and premature resource retirement.

Logical index view. The abstract searchable relation seen by query code, independent of whether its physical representation is exact, ANN, sharded, or base-plus-delta.

Noninferiority. A statistical decision that a candidate is not worse than a baseline by more than a declared margin on a protected metric or stratum.

Observation. A versioned, typed trace event emitted by build, query, activation, or projection machines.

Operational semantics. The labelled transition rules by which runtime states evolve through source changes, builds, searches, provider calls, streams, failures, and activation.

Overlay. A physical or logical delta layer queried together with a base index before compaction.

Pinned query/turn. An execution whose every stage, evidence item, structured fact, citation, and event belongs to one release lease.

Presentation interpreter. Product-owned logic that projects evidence and answer state into typed frontend entities and events.

Projection reducer. A deterministic state reducer that reconstructs frontend state from a snapshot and event suffix under deduplication and version rules.

Query algebra. The ordered composition of rewrite, channels, collapse, fusion, policy, rerank, hydrate, evidence admission, generation, validation, and projection operators.

Release registry. Durable custody of registered releases, eligibility state, active routing heads, activation history, and retirement state.

Representation. Searchable derived material linked to source evidence, such as contextual text, summary, question, or entity text. It aids retrieval but carries no independent authority by default.

Retrieve-then-generate interpreter. A query interpreter that performs a bounded retrieval plan once, constructs context, generates, validates, and emits a terminal answer.

Revision. A source-system identity for a particular state of an object or policy. Revision semantics are connector-specific but must support ordering or conflict rules.

Routing key. The tenant, product, environment, corpus, or cohort key whose active release is changed atomically.

Semantic class. The declared kind of change made by an intervention: operational, approximation, relevance, knowledge, policy/security, interaction, or presentation.

Snapshot-plus-suffix law. The requirement that hydrating a frontend snapshot and reducing all later valid events yields the same state as the authoritative event prefix, despite permitted duplicates and reordering.

Source frontier. The connector cursor, revisions, barriers, and policy state delimiting the source world captured by a build.

Staged release. A verified, loadable release prepared for activation but not yet selected for new queries.

Subject. The authenticated and authorized principal, including tenant, roles, scopes, purpose, and relevant policy context.

Tombstone. A durable negative change that suppresses a deleted logical item in overlays and future releases until correctly compacted or recreated.

Trace equivalence. Equality or an explicit relation over intensional behavior, stronger than final-answer equality and parameterized by what the comparison is intended to preserve.

Watermark. A measured source or build frontier used to quantify freshness and progress; it may represent event time, revision position, capture completion, or activation.